Prácticas de PSyD

Daniel Báscones García

26 de octubre de 2026

Índice

1. Configuración del entorno de programación **4**
1.1. Configuración para máquina virtual 4
1.2. Creación de la Máquina Virtual 4
1.3. Configuración del ToolKit de STM 5
1.4. Descarga de manuales y recursos extra 5
1.5. Descarga del material de la asignatura 7
1.6. Conexión de la placa a la máquina virtual 8
1.7. Uso en los laboratorios de la facultad 8

2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO) **11**
2.1. Objetivos 11
2.2. Fundamentos teóricos 11
2.3. Desarrollo de la práctica 14
2.3.1. Creación del proyecto 14
2.3.2. Depuración del proyecto 17
2.3.3. Desarrollo del proyecto 20
2.4. Para hacer más 21

3. Práctica 2: Creación de un *Board Support Package* (BSP) **22**
3.1. Objetivos 22
3.2. Fundamentos teóricos 22
3.2.1. Definiciones de direcciones 22
3.2.2. Definiciones de registros 23
3.2.3. Gestión de registros 24
3.2.4. Funciones de control de registros 24
3.2.5. Gestión directa bit a bit de registros 25
3.3. Desarrollo de la práctica 26
3.4. Para hacer más 27

4. Práctica 3: Entrada / Salida programada - UART TX **28**
4.1. Objetivos 28
4.2. Fundamentos teóricos 28
4.2.1. UART 28
4.2.2. UART en el **STM32F723IE** 29
4.3. Desarrollo de la práctica 29
4.3.1. Envío de cadenas de texto por UART 31
4.4. Para hacer más 32

5. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones **33**
5.1. Objetivos . . . 33
5.2. Fundamentos teóricos . . . 33
5.2.1. Configuración de las interrupciones en el procesador . . . 33
5.2.2. Gestión de interrupciones mediante funciones . . . 34
5.3. Desarrollo de la práctica . . . 34
5.4. Para hacer más . . . 38

6. Práctica 5: Configuración de Timers **39**
6.1. Objetivos . . . 39
6.2. Fundamentos teóricos . . . 39
6.3. Desarrollo de la práctica . . . 40
6.4. Para hacer más . . . 43

7. Práctica 6: Medida del tiempo de programa **44**
7.1. Objetivos . . . 44
7.2. Fundamentos teóricos . . . 44
7.3. Desarrollo de la práctica . . . 44
7.4. Para hacer más . . . 46

8. Práctica 7: Conexión con periféricos externos - Display y drivers **47**
8.1. Objetivos . . . 47
8.2. Fundamentos teóricos . . . 47
8.2.1. Funcionamiento del periférico FMC . . . 49
8.2.2. Funcionamiento del controlador ST7789H2 . . . 50
8.3. Desarrollo de la práctica . . . 51
8.3.1. Creación de la Librería Gráfica (GFX Library) . . . 53
8.4. Para hacer más . . . 55

9. Práctica 8: Buses de Expansión - I2C y pantalla táctil **56**
9.1. Objetivos . . . 56
9.2. Fundamentos teóricos . . . 56
9.2.1. Conexiones en nuestra placa . . . 56
9.2.2. Funcionamiento del periférico I2C . . . 57
9.3. Desarrollo de la práctica . . . 57
9.3.1. Configuración del bus I2C . . . 58
9.3.2. Interrupciones del CTP . . . 59
9.3.3. Driver del I2C . . . 60
9.3.4. Driver del CTP . . . 62
9.3.5. Integración . . . 64
9.4. Para hacer más . . . 64

10. Práctica 9: Persistencia - Memoria FLASH **65**
10.1. Objetivos . . . 65
10.2. Fundamentos teóricos . . . 65
10.2.1. Organización de la flash . . . 65
10.3. Desarrollo de la práctica . . . 65
10.3.1. Bucle de control principal . . . 65
10.3.2. Trabajando con la FLASH . . . 66
10.3.3. Creando las estructuras para la persistencia . . . 68
10.4. Para hacer más . . . 70

11. Práctica 10: Multitarea Cooperativa **71**

11.1. Objetivos . . . 71
11.2. Fundamentos teóricos . . . 71
11.3. Desarrollo de la práctica . . . 71
11.3.1. Fragilidad de la aproximación . . . 75
11.4. Para hacer más . . . 76

12.Práctica 11: Comunicación remota con *ThingsBoard* 77
12.1. Objetivos . . . 77
12.2. Fundamentos teóricos . . . 77
12.2.1. El módulo Wifi ESP-01S . . . 78
12.3. Desarrollo de la práctica . . . 78
12.3.1. Creando cuenta y dispositivo en *ThingsBoard* . . . 78
12.3.2. Programando nuestro dispositivo para conectar . . . 79
12.3.3. Visualización en *ThingsBoard* . . . 82
12.3.4. Recibiendo comandos desde *ThingsBoard* . . . 85
12.4. Para hacer más . . . 86

13.Práctica 12: Actualización remota (OTA) utilizando *ThingsBoard* 87
13.1. Objetivos . . . 87
13.2. Fundamentos teóricos . . . 87
13.2.1. El problema de la compilación . . . 87
13.3. Desarrollo de la práctica . . . 89
13.3.1. Preparación del código . . . 89
13.3.2. Preparación de la compilación . . . 92
13.3.3. Configuración en *Thingsboard* . . . 96
13.4. Para hacer más . . . 99

14.Práctica 13: Creación automática de BSP 100
14.1. Objetivos . . . 100
14.2. Fundamentos teóricos . . . 100
14.3. Desarrollo de la práctica . . . 100
14.3.1. Creación del BSP con STMCubeMX . . . 100
14.3.2. Uso del BSP desde STMCubeIDE . . . 106
14.4. Para hacer más . . . 111

15.Práctica 14: Proyecto final - Práctica Libre 112
15.1. Objetivos . . . 112
15.2. Requisitos de la práctica . . . 112
15.3. Ideas de desarrollo para la práctica final . . . 112
15.4. Ejemplo de documento final explicativo sobre la práctica . . . 114
15.4.1. Manual de Usuario . . . 114
15.4.2. Arquitectura del sistema . . . 114
15.4.3. Diagrama de Flujo . . . 115

1. Configuración del entorno de programación

Las prácticas de la asignatura de PSyD se desarrollan en un entorno ya preparado dentro de una máquina virtual. Esto es debido a la comodidad y portabilidad que ofrece, ya que con trasladar un simple fichero, cualquiera puede tener todas las herramientas disponibles en su ordenador. No obstante, también se ofrece aquí una guía de instalación para quien quiera instalarlo todo de manera nativa en su ordenador. La guía se ha realizado para Ubuntu. Su adaptación a otros sistemas operativos (MacOS, o Windows) es relativamente directa, al estar las herramientas disponibles para todas las plataformas.

1.1. Configuración para máquina virtual

El software y los recursos necesarios ya están preinstalados en una máquina virtual basada en **Debian 13.6**. Cuenta con un entorno ligero (gestor de ventanas, navegador básico) y las **VirtualBox Guest Additions** para habilitar el passthrough de dispositivos USB (necesario para conectar la placa).

Puedes descargar la máquina virtual ya configurada y lista para importar en VirtualBox desde `https://drive.google.com/file/d/1U7Qvnkb-LfkJrcboyH4Jzs9iNczMntQg/view?usp=drivesdk`. También es necesario descargar **VirtualBox** para arrancarla. Para ello sigue las siguientes instrucciones:

- Asegúrate de tener instalado **VirtualBox** en su versión más reciente. Puedes descargarlo desde `https://www.virtualbox.org/wiki/Downloads`.
- Importante también descargar, desde el mismo enlace, el "VirtualBox Extension Pack". Sin él, perderemos funcionalidades de la MV, entre otros, la capacidad de conectarle USBs (como el depurador ST-Link de la placa).

1.2. Creación de la Máquina Virtual

Sigue esta parte sólo si quieres crear una máquina virtual propia desde cero. Si quieres hacerlo de manera nativa, salta a Apartado 1.3.

Crea una nueva máquina virtual e instala **Debian** en su última versión (`https://www.debian.org/index.es.html`) o **Ubuntu** (recomendado versión **24.04 LTS**, descargable desde `https://ubuntu.com/download`). Configura el hardware asignándole los siguientes recursos recomendados:

- **Memoria RAM:** 4096 MB
- **Procesadores:** 2 núcleos (Cores).
- **Memoria de Vídeo:** 128 MB.
- **Tamaño del disco:** 60GB.
- **Tipo de instalación:** Manual. No utilizar la automática ya que instala cosas de más.

Una vez seleccionados los recursos, lanza la máquina y el instalador comenzará automáticamente. Deberás seleccionar cuestiones como el nombre de la máquina, usuario, contraseña... elige el que más te guste. El resto de la configuración se puede dejar por defecto. Ten en cuenta que el proceso de instalación puede tardar una una media hora (dependiendo del sistema anfitrión). Se recomienda utilizar las instalaciones gráficas para tener todo el sistema operativo bien preconfigurado. No obstante, también se pueden hacer instalaciones manuales para ajustar mejor las herramientas disponibles y, con ello, el tamaño de la máquina.

Una vez arranque la máquina, es importante instalar los extras de invitado ("Guest additions"), para conseguir que anfitrión y host se comunique adecuadamente (y puedan por ejemplo compartir portapapeles, ajustar el tamaño de pantalla, etc). Para ello haz click en `devices >insert guest additions CD image`. Esto cargará un disco en la MV (panel izquierdo). Abre una terminal en la ruta donde se monta el CD (normalmente /media/cdrom) y ejecuta:

```
# Actualizar paquetes existentes
> sudo apt update && sudo apt upgrade
# Instalar herramientas para compilar las guest additions
> sudo apt install build-essential dkms linux-headers-$
# Instalar guest additions
> sudo ./VBoxLinuxAdditions.run
```

1.3. Configuración del ToolKit de STM

Independientemente de si quieres utilizar tu propio sistema operativo para instalar las herramientas, o instalarlas en la máquina virtual, se hace de la misma manera. A continuación se muestran todas las herramientas, repositorios, etc necesarios para tener todo lo necesario en la asignatura. Si prefieres no realizar la instalación, descarga la máquina virtual ya configurada como se indica en el primer paso (Apartado 1.1).

- Instalar STM32CubeIDE (la herramienta principal de desarrollo) desde aquí: `https://www.st.com/en/development-tools/stm32cubeide.html`
- Instalar STM32CubeMX (para configuración avanzada) desde aquí: `https://www.st.com/en/development-tools/stm32cubemx.html`
- Instalar STM32CubeProgrammer (para compatibilidad con zephyr) desde aquí: `https://www.st.com/en/development-tools/stm32cubeprog.html`
- Instalar picocom para comunicar con la placa con posterioridad:

```
> sudo apt-get install picocom
```

1.4. Descarga de manuales y recursos extra

Dentro de la MV ya creada, en la carpeta **Documentos** del entorno se encuentran varios recursos en forma de manuales. Si se está haciendo la instalación manual, estos son los recursos que ahí se encuentran y haría falta descargar:

- `rm0431` : `https://www.st.com/resource/en/reference_manual/rm0431-stm32f72xxx-and-stm32f73xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf`

Manual principal de referencia. Detalla toda la funcionalidad de los procesadores de la serie **STM32F723**, incluyendo periféricos, mapa de memoria, registros, modos de uso, etc.

- `um2140` : `https://www.st.com/resource/en/user_manual/um2140-discovery-kit-with-stm32f723ie-mcu-stmicroelectronics.pdf`

  Manual con la descripción de conexiones y *pinout* detallado del kit de desarrollo **32F723EDISCOVERY**.

- `ds11853` : `https://www.st.com/resource/en/datasheet/stm32f722ic.pdf`

  Funcionalidad genérica, especificaciones físicas y características electrónicas del procesador **STM32F723IE**.

- `pm0253` : `https://www.st.com/resource/en/programming_manual/pm0253-stm32f7-series-and-stm32h7-series-cortexm7-processor-programming-manual-stmicroelectronics.pdf`

  Manual de programación de la serie **Cortex-M7**, con la referencia completa del conjunto de instrucciones ensamblador.

- `ST7789H2` : `https://www.crystalfontz.com/controllers/Sitronix/ST7789H2`

  Manual de características del panel LCD `ST7789H2` integrado en la placa.

- `an2606` : `https://www.st.com/resource/en/application_note/cd00167594-stm32-microcontroller-system-memory-boot-mode-stmicroelectronics.pdf`

  Manual sobre las secuencias de inicialización (*bootloading*) en los dispositivos de ST.

- **DB3105**: `https://www.st.com/resource/en/data_brief/32f723ediscovery.pdf`

  Resumen de características del kit de desarrollo **32F723EDISCOVERY**.

Adicionalmente, también se incluye en la MV creada la colección de ejemplos oficiales **STM32CubeF7**.

Nota: Esta colección utiliza librerías de alto nivel (HAL/LL). En la primera parte de las prácticas programaremos "desde cero" (a nivel de registro), por lo que no usaremos estas librerías directamente. Sin embargo, los ejemplos son un buen punto de referencia para explorar las capacidades de la placa.

- **STM32CubeF7 (GitHub)** `https://github.com/STMicroelectronics/STM32Cubef7`

  Colección de ejemplos base. Incluye proyectos para múltiples placas, pero nos centraremos únicamente en la **32F723EDISCOVERY**.

  Al clonar manualmente el repositorio, es imprescindible inicializar los submódulos para incluir todas las librerías requeridas. Mira bien las instrucciones del repositorio.

- **STM32CubeF7 Patch** `https://www.st.com/en/embedded-software/stm32cubef7.html`

  Parche que se debe sobrescribir en el repositorio una vez clonado. Añade librerías propietarias de terceros necesarias para funciones como el control táctil o el procesamiento de audio.

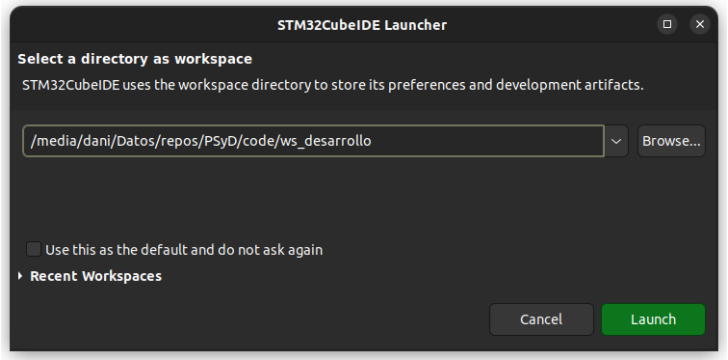


Figura 1: Elección de la carpeta para importar

1.5. Descarga del material de la asignatura

El material de la asignatura se encuentra completo en `https://github.com/Daniel-BG/PSyD`, incluyendo teoría, práctica y códigos de ejemplo. Se recomienda descargarlo entero mediante la herramienta de terminal `git` (o cualquier otra herramienta que permita descargar repositorios). Desde la terminal de la máquina virtual (o la terminal nativa de tu ordenador), ejecuta:

```
> git clone https://github.com/Daniel-BG/PSyD
```

NOTA: Si la herramienta `git` no estuviera disponible, en linux se puede instalar con:

```
>  sudo apt-get install git
```

Se descargará y creará una carpeta llamada `PSyD`, con los siguientes contenidos:

```
code/ ..... – Código fuente de las prácticas
    ws_desarrollo/ ..... – Proyectos SIN solución (punto de partida)
        <nombre_practica>/ ..... – Ficheros de proyecto STM32/CubeIDE
    ws_soluciones/ ..... – Proyectos CON solución (para consulta)
        <nombre_practica>/ ..... – Misma estructura resuelta
practicas/ ..... – Enunciados y documentación
    main.{pdf,tex} ..... – Compilación completa de prácticas
    practicas.md ..... – Información general
    <nombre_practica>/ ..... – Carpeta individual por práctica
        <nombre_practica>.{pdf,tex} ..... – Enunciado en fuente y renderizado
        res/ ..... – Recursos gráficos e imágenes
teoria/ ..... – Material teórico
    main.{pdf,tex} ..... – Libro/Documento completo de teoría
    <capitulo>/ ..... – Capítulo individual
        <capitulo>.{pdf,tex} ..... – Fuente y PDF del capítulo
```

Ahora, abriendo la herramienta STM32CubeIDE, podremos importar los proyectos. En primer lugar debemos establecer como *workspace* la carpeta donde tenemos los códigos. Por ejemplo vamos a hacerlo con `ws_desarrollo`, como se ve en la Figura 1.

A continuación, haremos click en `file >STM32 Project create/import` para abrir la herramienta de importación, seleccionándola desde el menú que se abre (Figura 2).

Finalmente, buscamos de nuevo la carpeta del workspace (que es donde están los proyectos)

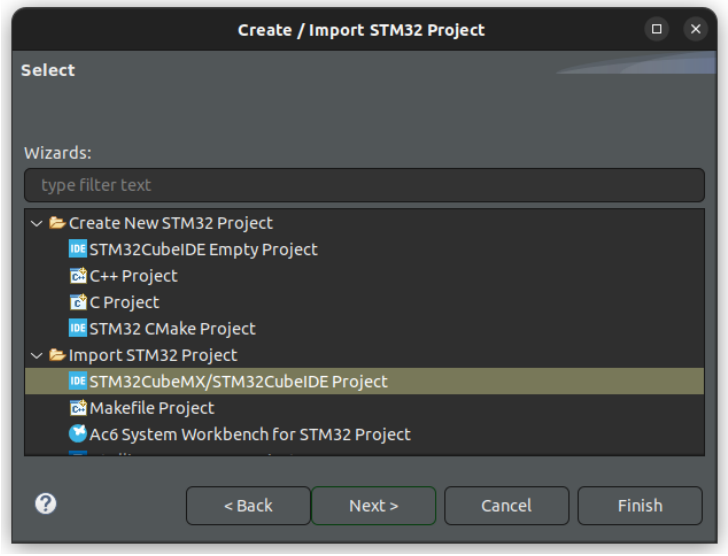


Figura 2: Elegir importación de proyectos

y la elegimos en el menú. Esto nos mostrará los proyectos, y podemos elegir cuáles importar (si alguno no se importa ahora, se puede hacer más tarde). Ver la Figura 3.

Tras importar los proyectos, los deberíamos ver en el panel lateral de la herramienta, ver Figura 4.

1.6. Conexión de la placa a la máquina virtual

Si utilizas tu ordenador con STM32CubeIDE instalado nativamente, la placa debería detectarse automáticamente por el programa. Si usas la máquina virtual, hace falta indicarle al ordenador que pase el control del USB del sistema operativo anfitrión (el vuestro) al huésped (la máquina virtual). Para ello, con la máquina virtual ya abierta, seleccionar en el menú `devices >USB >ST-LINK` y activarlo.

1.7. Uso en los laboratorios de la facultad

En los laboratorios de la facultad, la máquina está instalada en **Linux**, por tanto el primer paso es entrar en ese sistema operativo (si está arrancado en windows, tienes que reiniciar el ordenador, y elegir Linux al arrancar). Una vez dentro, hay varios usuarios, elige el **usuario** normal, NO el de VMs. Ya iniciada la sesión, tendrás que abrir una terminal, y para lanzar la máquina escribir:

```
> VM_PsyD_debian.sh
# si no sale el nombre exactamente igual, escribir solo VM
# en la terminal y pulsar "tab" para ver las opciones disponibles
```

Tras esto, se lanzará la máquina virtual con las herramientas. A continuación puedes abrir otra terminal dentro, y en el directorio de tu elección (Se recomienda la carpeta `Documentos`), descargar el repositorio con el contenido de la asignatura siguiendo el Apartado 1.5. Finalmente, para conectar la placa, sigue el Apartado 1.6.

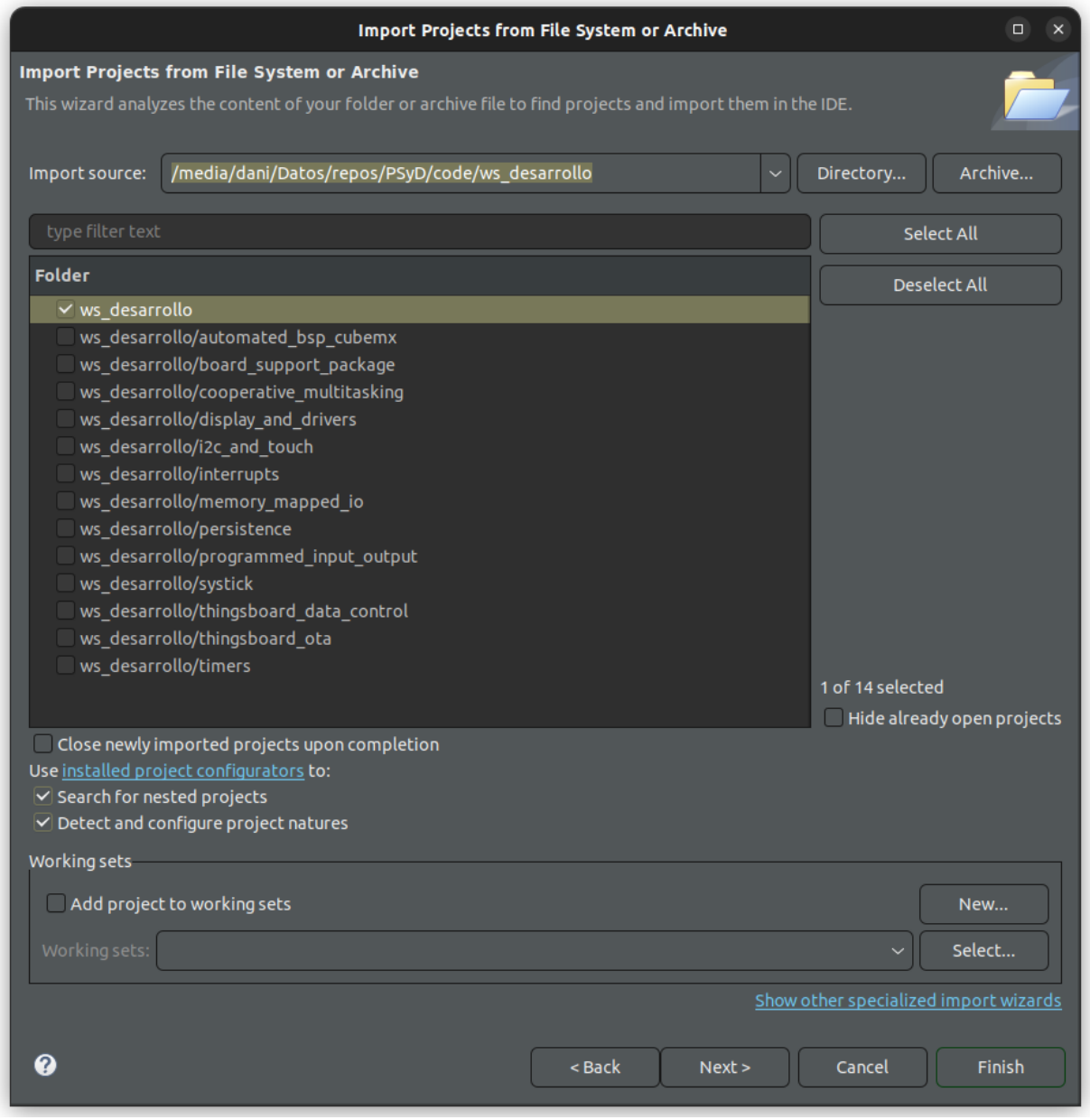


Figura 3: Elegir importación de proyectos

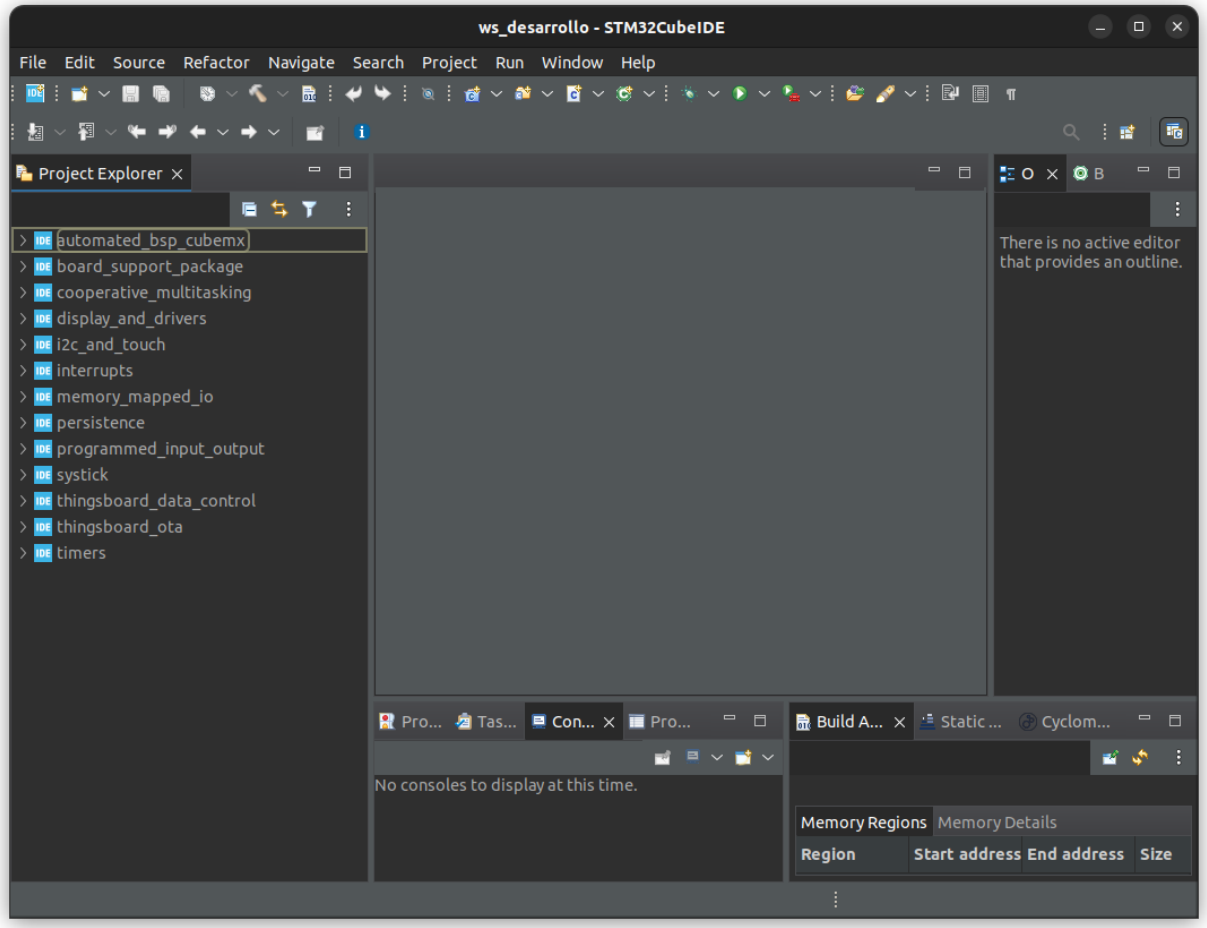


Figura 4: Elegir importación de proyectos

2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)

2.1. Objetivos

- Comprender el concepto de **Memory-Mapped I/O (MMIO)**.
- Analizar la organización del mapa de memoria del microcontrolador **STM32F723IE**.
- Aprender a manipular directamente los registros de entrada/salida mediante máscaras de bits en C.

2.2. Fundamentos teóricos

En un procesador de propósito general, numerosas funciones se realizan a través de *periféricos*, componentes anexos al procesador con funciones específicas.

En la arquitectura ARM, los periféricos no se controlan con instrucciones dedicadas, sino que se comunican con la CPU a través del bus del sistema (*AXI-AHB Bus Matrix*) asignándoles direcciones físicas dentro del mapa de memoria de 32 bits (4GB).

Cada periférico posee un conjunto de **registros de control, estado y datos** ubicados en direcciones consecutivas a partir de una dirección base (*Base Address*). Escribiendo y leyendo en esas direcciones, conseguimos comunicarnos con los periféricos, que a su vez se conectarán con los diferentes pines del procesador. Los pines servirán para su conexión con el exterior con otros elementos, como LEDs, botones, pantallas, puertos, etc.

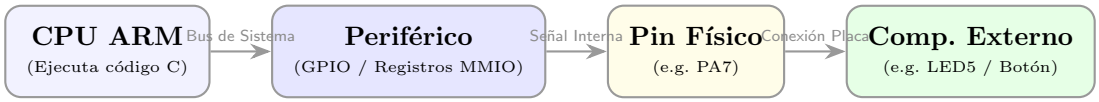


Figura 5: Flujo de control desde el código en CPU hasta el componente exterior a través de MMIO.

En nuestro caso, disponemos de un procesador **STM32F723IE**. Podemos consultar el manual de referencia `rm0431` para ver una lista completa de sus periféricos y las direcciones de memoria donde se ubican sus registros. La sección `rm0431 1.6` detalla todo este mapa de memoria, que se muestra en las Figuras 6 y 7

De aquí ya podemos deducir las direcciones de los diferentes controladores de GPIO. Vemos que están nombrados con letras (desde GPIOA hasta GPIOI). Por ejemplo, el controlador de GPIOA está en la dirección `0x40020000`. ¿Pero claro, cómo lo manejamos? Para ello debemos conocer todos los registros que se encuentran en esa zona de memoria. Esa información la tenemos en `rm0431 6.4`.

A modo de ejemplo, se muestra en la Figura 8 la información referente al registro `MODER` del controlador de GPIO. Observamos que, en una palabra de 32 bits, contiene la configuración de modo de pin (Input/Output/Función alternativa/Analógico) para hasta 16 pines diferentes. Es decir, utiliza 2 bits para la configuración de cada pin. Los dos bits menos significativos corresponden al pin 0, y los 2 más significativos al pin 15, con el resto entre medias. Escribiendo en este registro configuraremos el modo de cada uno de estos pines.

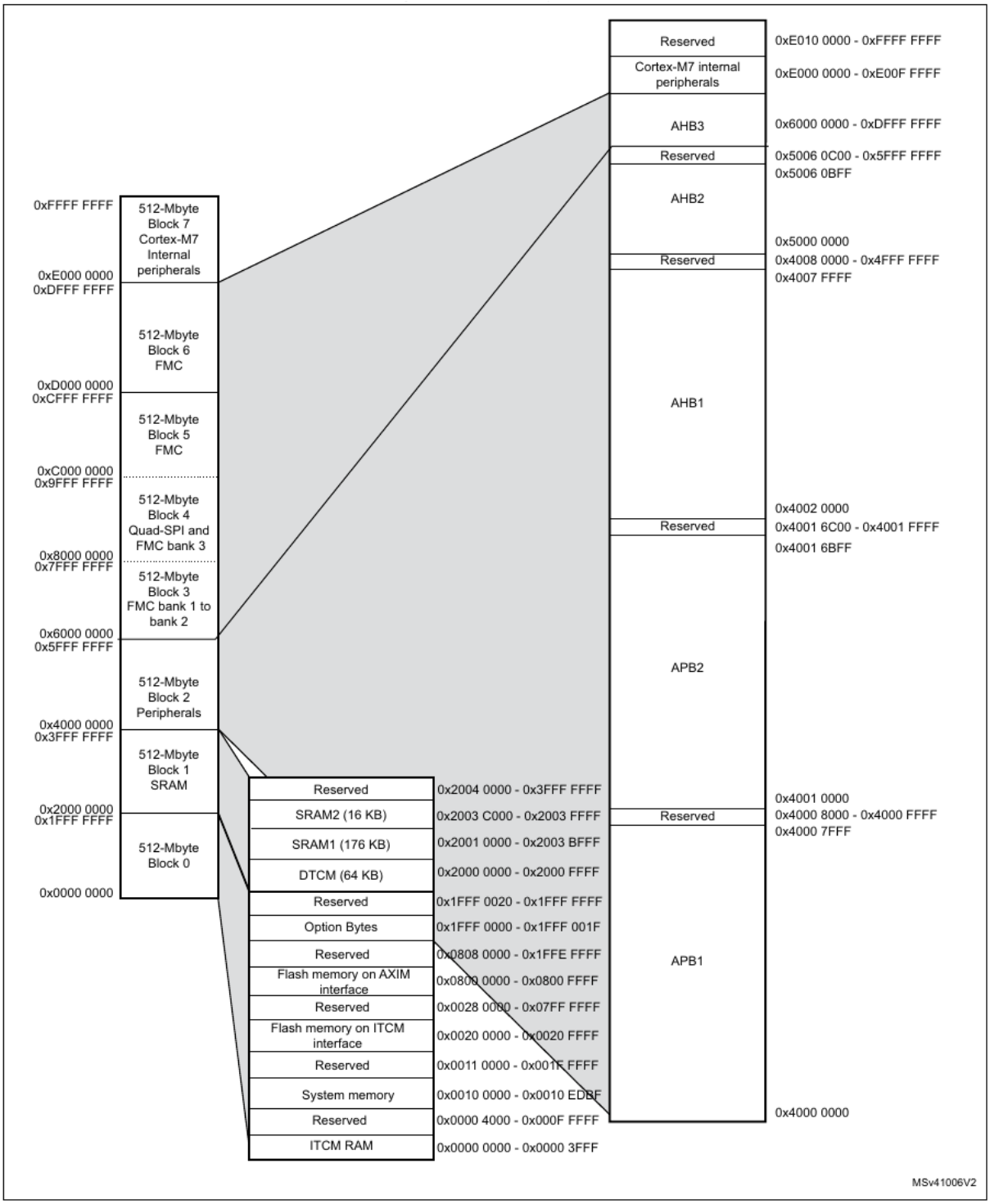


Figura 6: Mapa de memoria del **STM32F723IE**

Table 1. STM32F72xxx and STM32F73xxx register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 1000 - 0xA0001FFF	QuadSPI Control Register	AHB3	*Section 13.5.14: QUADSPI register map on page 378*
0xA000 0000 - 0xA000 0FFF	FMC control register		*Section 12.8.6: FMC register map on page 349*
0x5006 0800 - 0x5006 0BFF	RNG	AHB2	*Section 16.7.4: RNG register map on page 458*
0x5006 0000 - 0x5006 03FF	AES		*Section 17.7.18: AES register map on page 510*
0x5000 0000 - 0x5003 FFFF	USB OTG FS		*Section 32.15.64: OTG_FS/OTG_HS register map on page 1288*
0x4004 0000 - 0x4007 FFFF	USB OTG HS	AHB1	*Section 32.15.64: OTG_FS/OTG_HS register map on page 1288*
0x4002 6400 - 0x4002 67FF	DMA2		*Section 8.5.11: DMA register map on page 252*
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		*Section 5.3.27: RCC register map on page 192*
0x4002 3C00 - 0x4002 3FFF	Flash interface register		*Section 3.7.9: Flash interface register map*
0x4002 3800 - 0x4002 3BFF	RCC		*Section 5.3.27: RCC register map on page 192*
0x4002 3000 - 0x4002 33FF	CRC		*Section 11.4.6: CRC register map on page 275*
0x4002 2000 - 0x4002 23FF	GPIOI		*Section 6.4.11: GPIO register map on page 212*
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4002 1000 - 0x4002 13FF	GPIOE		
0x4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		

Figura 7: Detalle del mapa de memoria del **STM32F723IE**

6.4.1 GPIO port mode register (GPIOx_MODER) (x = A to K)

Address offset:0x00

Reset value: 0xA800 0000 for port A

Reset value: 0x0000 0280 for port B

Reset value: 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODER[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

Figura 8: Detalle del registro MODER

Hemos visto pues que podemos configurar ciertos puertos de entrada/salida, que a su vez cuentan con numerosos pines. Pero, ¿en qué puerto/pin se sitúa, por ejemplo, un LED?. Para ello, debemos acudir a la documentación de la placa, donde se han diseñado físicamente las conexiones desde el procesador, hasta los diferentes elementos de soporte. En concreto, en la sección `um2140 5.16` tenemos la lista de todas las conexiones de botones y LEDs. Lo vemos en la Figura 9

Finalmente, sabemos que, por ejemplo, el LED5 se controla desde el puerto GPIOA, en el PIN 7. El puerto A está en la dirección `0x40020000`, y dispone de varios registros que deberemos configurar adecuadamente para controlar, finalmente, el LED.

2.3. Desarrollo de la práctica

2.3.1. Creación del proyecto

Vamos a controlar, directamente y escribiendo en registros del procesador, un LED (en concreto, el LED5 que acabamos de ver). Pero antes, debemos crear un proyecto en STMCubeIDE, la herramienta de desarrollo de ST. Para ello:

1. Abrimos STM32CubeIDE, buscándolo en el menú del sistema. Elegimos un workspace (o carpeta de trabajo) donde trabajar. Tras abrirlo, veremos el programa de la Figura 10
2. Creamos un nuevo proyecto con `File >STM32 Project Create / Import`.
3. Seleccionamos `STM32CubeIDE Empty Project` (Figura 11)
4. En `Board Selector`, buscar `723e` y seleccionar la única placa disponible (Figura 12).

Table 5. Control port assignment

Reference	Color	Name	Comment
B1	BLUE	USER	Alternate function Wake-up
B2	BLACK	RESET	-
LD1	BLUE	ARDUINO	PA5
LD2	RED	5 V Power	-
LD3	RED	Fault Power	Current upper than 625 mA
LD4	RED/GREEN	ST-LINK COM	Green during communication
LD5	RED	USER1	PA7
LD6	GREEN	USER2	PB1
LD7	RED	USB OTG HS OVCR	PH10
LD8	GREEN	V_{BUS} USB HS	PB13
LD9	RED	USB OTG FS OVCR	PB10
LD10	GREEN	V_{BUS} USB FS	PA9

Figura 9: Pines para LEDS y botones de la placa **32F723EDISCOVERY**

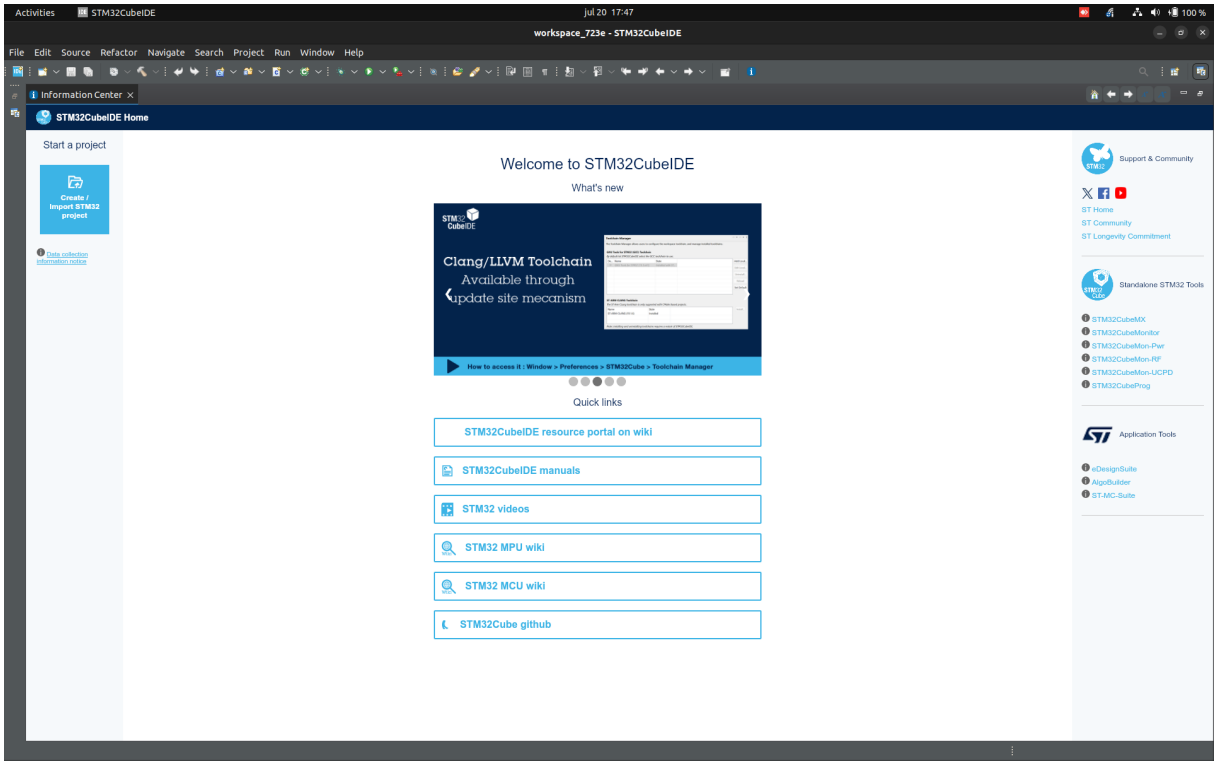


Figura 10: Ventana inicial de la aplicación

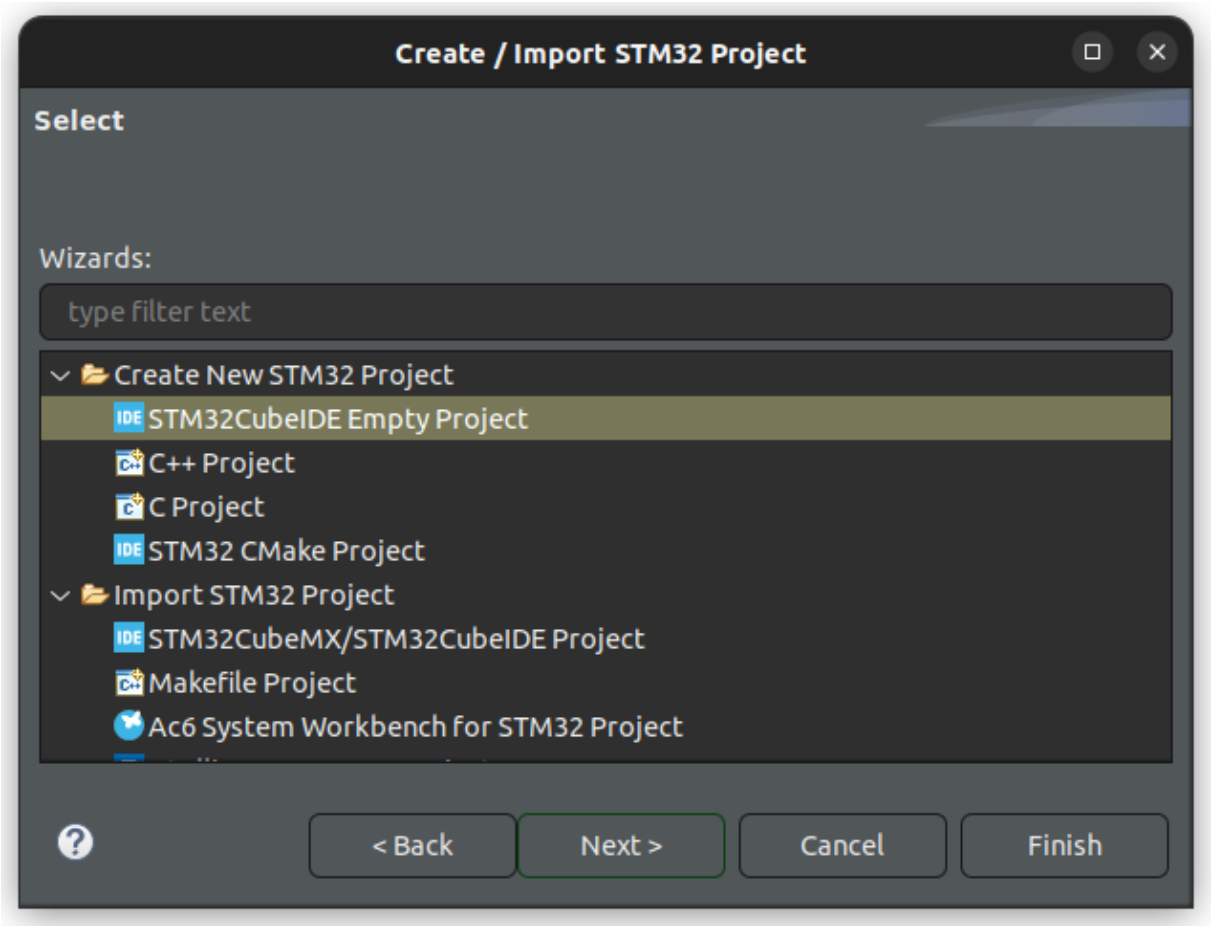


Figura 11: Creación de proyecto

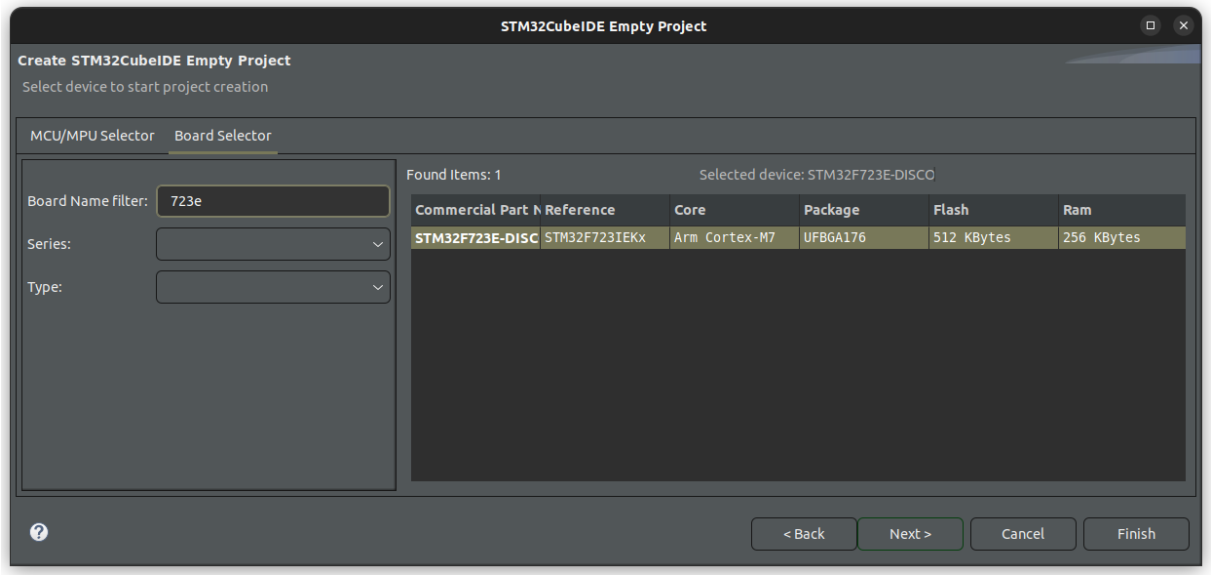


Figura 12: Selección de placa

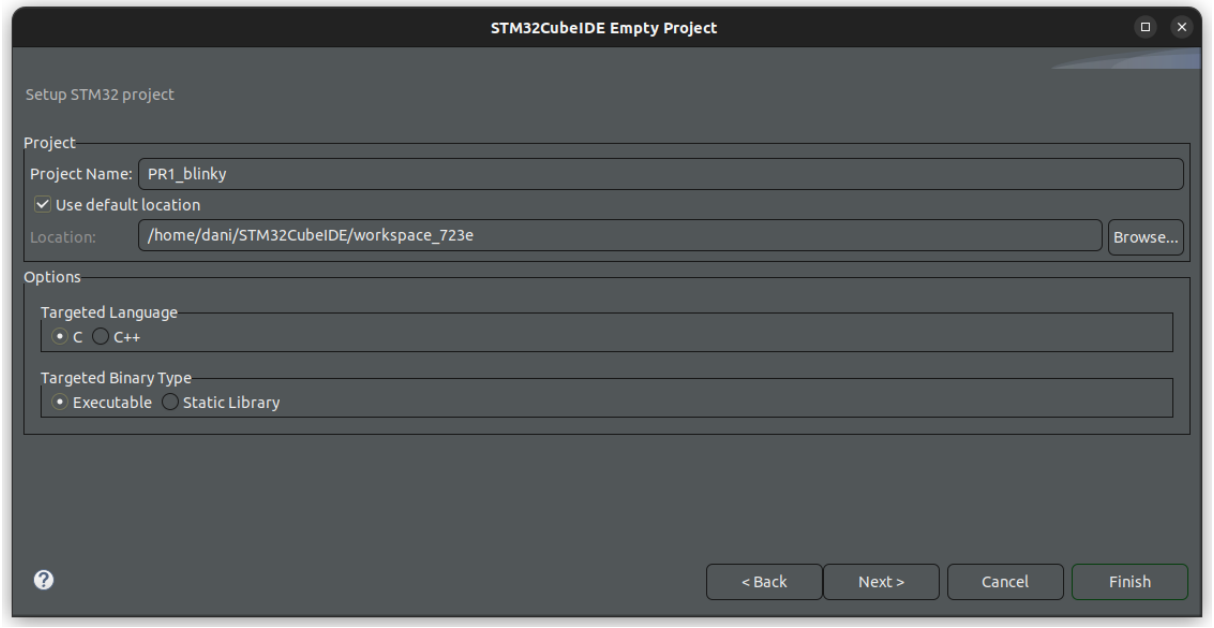


Figura 13: Selección de nombre de proyecto

5. Finalmente, dar un nombre al proyecto y terminar (Figura 13).

2.3.2. Depuración del proyecto

Ahora deberíamos tener un proyecto en blanco para trabajar (Figura 14). Antes de nada, vamos a probar que la conexión con la placa funciona, y podemos compilar y depurar el proyecto. Para ello:

1. Haz doble click en la carpeta `Src` dentro de tu proyecto, y luego, en `main.c`.
2. Con el archivo abierto, haz click en el martillo (build) que se encuentra en la barra superior de tareas. Verás el log de compilación en la terminal (Figura 15).
3. Finalmente, y si no ha habido problemas, haz click en el icono de depuración (debug) en la misma barra superior (el bichito verde).
4. Acepta la configuración de depuración por defecto (Figura 16) y vuelve a aceptar cuando pida cambiar a la vista de depuración.

Ahora puedes depurar el proyecto poco a poco. Entre otras, el depurador permite:

- Ir paso a paso por las líneas de código.
- Colocar *breakpoints* para parar la ejecución al llegar a ciertas líneas.
- Inspeccionar el contenido de los registros del procesador.
- Inspeccionar el contenido de los registros de los diferentes periféricos.
- Inspeccionar el contenido de la memoria.

Se recomienda explorar todas estas funcionalidades, que se pueden ver en la Figura 17.

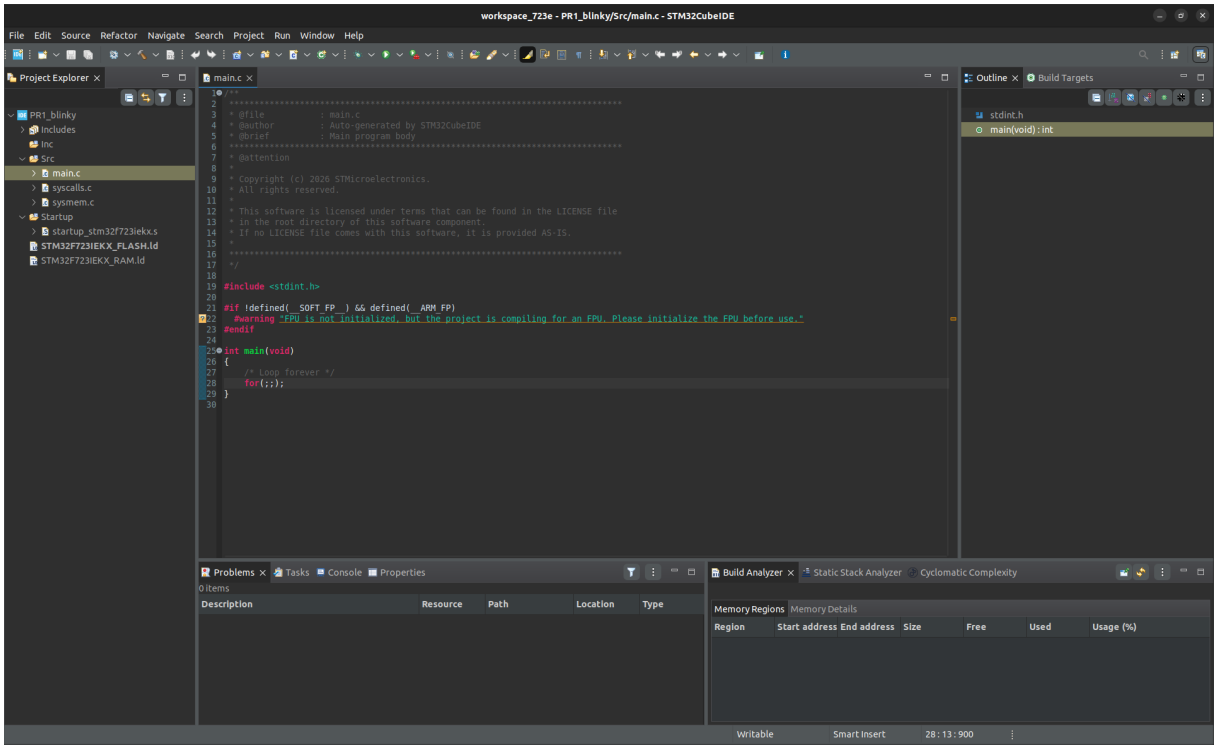


Figura 14: Aplicación por defecto

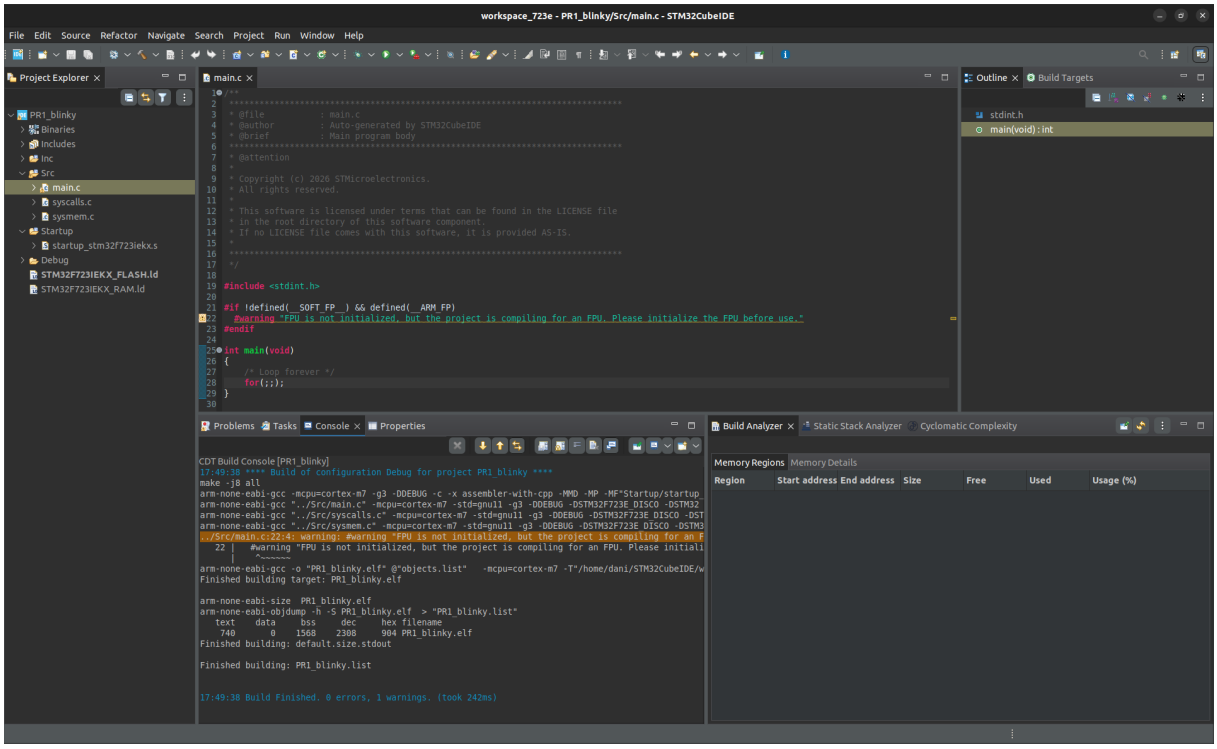


Figura 15: Log tras construir la aplicación

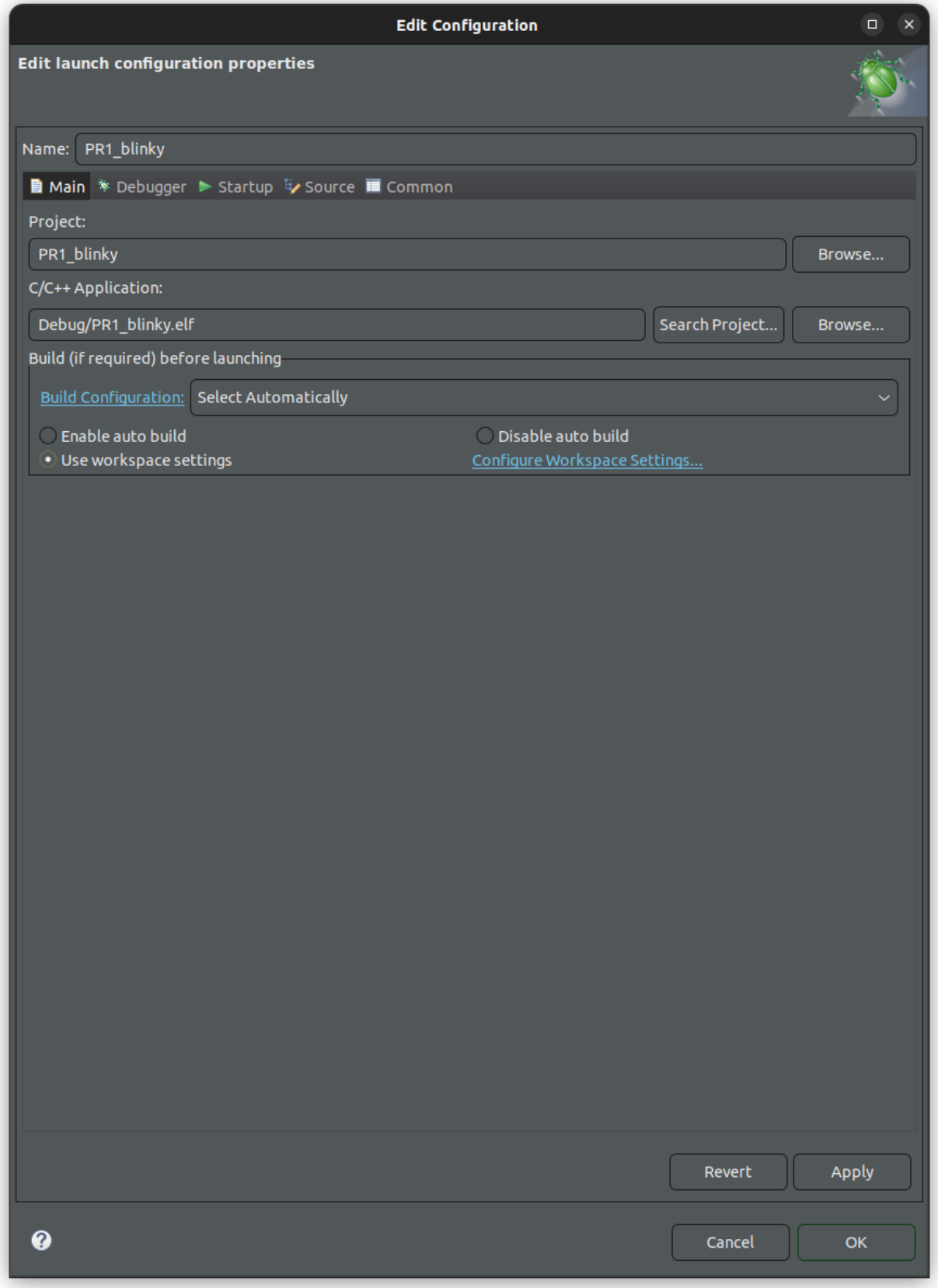


Figura 16: Selección de depuración por defecto

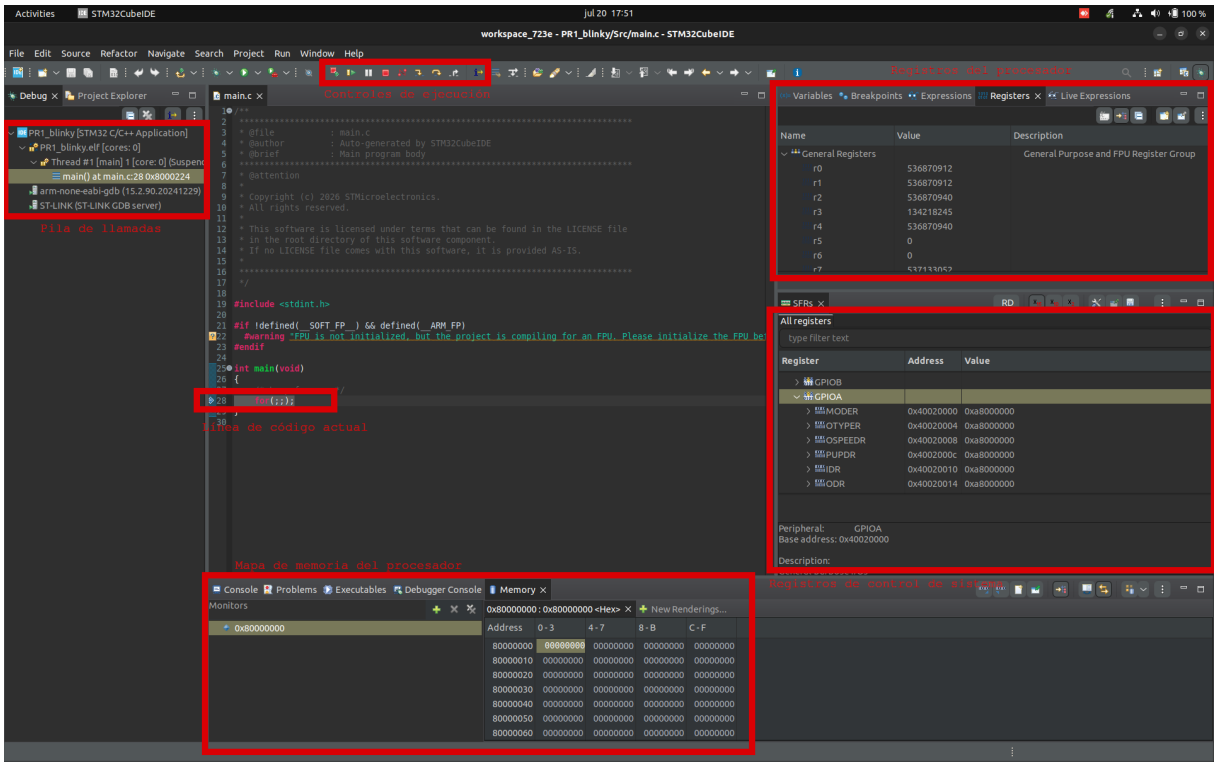


Figura 17: Ventana de depuración personalizada

2.3.3. Desarrollo del proyecto

Finalmente, debemos programar nuestro código para encender el LED. Para ello, habrá que escribir en las direcciones concretas de los registros de GPIO, en este caso de GPIOA-PIN7. Recuerda que la documentación de los registros disponibles está en `rm0431 6.4`.

- Configurar GPIOA-PIN7 como salida en `MODER`.
- Poner el pin en tipo *push-pull* en `OTYPER`.
- Configurar la velocidad a alta en `OSPEEDR`.
- Deshabilitar las resistencias en `PUPDR`.
- Finalmente, se podrá escribir el valor del led (0/1) a través de `ODR`. Otra opción es escribir directamente al registro `BSRR`, que tiene la ventaja de evitar lecturas o escrituras múltiples (aunque para nuestro caso no afecta en nada).

Adicionalmente, tenemos que activar el reloj del periférico GPIOA a través del registro `RCC_AHB1ENR` (`rm0431 5.3.10`). La dirección base la podemos consultar en `rm0431 1.6.2`.

Quizá te quede la pregunta de cómo narices escribo yo en una dirección de memoria concreta... la respuesta son... ¡punteros!

```
1 //Imaginemos que quiero escribir en la dirección 0x40020000
2 //justo en los bits 14-15
3 // (por lo que sea)
4
5 //borro configuración actual (máscara con ceros en las posiciones de interés)
```

```
* ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
//pongo la configuración que me interesa
* ((uint32_t *) (0x40020000)) |=   0b01 << (7*2);
```

Aprovecha también esta función para hacer que parpadee cada cierto tiempo:

```
void delay(int loops) {
    for (int i = 0; i < loops; i++) {
        __asm("nop");
    }
}
```

Nuestra función `main.c` tendrá por tanto esta pinta:

```
int main(void)
{
    //activar reloj de GPIOA en RCC_AHB1ENR
    //configurar Pin0-GPIOA como salida
    //configurar Pin0-GPIOA modo push-pull
    //configurar Pin0-GPIOA velocidad alta
    //configurar Pin0-GPIOA con pullup/down desactivado

    /* Loop forever */
    for(;;) {
        //encender LED
        delay(1000000);
        //apagar LED
        delay(1000000);
    }
}
```

2.4. Para hacer más

Busca el GPIO del botón de la placa, configúralo en modo entrada, y haz que el LED se encienda o apague cada vez que se pulse el botón, en lugar de hacerlo de manera automática.

3. Práctica 2: Creación de un *Board Support Package* (BSP)

3.1. Objetivos

- Desarrollar un *Board Support Package* minimalista.
- Entender la importancia de crear, en la medida de lo posible, código no dependiente del hardware.

3.2. Fundamentos teóricos

Un *Board Support Package* o BSP comprende una colección de bibliotecas que ayudan al programador a trabajar con una placa o procesador determinado. Generalmente, incluyen definiciones, funciones, macros y demás utilidades para que el programador no tenga que trabajar directamente con código de muy bajo nivel como, por ejemplo, hemos hecho en la práctica anterior. Por ejemplo:

```
// En lugar de un código así
* ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
* ((uint32_t *) (0x40020000)) |=   0b01 << (7*2);

// Buscamos algo así
GPIO_SET_MODE(GPIOA, PIN7, MODE_OUTPUT);
```

Por supuesto, alguien tiene que ser el encargado de crear, por primera vez, este BSP. En sistemas empotrados, normalmente el BSP tiene que ser creado a medida, pues la combinación de procesador, periféricos y pines es particular para cada placa desarrollada. Para la placa **32F723EDISCOVERY**, ya existe un BSP, y se anima al alumno a consultarlo y ver cómo está construido. A lo largo de estas prácticas, desarrollaremos otro, mucho más minimalista y comedido, que aporte la funcionalidad justa que necesitamos. En esta práctica comenzamos con ese trabajo.

Vamos a ver el ejemplo concreto de cómo desarrollar la función anterior, para enteder mejor las partes que puede tener un BSP.

3.2.1. Definiciones de direcciones

En primer lugar, necesitamos definir las direcciones de memoria donde se encuentran todos nuestros periféricos. Esto cumple una doble función: Por un lado, si cambiaran, las podemos modificar fácilmente al estar definidas como macros. Por otro lado, hacemos el código reutilizable para procesadores de la misma familia (por ejemplo, todos los Cortex-M7 de ARM (como el **STM32F723IE**que utilizamos) reutilizan partes de los mismos BSPs).

```
#define GPIO_BASE_ADDR 0x40020000
#define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
#define GPIOB_BASE_ADDR (GPIO_BASE_ADDR + 0x0400)
...
```

Table 24. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00	GPIOB_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x00	GPIOx_MODER (where x = C..K)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	GPIOx_OTYPER (where x = A to K)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figura 18: Extracto parcial de registros del controlador de GPIO

3.2.2. Definiciones de registros

Lo siguiente es definir los registros que se encuentran en dichas direcciones. Como suelen ser varios, y consecutivos, se suele definir una estructura de tipo (en lenguaje C) que los englobe todos, por facilidad de uso. Por ejemplo, mirando `rm0431 6.4.11`, podemos ver todos los registros contenidos en cada controlador de GPIO. Se muestra la primera parte de la tabla de registros en la Figura 18

Un detalle interesante a ver aquí es que, por ejemplo, para el registro `MODER`, nos dice que tiene tres valores por defecto (reset) diferentes, según si se trata del puerto A, B, u otros. El registro `TYPER`, sin embargo, tiene el mismo valor por defecto para todos los puertos (A-K). En cada registro, además, nos indica qué bit controla qué cosa. Para los registros `MODER`, los bits, de dos en dos, controlan el modo de GPIO (especificados en `rm0431 6.4.1`). Para `OTYPER`, controla el tipo de salida (`rm0431` 6.4.2). Si hacemos el código para tener una estructura que nos permita controlar estos registros, tendríamos algo de este estilo:

```
typedef struct
{
    volatile uint32_t MODER;    /*!< GPIO port mode register, Address offset: 0
        x00      */
    volatile uint32_t OTYPER;   /*!< GPIO port output type register, Address
        offset: 0x04      */
    ...
} GPIO_TypeDef;
```

Importante observar la declaración de las variables como tipo `volatile`, ya que, además de modificarse internamente desde el procesador, son registros cuyo valor puede modificarse desde fuera (por el propio control del periférico). Con la palabra clave `volatile` conseguimos que el compilador no optimice esas variables y siempre las consulte directamente de memoria.

Pregunta: ¿A qué se debe que todos los registros queden colocados en direcciones sucesivas de memoria al ser declarados con esta sintaxis?

Pregunta: Explora qué hace exactamente el compilador al trabajar con objetos volátiles.

3.2.3. Gestión de registros

Tenemos definidas las direcciones, y las estructuras de registros. Ahora necesitamos tener una forma de escribir en dichas estructuras. Para ello, vamos a crear punteros a las direcciones de memoria base, con el tipo de estructura definido:

```
#define GPIOA                ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
```

Si la base del controlador se encuentra en la dirección `0x40020000` (como es el caso de GPIOA), los registros se encontrarán a continuación (el primero `MODER` en `0x40020000`, el segundo `OTYPER` en `0x40020004`, y así sucesivamente). Así, lo que conseguimos es que, posteriormente, en el código de nuestro programa, podamos hacer algo de este estilo:

```
GPIOA->MODER = 0xA8000000;
uint32_t temp = GPIOA->MODER;
```

Exactamente equivalente a haber escrito (o leído) desde las direcciones de memoria correspondientes. Es decir, de una forma legible, acceder (para escribir o leer) los contenidos de estos registros, que a la vez sirven para control de los periféricos.

Pregunta: Cada registro normalmente controla, desde cada uno de sus bits, elementos diferentes del periférico. ¿Cómo puedo leer (o escribir) en unos bits concretos del registro?

3.2.4. Funciones de control de registros

Aunque ahora es más cómodo trabajar con los registros, lo ideal es contar con funciones que nos ayuden, aún más, a abstraer todos estos códigos de bajo nivel que son propensos a errores por parte del programador. Por ejemplo, ahora mismo, si quisiéramos cambiar el modo del pin 7, del puerto A, a *output*, deberíamos hacer algo de este estilo:

```
GPIOA->MODER &= ~(0b11 << (7*2));
GPIOA->MODER |=   0b01 << (7*2);
```

Es decir, borramos primero los dos bits de las posiciones 14 y 15 (Ver tabla anterior) y luego les ponemos el valor "01"para configurarlo a nuestro gusto. Pero, ¿y si ahora queremos cambiar el pin 4 del puerto B? ¿o ponerlo en modo *input*?. Es mejor crear una función genérica que nos permita hacer todo.

Aunque esto ya es a gusto del consumidor, la sugerencia para estas funciones es hacerlas lo más genéricas posibles. En este caso, deberíamos poder permitir trabajar con cualquier puerto, cualquier pin, y cualquier modo. Por ejemplo, podemos hacer algo así:

5.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0010 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	Res.	Res.	Res.	Res.	Res.	Res.	DMA2 EN	DMA1 EN	DTCMRA MEN	Res.	BKPSR AMEN	Res.	Res.
	rw	rw							rw	rw	rw		rw		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	Res.	Res.	GPIOI EN	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIO BEN	GPIO AEN
			rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Figura 19: Detalle de bits del registro `AHB1ENR`

```c
typedef enum {
    GPIO_MODE_INPUT  = 0x00,
    GPIO_MODE_OUTPUT = 0x01,
    GPIO_MODE_ALT    = 0x02,
    GPIO_MODE_ANALOG = 0x03
} GPIO_Mode_t;

static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin,
    GPIO_Mode_t mode) {
    uint32_t temp = GPIOx->MODER; //leemos registro
    temp &= ~(0b11 << (pin * 2)); //borramos modo actual
    temp |=  (mode << (pin * 2)); //ponemos modo nuevo
    GPIOx->MODER = temp;          //escribimos de vuelta
}
```

Con esto, ya podríamos hacer desde nuestro código principal, lo siguiente:

```c
GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
```

3.2.5. Gestión directa bit a bit de registros

Otros registros, como por ejemplo `AHB1ENR` (`rm0431 5.3.10`) tienen una estructura que no es regular, ya que cada bit controla una cosa diferente, como podemos ver en la Figura 19.

Para estos casos, C nos permite una sintaxis como la siguiente, que nos deja acceder individualmente a nivel de bit a los registros:

```c
//Base address
#define RCC_AHB1EN_BASE_ADDR 0x40023830

//Structure for Reset and Clock Control peripheral clock register 1
typedef union {
    volatile uint32_t reg;

    // 2. Access individual bits or bit-groups for the register above
    struct {
        volatile uint32_t GPIOAEN       : 1;  // Bit 0: Enable peripheral (1 bit
            )
```

```
        volatile uint32_t GPIOBEN       : 1;  // Bit 1: Enable peripheral (1 bit
            )
        ...
    } bits;
} RCC_AHB_ENR_TypeDef;

//Handle:
#define RCC_AHB1ENR         ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)
```

Posteriormente, podemos acceder a los campos individuales de la siguiente manera:

```
RCC_AHB1ENR->bits.GPIOAEN = 1;
```

Este registro en concreto es muy importante, pues activa los periféricos. De no activarlos, no funcionarán aunque se configuren adecuadamente.

3.3. Desarrollo de la práctica

Crea un BSP para que, en el código de la práctica anterior, no haya que acceder directamente a registros del periférico GPIOA. Necesitarás funciones para configurar `MODER`, `OTYPER`, `OSPEEDR`, `PUPDR` y también para escribir 1 o 0 (esto lo puedes hacer desde el registro `ODR` o `BSRR`). Consulta `rm0431 6.4` para ver todos los registros y sus funcionalidades. Deberás:

1. Completar la definición de tipo para incluir todos los registros. También completar la definición de tipo de `AHB1ENR`.

```
//... completar direcciones
#define GPIO_BASE_ADDR
#define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
#define RCC_AHB1EN_BASE_ADDR

//Structure for GPIO controllers
typedef struct
{
    volatile uint32_t MODER;    /*!< GPIO port mode register,
                      Address offset: 0x00      */
    volatile uint32_t OTYPER;   /*!< GPIO port output type register,
                Address offset: 0x04      */
    //... completar
} GPIO_TypeDef;

//Structure for Reset and Clock Control peripheral clock register 1
typedef union {
    volatile uint32_t reg;

    // 2. Access individual bits or bit-groups for the register above
    struct {
        volatile uint32_t GPIOAEN       : 1;  // Bit 0: Enable peripheral
            (1 bit)
        volatile uint32_t GPIOBEN       : 1;  // Bit 1: Enable peripheral
            (1 bit)
        //... completar
    } bits;
} RCC_AHB_ENR_TypeDef;

#define GPIOA               ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
#define RCC_AHB1ENR         ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)
```

2. Crear las funciones, junto con sus definiciones de valores, para controlar los diferentes registros. Toma el ejemplo previo de la función `GPIO_MODE_SET` para hacer las demás.

```
static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin
    , GPIO_Mode_t mode);
static inline void GPIO_OTYPE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t
    pin, GPIO_OType_t otype);
static inline void GPIO_OSPEED_SET(volatile GPIO_TypeDef *GPIOx, uint32_t
    pin, GPIO_OSpeed_t ospeed);
static inline void GPIO_PUPD_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin
    , GPIO_PuPd_t pupd);
static inline void GPIO_PIN_WRITE(volatile GPIO_TypeDef *GPIOx, uint32_t
    pin, GPIO_State_t state);
```

3. Recuerda empaquetar toda esta funcionalidad en un fichero `bsp.h` que irá dentro de la carpeta `Inc` del proyecto.

4. Adaptar el código original `main.c` de la práctica para utilizar el BSP.

```
int main(void)
{
    RCC_AHB1ENR->bits.GPIOAEN = 1;

    GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
    //... completar

    for(;;) {
        GPIO_PIN_WRITE(GPIOA, 7, GPIO_STATE_ONE);
        delay(1000000);
        //... completar
        delay(1000000);
    }
}
```

3.4. Para hacer más

Abre el proyecto BSP de ejemplo (dentro del paquete `STM32CubeF7`) e investiga cómo están organizadas las funciones, definiciones, registros... Para navegar por el proyecto, puedes ir haciendo `ctrl+click` en las llamadas a funciones o variables, y te llevará al fichero donde están definidas.

4. Práctica 3: Entrada / Salida programada - UART TX

4.1. Objetivos

- Comprender más en profundidad el concepto de "periférico"dentro de un procesador.
- Aprender a programar un periférico.
- Interactuar con el periférico desde fuera de la placa.

4.2. Fundamentos teóricos

4.2.1. UART

La comunicación **UART** (*Universal Asynchronous Receiver-Transmitter*) es un protocolo de comunicación serie asíncrono utilizado ampliamente en sistemas embebidos. Al ser asíncrono, no requiere de una señal de reloj compartida entre el emisor y el receptor; en su lugar, ambos dispositivos deben estar configurados previamente a la misma velocidad de transmisión (*baud rate*).

Estructura de la Trama UART:

En estado de reposo (*Idle*), la línea de transmisión se mantiene en nivel lógico alto (**1**). Una trama estándar de datos está compuesta por los siguientes elementos:

1. **Bit de Inicio (*Start Bit*):** Transición de nivel alto (**1**) a nivel bajo (**0**) durante un período de bit. Notifica al receptor que una nueva trama está comenzando.
2. **Longitud de Palabra (*Data Bits*):** Bloque de datos útil, transmitido habitualmente empezando por el bit menos significativo (*LSB first*). Suele configurarse entre 5 y 9 bits, siendo 8 bits el valor más común.
3. **Bit de Paridad (*Parity Bit*, opcional):** Bit añadido al final de los datos para la detección elemental de errores:
   - **Paridad Par (*Even*):** Se ajusta a 1 o 0 para que el número total de unos en la palabra de datos (incluyendo el bit de paridad) sea par.
   - **Paridad Impar (*Odd*):** Se ajusta para que el número total de unos sea impar.
4. **Bits de Parada (*Stop Bits*):** Uno o varios bits en nivel lógico alto (**1**) al final de la trama que garantizan un tiempo de resincronización mínimo para la línea antes de la siguiente transmisión.

Diagrama de Tiempos Típico:

En la Figura 20 se muestra la representación gráfica de una trama serie de 8 bits de datos con paridad y 1 bit de stop:

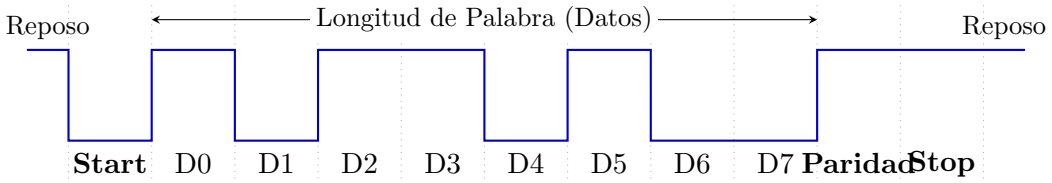


Figura 20: Cronograma de una trama de datos UART de 8 bits con paridad y 1 bit de parada.

4.2.2. UART en el STM32F723IE

En el procesador de la placa, se disponen de varios periféricos de UART diferentes, numerados como USARTx, siendo la x el número de UART. La UART dispone de varios registros de control, otros de configuración, y finalmente registros para la recepción/envío de datos. Todos ellos se describen en `rm0431 27.8`. Las direcciones base de los periféricos USARTx se encuentran en `rm0431 1.6.2`. Cómo hacer la transmisión en sí se describe en `rm0431 27.5.2`.

¿Ahora, qué USARTx utilizamos? en `um2140 5.13` nos dice que la USART6 está conectada directamente a la salida desde ST-Link (El depurador que venimos utilizando hasta ahora para conectar con la placa). Por tanto, deberemos configurar USART6. Los pines donde se conecta (que habrá que activar) se pueden ver en `ds11853 Table 10`. En concreto, vemos que son el pin PC6 para USART6_TX y PC7 para USART6_RX. (De momento solo usamos el TX para transmitir). Para que por ese pin salga la UART (Y no el valor genérico que ponemos en el puerto de salida del GPIO), deberemos configurar la función alternativa 8 como indica `ds11853 Table 12`, que conecta la USART6 al puerto C.

4.3. Desarrollo de la práctica

Para el desarrollo de esta práctica hay que extender el BSP introduciendo las funciones y definiciones necesarias de la UART, y posteriormente controlarla para emitir datos hacia el ordenador. La documentación necesaria puede verse en `rm0431 27`. Los pasos a seguir son los siguientes:

1. Según `ds11853 Table 10`, el pin PC6 es quien saca al exterior la UART. Configúralo en modo de función alternativa en `MODER`, tipo push-pull en `OTYPER`, pull-up en `PUPDR`, velocidad alta en `OSPEEDR`, y configura la función alternativa en `AFR` para que sea la 8 (como indica `ds11853 Table 12`). No te olvides de activar el puerto C a través de `AHB1ENR`.

```
RCC_AHB1ENR->bits.GPIOCEN = 1;
GPIO_MODE_SET(GPIOC, 6, GPIO_MODE_ALT);
//... completar
```

2. Ahora le toca el turno a la UART6 como nos indica `um2140 5.13`. Actívala desde `APB2ENR` (`rm0431 5.3.14`) lo primero.

```
//en bsp.h
#define RCC_APB2EN_BASE_ADDR 0x40023844
//Structure for Reset Control peripheral register APB1
typedef union {
    volatile uint32_t reg;

    // 2. Access individual bits or bit-groups for the register above
```

```
    struct {
        volatile uint32_t TIM1EN        : 1;  // Bit 0:
        volatile uint32_t TIM8EN        : 1;  // Bit 1:
        volatile uint32_t RES1          : 2;  // Bit 2-3:
        //... completar
    } bits;
} RCC_APB2R_TypeDef;
#define RCC_APB2ENR         ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)


//en main.c
RCC_APB2ENR->bits.USART6EN = 1;          //enable UART clock
```

3. Configura la UART desde los registros `CR1` y `CR2` Activa el periférico, configura la longitud de palabra a 8 bits, 1 bit de stop y sin paridad, pone por último el oversampling a 16.

4. Configura la velocidad de la UART desde `BRR`. Tendrás que ponerle el valor del reloj del sistema (16MHz) entre la velocidad de la uart (115200).

5. Por último, activa el modo de transmisión desde `CR1`.

```
// UART STOP BITS
typedef enum {
    UART_STOP_ONEBIT = 0x00,
    UART_STOP_HALFBIT = 0x01,
    UART_STOP_TWOBIT   = 0x02,
    UART_STOP_ONEHALFBIT = 0x03
} UART_Stop_t;
#define UART_STOP_MASK 0b11
#define UART_STOP_SHIFT 12

static inline void UART_STOPBIT_SET(volatile USART_TypeDef *USARTx,
    UART_Stop_t stopbits) {
    //... completar configurar el registro CR2 con los bits de stop
}

// UART ENABLE
typedef enum {
    UART_DISABLE = 0x00,
    UART_ENABLE = 0x01
} UART_Enable_t;
#define UART_ENABLE_MASK 0b1
#define UART_ENABLE_SHIFT 0

static inline void UART_ENABLE_SET(volatile USART_TypeDef *USARTx,
    UART_Enable_t enable) {
    //... completar configurar el registro CR1 con el bit de enable
}

// UART WORD LENGTH
typedef enum {
    UART_WORD_8B = 0x00,
    UART_WORD_9B = 0x01,
    UART_WORD_7B = 0x02
} UART_WordLength_t;
#define UART_WORD_MASK  0b1
#define UART_WORD_SHIFT_12 12
#define UART_WORD_SHIFT_28 28

static inline void UART_WORDLENGTH_SET(volatile USART_TypeDef *USARTx,
    UART_WordLength_t length) {
```

```
38      //... completar escribir longitud en registro CR1
39  }
40
41  //... completar todas estas otras funciones auxiliares
42  static inline void UART_PARITY_SET(volatile USART_TypeDef *USARTx,
        UART_Parity_t parity);
43  static inline void UART_OVERSAMPLING_SET(volatile USART_TypeDef *USARTx,
        UART_Oversampling_t over);
44  static inline void UART_TX_MODE_SET(volatile USART_TypeDef *USARTx,
        UART_TX_Mode_t mode);
```

6. Cuando quieras transmitir un byte, espera primero a que quede libre el transmisor (aparece un 1 en el bit TC del registro `ISR`. A continuación escribe el dato en el registro `TDR`, la UART lo tramsmitirá solo.

```
1   //... en bsp.h
2   static inline void UART_TransmitByte(volatile USART_TypeDef *USARTx,
        uint8_t data)
3   {
4       while (!(USARTx->ISR & (1U << 7U))); //Wait until we can push data (TXE
            bit enabled)
5       USARTx->TDR = data; //push data (this clears TXE bit)
6   }
7
8
9   //... en main.c
10  UART_TransmitByte(USART6, 'X');
11  UART_TransmitByte(USART6, 'Y');
12  //...
```

Para ver ahora los mensajes desde el ordenador, utilizaremos la utilidad `picocom` (o cualquier otra terminal de vuestro gusto). Desde la máquina virtual, abrir una terminal y ejecutar el comando:

```
1   picocom /dev/ttyACM0 -b 115200 -d 8 -y n -p 1
```

NOTA: El nombre del puerto puede no ser necesariamente `ttyACM0`, aunque en linux al menos es el valor por defecto. En Windows, por ejemplo, suele llamarse `COMx`, siendo x un número arbitrario.

NOTA: Si no está instalada la utilidad `picocom`, se puede instalar con:

```
1   > sudo apt-get install picocom
```

4.3.1. Envío de cadenas de texto por UART

Aunque mola mucho programar en *bare-metal*, también es útil partir de librerías ya implementadas para facilitar la vida a los programadores. Podemos ampliar la funcionalidad de nuestra UART para imprimir mensajes de depuración (así en próximas prácticas podremos depurar vía mensajes, en ocasiones más cómodo e intuitivo que la depuración con *breakpoints*). Incluye las siguientes funciones y prueba a depurar tu código con mensajes.

En `bsp.h` tendrás que añadir una función para escribir cadenas completas:

```
void UART_WriteString(USART_TypeDef *usart, const char *str) {
    while (*str) {
        UART_TransmitByte(usart, *str++);
    }
}
```

También tendrás que añadir la función de depuración en sí con las librerías necesarias auxiliares:

```
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <stdarg.h>

void Debug_Printf(const char *fmt, ...)
{
    char buf[256];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buf, sizeof(buf), fmt, args);
    va_end(args);

    UART_WriteString(USART6, buf);
}
```

Y listo! Puedes llamar a la función `Debug_Printf` con el texto con formato que quieras, para imprimir los valores de variables, por ejemplo, o simplemente colocar mensajes de depuración por la UART.

4.4. Para hacer más

Investiga las interrupciones de la UART para evitar tener que hacer esperas activas que consuman recursos de CPU.

5. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones

5.1. Objetivos

- Aprender qué son las interrupciones, cómo funcionan, y sus características.
- Programar el controlador de interrupciones para la recepción de datos con UART.
- Programar el controlador de interrupciones para entrada con botón.

5.2. Fundamentos teóricos

5.2.1. Configuración de las interrupciones en el procesador

Para responder de manera rápida y consistente a eventos en un microcontrolador, es buena idea utilizar una *interrupción hardware* para ejecutar un pequeño fragmento de código al detectar dichos eventos. En esta práctica, vamos a ver cómo funciona el controlador de interrupciones en STM32, que puede configurarse para ejecutar ciertas funciones al ocurrir determinados eventos.

Cada procesador tiene su propia manera de tratar interrupciones. En la familia STM32, y en concreto de **STM32F723IE**, se utilizan dos periféricos diferentes para su gestión:

- `NVIC` : El (*Nested Vectored Interrupt Controller*) `pm0253 4.2` es el gestor de interrupciones interno del procesador Cortex-M7, núcleo dentro del **STM32F723IE**. Permite la programación de hasta 240 interrupciones, con control de prioridad en cada una de ellas. En el caso de nuestro procesador, **STM32F723IE**, están disponibles unas 100 (`rm0431 9.1.2`), aunque utilizaremos nada más que un par de ellas.
- `EXTI` : (*EXtended Interrupt Controller*). Su propósito es extender el número de interrupciones disponibles a los programadores a través de eventos externos al propio procesador. Cada periférico interno tiene sus propias líneas de interrupción en el `NVIC` , como puede verse en `rm0431 9.1.2` , sin embargo, para los eventos en pines de E/S (a través de GPIO), la configuración de la interrupción se hace desde el `EXTI` .

  En la familia STM32, los pines GPIO están agrupados (PA0, PB0, PC0...). El periférico `EXTI` no tiene una línea directa para cada pin individual de la placa, sino que usa líneas compartidas (la línea `EXTI0` gestiona PA0, PB0, PC0, etc. a través de un multiplexor en el `SYSCFG`). Por tanto, no se pueden usar, por ejemplo, PA0 y PB0 como interrupciones independientes simultáneamente.

Adicionalmente, los periféricos externos, aunque estén conectados a las líneas de interrupción, deben ser configurados para generar las señales de interrupción necesarias.

En resumen, estos procesadores tienen interrupciones internas a través del `NVIC` , que hay que activar, además de realizar la configuración de los periféricos y, algunos casos como el GPIO, configurar `EXTI` para recibir sus interrupciones. En esta práctica, configuraremos las interrupciones de `UART` directamente desde `NVIC` , y las de `GPIO` utilizando también `EXTI` . Podemos ver un diagrama de las interconexiones entre todo en la Figura 21.

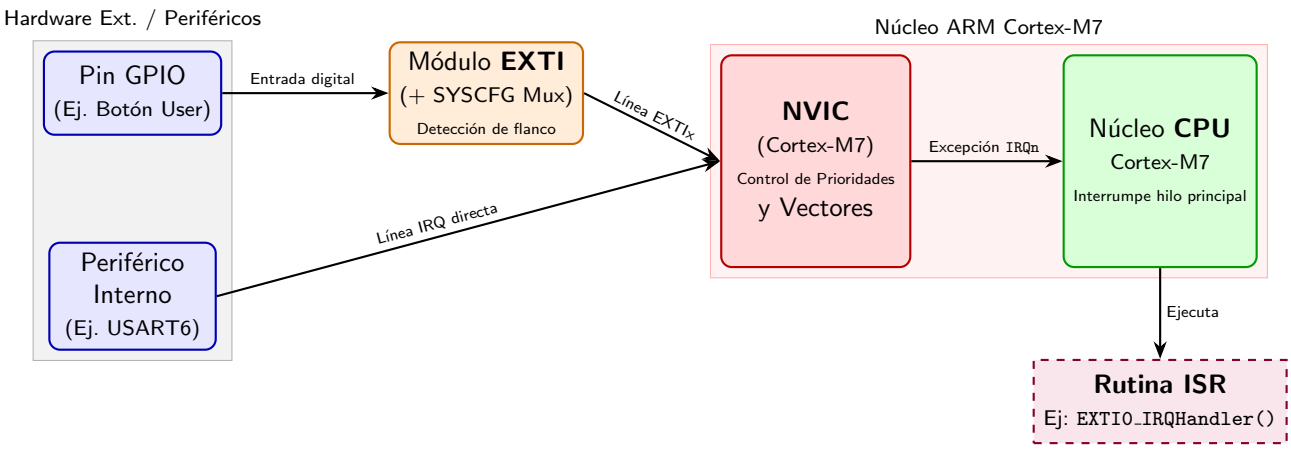


Figura 21: Flujo de interrupciones en el microcontrolador STM32F723E.

5.2.2. Gestión de interrupciones mediante funciones

Cuando se produce una interrupción, el procesador debe pausar su ejecución y saltar a la dirección de memoria donde se encuentra la rutina de respuesta o ISR (Interrupt Service Routine). Como la ubicación física de estas funciones en la memoria Flash no es fija (depende de la compilación), el procesador utiliza una Tabla de Vectores de Interrupción (Vector Table).

Esta tabla es un array de punteros ubicado al inicio de la memoria (por defecto desde `0x00000000`), donde cada posición está reservada para un evento concreto:

- La posición `0x00000000` almacena el valor inicial del puntero de pila (MSP).
- La dirección `0x00000004` contiene el vector de Reset.
- Direcciones posteriores contienen los vectores o punteros a las rutinas de interrupción, como por ejemplo `0x0000015C` que contiene el puntero a la rutina de la `USART6` (`rm0431 9.1.2`).

Para evitarnos la tarea de configurar esta tabla manualmente, el código de inicio del procesador (`Startup/startup_stm32f723iekx.s`) la genera automáticamente durante el arranque. Por defecto, todas las entradas de la tabla apuntan a un bucle infinito genérico (`Default_Handler`) mediante el atributo `.weak`. Si definimos en nuestro código C una función con el nombre exacto de la interrupción (por ejemplo, `USART6_IRQHandler`), el enlazador (linker) la sobrescribirá automáticamente, redirigiendo la tabla hacia nuestra rutina. Podemos ver este proceso en Figura 22.

5.3. Desarrollo de la práctica

Como en prácticas anteriores, el objetivo es seguir extendiendo nuestro BSP con las funciones necesarias como para controlar los periféricos y funcionalidades que utilizaremos en esta práctica. En este caso, debemos hacer varias cosas:

1. Apúntate las direcciones base de los periféricos `NVIC` en `pm0253 4.2`, y `EXTI` y `SYSCFG` en `rm0431 1.6.2`, crea además las estructuras de datos con las que accederás a los registros. Básate, respectivamente, en `pm0253 4.2` para `NVIC` (Ten en cuenta que los reigstros

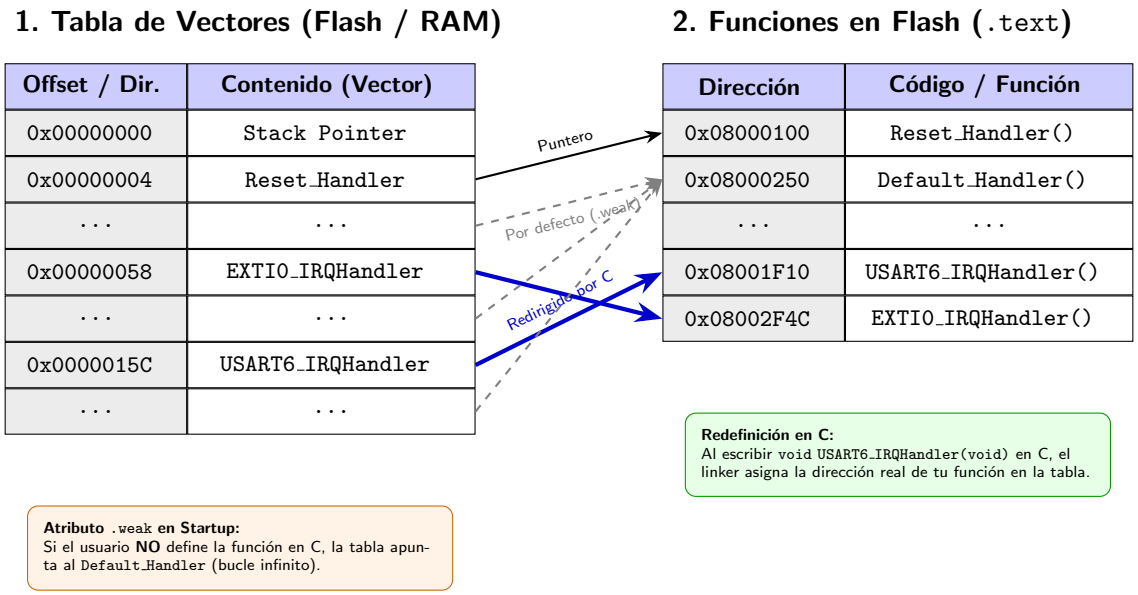


Figura 22: Mapeo de la Tabla de Vectores de Interrupción a la memoria del programa.

no están consecutivos), en rm0431 10.7 y posteriores para EXTI, y en rm0431 7.2.8 para SYSCFG. Crea además las variables para manejarlos. Puedes basarte en el siguiente código:

```
#define EXTI_BASE_ADDR      0x40013C00UL
//... completar para NVIC y SYSCFG

typedef struct
{
    volatile uint32_t ISER[8U];               /*!< Offset: 0x000 (R/W)
        Interrupt Set Enable Register */
    uint32_t RESERVED0[24U];
    //... completar
}  NVIC_TypeDef;

typedef struct
{
    volatile uint32_t MEMRMP;       /*!< SYSCFG memory remap register,
                          Address offset: 0x00      */
    //... completar
} SYSCFG_TypeDef;

typedef struct
{
    volatile uint32_t IMR;    /*!< EXTI Interrupt mask register,
                  Address offset: 0x00 */
    //... completar
} EXTI_TypeDef;

#define EXTI                ((EXTI_TypeDef *) EXTI_BASE_ADDR)
//... completar para NVIC y SYSCFG
```

2. Debemos activar el puerto y pin correspondiente al botón. Revisando la documentación, en um2140 Table 5 nos dice que el botón está enchufado a la función wake-up del procesador, y en um2140 Table 19 que dicha función SYS_WKUP1 está en el PA0. Por tanto, lo primero es configurar este pin como entrada, de tipo *push-pull* con el *pulldown* activado, y a velocidad alta. No te olvides de activar también el reloj de GPIOA a través de AHB1ENR.

3. A continuación, hay que activar también el pin de transmisión de la UART. En um2140 Table 19 podemos observar que la función USART6_RX está conectada al pin PC7. Tenemos que configurarlo en modo *alternativo* con la función 8 según nos indica ds11853 Table 12. Adicionalmente, configurarlo en modo *push-pull* con resistencia de *pullup* y velocidad alta.

4. Finalmente, configuramos la UART para que también funcione en modo recepción y tenga activas las interrupciones de recepción. Ambas cosas se pueden hacer desde el registro CR1 (rm0431 27.8.1). Crea funciones en el BSP y define las constantes necesarias. Puedes hacer algo de este estilo:

```
//en main.c
UART_RX_MODE_SET(USART6, UART_RX_ENABLE);
UART_RX_INT_SET(USART6, UART_RX_INT_ENABLE);


//en bsp.h
// RECEIVE enable
typedef enum {
    UART_RX_DISABLE = 0x00,
    UART_RX_ENABLE  = 0x01
} UART_RX_Mode_t;
#define UART_RX_MODE_MASK  0b1
#define UART_RX_MODE_SHIFT 2

static inline void UART_RX_MODE_SET(volatile USART_TypeDef *USARTx,
    UART_RX_Mode_t mode) {
    //... completar tocar el bit adecuado de CR1
}

//... completar hacer una función similar para RX INT enable
```

5. Ahora ya podemos comenzar a activar las interrupciones. Lo primero es activar el reloj para la configuración del sistema en APB2ENR (rm0431 5.3.14), para lo cual previamente deberemos definir los campos del registro.

```
//en bsp.h
#define RCC_APB2EN_BASE_ADDR 0x40023844 //APB2 clock control register

//Structure for Reset Control peripheral register APB1
typedef union {
    volatile uint32_t reg;

    // 2. Access individual bits or bit-groups for the register above
    struct {
        volatile uint32_t TIM1EN        : 1;  // Bit 0:
        volatile uint32_t TIM8EN        : 1;  // Bit 1:
        volatile uint32_t RES1          : 2;  // Bit 2-3:
        //... completar
    } bits;
} RCC_APB2R_TypeDef;

#define RCC_APB2ENR         ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)
```

A continuación, debemos configurar la interrupción externa EINT0 (que multiplexa todos los pines 0) para que trabaje con el puerto A (recordemos que el botón está en PA0). Esto se hace desde el registro EXTICR de SYSCFG (rm0431 7.2.3). Finalmente, hay que activar la interrupción externa 0 desde EXTI a través del registro IMR (rm0431 10.9.1) en modo *rising trigger* en el registro RTSR (rm0431 10.9.3).

```
//... completar en bsp.h crear y rellenar estas funciones
```

```
static inline void SYSCFG_EINT_MAP_PORT(uint32_t eintno, uint32_t portno) {
    //... completar modificar SYSCFG->EXTICR
}

static inline void EXTI_ENABLE_RISING_TRIGGER(uint32_t eintno, uint32_t
    enable) {
    //... completar activar/desactivar EXTI->RTSR
}

static inline void EXTI_UNMASK_INTERRUPT(uint32_t eintno, uint32_t unmask)
    {
    //... completar activar/desactivar EXTI->IMR
}


//en main.c
RCC_APB2ENR->bits.SYSCFGEN = 1;
//Activar EINT0 PUERTO A
SYSCFG_EINT_MAP_PORT(0, 0); //port a (0) to eint0
EXTI_ENABLE_RISING_TRIGGER(0, 1);   //enable on port a
EXTI_UNMASK_INTERRUPT(0, 1);        //unmask on port a
```

6. Ya tendríamos con esto el pin PA0 generando una interrupción en `EXTI0`, pero el procesador aún no responde a estas interrupciones. Adicionalmente, necesitamos activar la interrupción de la `UART`. Ambas cosas se hacen desde el `NVIC` en el registro `ISER` (`pm0253 4.2.2`). Para saber qué bits activar, debemos consular la numeración de las líneas de interrupción en `rm0431 9`.

```
//en bsp.h
//... completar buscar los números de línea
#define EXTI0_LINE_IRQn
#define USART6_INT_IRQn

static inline void NVIC_ENABLE_INT(uint32_t line) {
    //... completar modificar el registro NVIC->ISER
}

//en main.c
NVIC_ENABLE_INT(EXTI0_LINE_IRQn);
NVIC_ENABLE_INT(USART6_INT_IRQn);
```

7. Por último, necesitamos crear dos funciones que respondan a sendas interrupciones. En este caso, la función `USART6_IRQHandler` que responda a la `USART6`, y `EXTI0_IRQHandler` que responda al botón.

8. En el caso de la primera, `USART6_IRQHandler`, deberemos comprobar primero que la interrupción ha sido generada por recepción, consultando el bit `RXNE` del registro `ISR` de la `UART` (`rm0431 27.8.8`). Si es así, podremos leer el registro `RDR` (`rm0431 27.8.10`) para consultar el dato recibido, lo cual a su vez limpiará la interrupción. Dentro de la interrupción, para darle funcionalidad, podemos por ejemplo enviar de vuelta el dato leído al ordenador.

```
void USART6_IRQHandler(void)
{
    //... completar comprobar si el flag de interrupción está activo
    if () {
        //... completar leer el byte y retransmitirlo
    }
}
```

9. En el caso de la segunda `EXTI0_IRQHandler`, el proceso es diferente. Primero, debemos comprobar que la interrupción está pendiente desde el registro `PR` de `EXTI` (`rm0431 10.9.6`). A continuación, habrá que borrar dicho bit, escribiendo de nuevo en el registro, como indica la documentación. En esta función, por ejemplo, se puede poner como funcionalidad el dar la vuelta al LED que ya teníamos configurado anteriormente, para comprobar que funciona.

```
void EXTI0_IRQHandler(void)
{
    //... comprobar en EXTI->PR si la línea 0 generó interrupción
    if () {
        //... limpiar línea 0 escribiendo la posición 0 de EXTI->PR
        //... completar funcionalidad
    }
}
```

5.4. Para hacer más

Intenta cambiar "en caliente" la función que gestiona una interrupción, por ejemplo la del botón, modificando el puntero de las direcciones de memoria a las que salta el `NVIC`.

6. Práctica 5: Configuración de Timers

En esta práctica vamos a introducir el concepto de temporizador. Son unos periféricos del procesador que permiten, cada cierto tiempo, generar interrupciones y, con ello, disparar funcionalidades concretas que hayamos programado. En este caso programaremos uno para hacer parpadear un LED, y otro diferente para utilizar como *debouncer* del botón.

6.1. Objetivos

- Entender el funcionamiento de los temporizadores internos del sistema.
- Interconectar interrupciones de varios periféricos para lograr funcionalidades avanzadas.

6.2. Fundamentos teóricos

Un temporizador o *timer* en un microcontrolador es un periférico hardware que, en esencia, es un contador digital. La cuenta se realiza en función de una señal de reloj de entrada, y al llegar a ciertos valores (configurables) dispara interrupciones.

Es capaz de medir intervalos de tiempo de forma independiente a la ejecución de código en la CPU, liberándola pues para que haga otras tareas en lugar de esperas activas a ciertos eventos. Los temporizadores suelen contar con varios registros para su control, entre otros:

- `PSC` o *prescaler*: Un registro que sirve para dividir la frecuencia de entrada. Normalmente, la frecuencia del procesador están en el orden de MHz. El prescaler suele dividirlo al orden de KHz o incluso menos, según las necesidades de temporización.
- `CNT` o *counter*: El registro que lleva la cuenta actual del timer. Suma 1 cada vez que la señal de reloj, dividida entre el factor del prescaler, se activa. Su frecuencia de actualización f_{CNT} es por tanto:

$$f_{CNT} = \frac{f_{CLK}}{\texttt{PSC} + 1}$$

- `ARR` o *auto-reload*: Valor máximo de conteo. Cuando `CNT` alcanza este valor, se resetea y genera un evento de interrupción que podremos controlar. El periodo de evento T_{UEV} es por tanto:

$$T_{UEV} = \frac{\texttt{ARR} + 1}{f_{CNT}} = \frac{(\texttt{PSC} + 1)(\texttt{ARR} + 1)}{f_{CLK}}$$

- `CCR` o *capture compare*: Registros que permiten diversas configuraciones de comparación con CNT (mayor que, menor, igual ...) para medir la duración de señales externas o generar formas de onda PWM, entre otras funcionalidades. En ocasiones también permiten interupciones adicionales a la de `ARR`.

Podemos ver un diagrama de funcionamiento en la Figura 23.

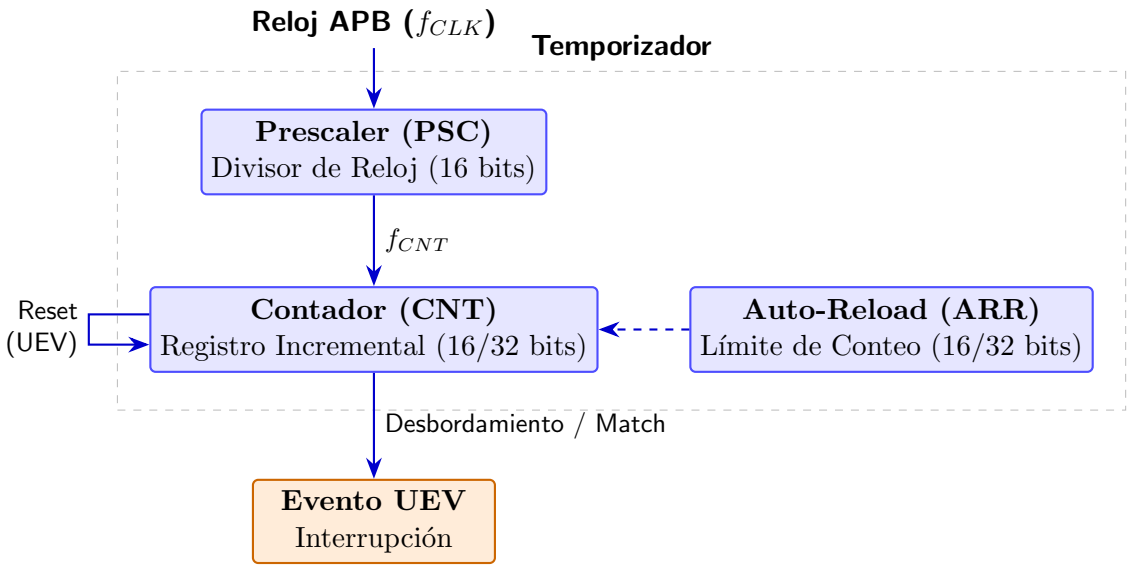


Figura 23: Diagrama de funcionamiento simplificado para los timers

6.3. Desarrollo de la práctica

Lo primero, como siempre, es ir añadiendo a nuestro `bsp.h` lo necesario para poder gestionar los timers. Vamos a utilizar dos, `TIM2` y `TIM3`. Consulta `rm0431 1.6` para ver las direcciones base de memoria, y `rm0431 19.4` para los registros de los temporizadores que permitan crear el `typedef` adecuado.

```
//... completar
#define TIM2_BASE_ADDR
#define TIM3_BASE_ADDR

//Structure for Timers (TIM2 / TIM3)
typedef struct
{
    volatile uint32_t CR1;        /*!< TIM control register 1,
        Address offset: 0x00 */
    volatile uint32_t CR2;        /*!< TIM control register 2,
        Address offset: 0x04 */
    //... completar
} TIM_TypeDef;

#define TIM2                ((TIM_TypeDef *) TIM2_BASE_ADDR)
#define TIM3                ((TIM_TypeDef *) TIM3_BASE_ADDR)
```

A continuación tenemos que crear las funciones necesarias para el control de los timers:

```
//funciones de configuración
//... activa en CR1 el bit correspondiente para activar el contador
static inline void TIM_ENABLE_COUNTER(volatile TIM_TypeDef *TIMx, uint32_t
    enable);
//... activa el modo oneshot desde CR1
static inline void TIM_ENABLE_ONESHOT(volatile TIM_TypeDef *TIMx, uint32_t
    enable);
//... configura el registro PSC con el valor de preescalado
static inline void TIM_SET_PRESCALER(volatile TIM_TypeDef *TIMx, uint32_t value)
    ;
//... configura el registro ARR con el valor de autoreload
static inline void TIM_SET_AUTO_RELOAD(volatile TIM_TypeDef *TIMx, uint32_t
```

```
    value);
//... activa la interrupción UIE en el registro DIER
static inline void TIM_ENABLE_INT(volatile TIM_TypeDef *TIMx);
//funciones de uso
//... borra el flag pendiente del registro SR (correspondiente a UIF)
static inline void TIM_CLEAR_FLAG(volatile TIM_TypeDef *TIMx);
//... comprueba si hay flag pendiente en el registro SR
static inline uint32_t TIM_CHECK_FLAG(volatile TIM_TypeDef *TIMx);
```

También es necesario crear una función que nos permita inicializar el timer, llamando a las anteriores de configuración:

```
//congiura prescaler, autoreload, activa counter y oneshot si es necesario y
    activa interrupciones
static void TIM_INIT(TIM_TypeDef *TIMx, uint32_t preescaler, uint32_t autoreload
    , uint32_t enable_counter, uint32_t enable_oneshot);
```

Y por último tendremos que activar los bits correspondientes en `APB1ENR` (`rm0431 5.3.13`), y adicionalmente activar en `NVIC` (`rm0431 9.1.2`) las líneas de interrupción adecuadas:

```
RCC_APB1ENR->bits.TIM2EN = 1;
RCC_APB1ENR->bits.TIM3EN = 1;
TIM_INIT(TIM2, 15999, 500, 1, 0); //500 milisegundos, activo
TIM_INIT(TIM3, 15999,  10, 0, 1); //10 milisegundos, inactivo, oneshot
//Enable interrupts
NVIC_ENABLE_INT(TIM2_IRQn);
NVIC_ENABLE_INT(TIM3_IRQn);
```

Ahora podemos, por fin, hacer uso de las funciones de los timers para ver su efecto:

```
// Handles Task 1: Blink LED1 (Green) each 0.5 seconds
void TIM2_IRQHandler(void) {
    //... completar comprobar que el flag está activo
    if () {
        //... completar borrar el flag
        //... completar funcionalidad deseada
    }
}
```

La funcionalidad que queremos se muestra en la Figura 24

- `TIM2` enciende o apaga el led verde (PB1) cada vez que se dispara. Para ello, asegúrate de haber activado dicho puerto y pin como `GPIO`.
- `TIM3` sirve como *debouncer* para el botón (en PA0). Para ello debemos conseguir que, al pulsarse el botón, se inhabilite durante el tiempo que tarde el timer en dispararse. Si tras ese tiempo el botón sigue pulsado, contaremos que es una pulsación de verdad. En caso contrario, lo omitiremos. En ambos casos, al terminar el temporizador volveremos a activar las interrupciones del botón. Como está en modo *oneshot*, cada vez que lo activemos se disparará solo una vez, evitando problemas.

Para el botón, debemos comprobar tras la activación que el bit correspondiente de `PR` está activo en `EXTI` (`rm0431 10.9.6`). Si es así, lo borramos y a continuación desactivamos la interrupción desde el registro `IMR` (`rm0431 10.9.1`). Finalmente activamos el temporizador, reseteando `CNT` a cero y activándolo desde `CR1` (`rm0431 19.4.1`).

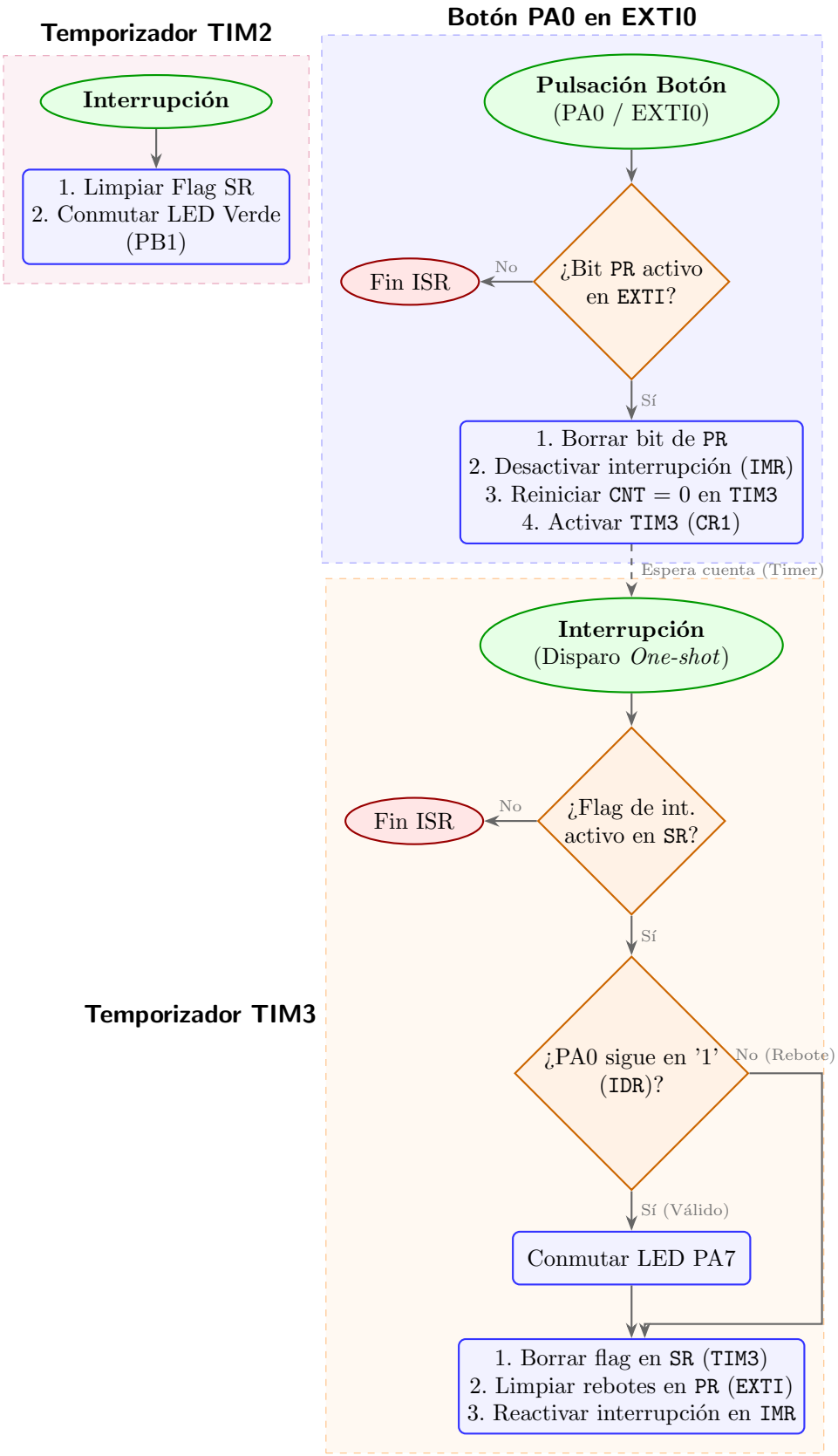


Figura 24: Diagrama de flujo funcional de las interrupciones.

```
void EXTI0_IRQHandler(void)
{
    //... comprobar si la interrupción está pendiente
    if () {
        //... borrar bit pendiente
        //... desactivar interrupciones de botón
        //... resetear cuenta de timer
        //... activar timer
    }
}
```

En cuanto al timer 3, debemos en primer lugar comprobar y borrar si es necesario el flag de interrupción desde `SR` (`rm0431 19.4.5`). A continuación, y si el botón sigue pulsado (podemos comprobarlo desde el registro `IDR` del `GPIO` (`rm0431 6.4.5`)), efectuar la función deseada (por ejemplo hacer parpadear el LED conectado a PA7). Tras ello, borraremos interrpciones pendientes del botón (por si se hubiera quedado pillado con rebotes) desde `PR` de `EXTI` (`rm0431 10.9.6`), y re-activaremos las interrupciones de botón en `IMR` (`rm0431 10.9.1`). Una cuestión importante es que, gracias a haber configurado el temporizador en modo *oneshot*, no es necesario desactivar sus propias interrupciones ya que no volverá a dispararse hasta reiniciar la cuenta.

```
// Debounce Lockout Timer Handler (Fires 10ms after initial edge)
void TIM3_IRQHandler(void) {
    //... comprobar flag de interrupción
    if () {
        //... borrar flag de interrpción
        //... verificar que sigue pulsado PA0
        if () {
            //... funcionalidad... Toggle led de PA7
        }
        //... borrar interrpciones pendientes para EXTI0 (botón)
        //... reactivar interrupciones para EXTI0
    }
}
```

6.4. Para hacer más

Aunque hay varios temporizadores (14 en concreto en el caso de **STM32F723IE**(`rm0431 18,19,20`)) para sistemas complejos con necesidad de medir una multitud de intervalos, pueden quedarse cortos. Para solventar este problema, cada temporizador es capaz de generar interrupciones en varios modos diferentes y con varios contadores distintos. Como todas estas interrupciones comparten ciertos aspectos de configuración, no podrá haber demasiada diferencia entre los tiempos que midan, aún así da mucha versatilidad. Investiga cómo puedes hacer que haya varias interrupciones diferentes desde un mismo temporizador (`rm0431 19`). (**NOTA:**) las interrupciones vendrán por la misma línea, y habrá que averiguar, programáticamente, qué parte del temporizador las causó (`rm0431 19.4.4,19.4.5`).

7. Práctica 6: Medida del tiempo de programa

7.1. Objetivos

- Entender la importancia de tener un *ticker* en el sistema, que provea de una referencia consistente del tiempo.
- Utilizar el *ticker* tanto para esperas como para medidas de tiempo entre eventos.

7.2. Fundamentos teóricos

Los *timers* son una herramienta extremadamente útil para tomar medidas de tiempo. Sin embargo, ofrecen únicamente una medida *local* del tiempo. Es decir, si se dispara un timer sabemos que fue hace un tiempo dado concreto, pero no tenemos más referencia temporal. Cuando se quiere saber el tiempo que pasa entre eventos, lo habitual es usar un *ticker* dentro del sistema que, desde un origen predifinido, va incrementado sistemáticamente un contador cada cierto tiempo dado. Así, mirando el valor del *ticker*, sabremos cuánto tiempo ha pasado desde cualquier otra referencia temporal, con solo calcular la diferencia entre ambos valores.

Hasta ahora, hemos tenido en ocasiones la necesidad de contar tiempo, y para ello hemos utilizado una función similar a esta:

```
void delay(int loops) {
    for (int i = 0; i < loops; i++) {
        __asm("nop");
    }
}
```

Básicamente introducimos un retardo con un bucle que da un cierto número de ciclos antes de terminar. El problema con esta aproximación es doble: por un lado, no sabemos cuánto tarda exactamente una vuelta del bucle (depende, en cierta medida, del número de instrucciones que genere el compilador). Por otro, aunque supiéramos el número de instrucciones, el tiempo que tarda dependerá de la frecuencia del reloj, aciertos en caché... En resumen, no es preciso.

7.3. Desarrollo de la práctica

En esta práctica vamos a utilizar el periférico `SYSTICK` del sistema (`pm0253 4.4`), que nos permite precisamente contar el número de eventos o *ticks* que han ocurrido desde el reseteo del procesador. El tiempo de un *tick* será configurado por nosotros.

El primer paso es definir el periférico junto a su dirección mapeada en memoria:

```
#define SYSTICK_BASE_ADDR       (0xE000E010UL)

typedef struct {
    volatile uint32_t CTRL;     // 0x00
    volatile uint32_t LOAD;     // 0x04
    volatile uint32_t VAL;      // 0x08
    volatile uint32_t CALIB;    // 0x0C
} SysTick_TypeDef;

#define SYSTICK             ((SysTick_TypeDef *) SYSTICK_BASE_ADDR)
```

A continuación debemos crear dos funciones para desactivar y activar el `SYSTICK`. Para ello da el valor adecuado a cada registro. En `LOAD`, ten en cuenta que tendrás que configurar cada cuánto tiempo quieres que se cuente un *tick*. Lo normal es que los *ticks* sean de 1ms. El reloj base es de 16MHz, así que calcula su valor en base a ello. En `VAL` configurarás el valor inicial del contador interno del *ticker*. Finalmente en `CTRL` deberás activar las interrupciones y el periférico, así como establecer la fuente de reloj al oscilador del procesador.

```
static void inline SysTick_Init(void) {
    // Default HSI clock is 16MHz on reset. 1ms tick = 16,000 cycles.
    // ... completar dar los valores adecuados a cada registro.
    SYSTICK->LOAD =
    SYSTICK->VAL  =
    SYSTICK->CTRL =
}

static void inline SysTick_Disable(void) {
    SYSTICK->LOAD = 0U;
    SYSTICK->VAL  = 0U;
    SYSTICK->CTRL = 0U;
}
```

Ahora que tenemos nuestro *ticker* configurado, falta saber en qué *tick* estamos. Para ello, tenemos que declarar una variable que poder consultar.

```
volatile uint32_t msTicks;

static uint32_t inline GetTick(void) {
    return msTicks;
}
```

¿Y... cómo enlazamos esta variable con el *ticker*? Previamente, hemos activado en `CTRL` de `SYSTICK` las interrupciones. Basta con capturar la interrupción del `SYSTICK` y actualizar nuestra variable de conteo:

```
void SysTick_Handler(void) {
    // ... completar añadir un tick a nuestra variable
}
```

Como sabemos que se dispara cada milisegundo, sabemos también que la cuenta de `msTicks` tendrá el valor, en milisegundos, desde que inicializamos `SYSTICK`.

Y ahora sí, estamos listos para sustituir a nuestra función de espera `delay`. Para ello, primero capturaremos el *tick* actual, y simplemente esperaremos a que se cumpla el tiempo aportado:

```
static void inline  Delay_ms(uint32_t ms) {
    // ... completar guardar tick actual y esperar en un bucle a que pasen
    // tantos ticks como se indican
}
```

Como la variable `msTicks` es un entero de 32 bits, tenemos suficiente margen como para contar unos ≈ 50 días. En ese momento, la cuenta volverá a cero. No es un problema en cualquier caso para diferencias relativas de tiempo, donde, aunque hayamos hecho *overflow*, la resta de dos tiempos siempre estará en ese rango de ≈ 50 días.

Con este contador de tiempo preparado, prepara ahora un código donde seas capaz de detectar si hay *dobles clicks* en un botón, tomando las referencias temporales con `GetTick()` cada vez que se pulse, y comprobando que dos consecutivas estén por debajo de los, por ejemplo,

500ms. Recuerda llamar a `Systick_Init()` desde tu código main. No es necesario que actives interrupciones en `NVIC` pues ya están por defecto activadas.

```
void TIM3_IRQHandler(void) {
    // Check if Update Interrupt flag is set
    if (TIM_CHECK_FLAG(TIM3)) {
        // Clear the timer interrupt flag
        TIM_CLEAR_FLAG(TIM3);

        // ... completar comprobar lógica de doble click
        // mirando el estado del botón y el tiempo transcurrido
        // tras la última pulsación

        EXTI_CLEAR_PENDING(0);
        EXTI_UNMASK_INTERRUPT(0, 1);
    }
}
```

7.4. Para hacer más

Hay un "temporizador" especial llamado `RTC` (*Real Time Clock*) capaz de llevar la cuenta de la hora y fecha actual de forma precisa. Con ello, además, puede generar interrupciones a largo plazo. Se utiliza principalmente para realizar esperas largas donde el procesador está dormido (potencialmente durante horas o días) mientras espera la interrupción. Consulta la documentación (`rm0431 25`) y utilízalo para hacer parpadear el led azul (PA5) cada 5 segundos.

8. Práctica 7: Conexión con periféricos externos - Display y drivers

8.1. Objetivos

- Realizar conexión con periféricos externos, en concreto el display.
- Programar un driver sencillo para controlar el pintado.

8.2. Fundamentos teóricos

Cualquier procesador, sea del tipo que sea, se acompaña siempre, además de los periféricos internos, de periféricos externos. De lo contrario, la interfaz de comunicación con el usuario resultaría imposible. Desde simples pines con los que comunicar vía `UART`, como ya hemos visto, hasta sistemas complejos como los que veremos en esta práctica. Las conexiones de las que dispone nuestra placa **32F723EDISCOVERY**se pueden observar en `um2140 5`, aunque las reproducimos en la Figura 25 para facilitar la consulta.

En concreto, observando dicha imagen, vemos que el display (a la izquierda) se conecta con la placa a través de GPIOs, y un periférico interno conocido como `FMC`. El `FMC` o *Flexible Memory Controller* (`rm0431 12`) es un periférico que permite gestionar la conexión con memorias externas de diferentes tipos (SDRAM, NAND flash, PSRAM, NOR flash...) además de dispositivos basados en memorias, como los displays (que guardan la imagen de manera similar a como una memoria guarda datos). Esto libera la memoria interna del **STM32F723IE**para poder guardar programas, en lugar de pesadas imágenes.

Dichas conexiones requieren el manejo simultáneo de múltiples pines de entrada y salida, lo cual sería imposible de realizar directamente con el procesador. Por tanto, este periférico `FMC` permite, con una configuración apropiada, gestionar las transferencias de datos de manera autónoma. Podemos efectivamente comprobar que el display está conectado al `FMC` observando `um2140 5.14`. Adicionalmente, en `um2140 6.7`, nos dice qué pines se utilizan para las conexiones. (Estos nombres no son directamente los puertos GPIO de la placa, que se pueden consultar en `um2140 Table 19`.) Cruzando estos datos con `rm0431 12.3`, podemos ver qué señales concretas se utilizan para esta placa concreta.

Pero hay una peculiaridad, y es que el `FMC` en este caso no se comunica directamente con el display, sino que, de por medio, hay un chip controlador `ST7789H2`. Este chip está encargado de recibir comandos a través del `FMC`, configurando la memoria del display, así como sus características. Así pues, toda la información o comandos que queramos pasar al display deberán pasar por el `FMC`.

Adicionalmente, el propio panel LCD (modelo `FRD154BP2901` de *Frida*) cuenta con unas conexiones especiales que no controla el `ST7789H2`. Estas conexiones, concretamente, son tres pines (`um2140 Table 14`): PH7 (resetea el panel), PH11 (se encarga de encender la luz trasera) y PC8 (Interrupción de Tear Effect, no hace falta utilizar este último).

Es decir, que parte del control lo haremos directamente, y parte a través del periférico `FMC`. Podemos ver todas estas conexiones de manera simplificada en Figura 26.

Ahora, con todas las conexiones hechas, hay que activar el panel. Para activar periféricos, se siguen ciertas secuencias de inicialización que ponen el hardware en un estado inicial. Aunque

Figure 3. Hardware block diagram

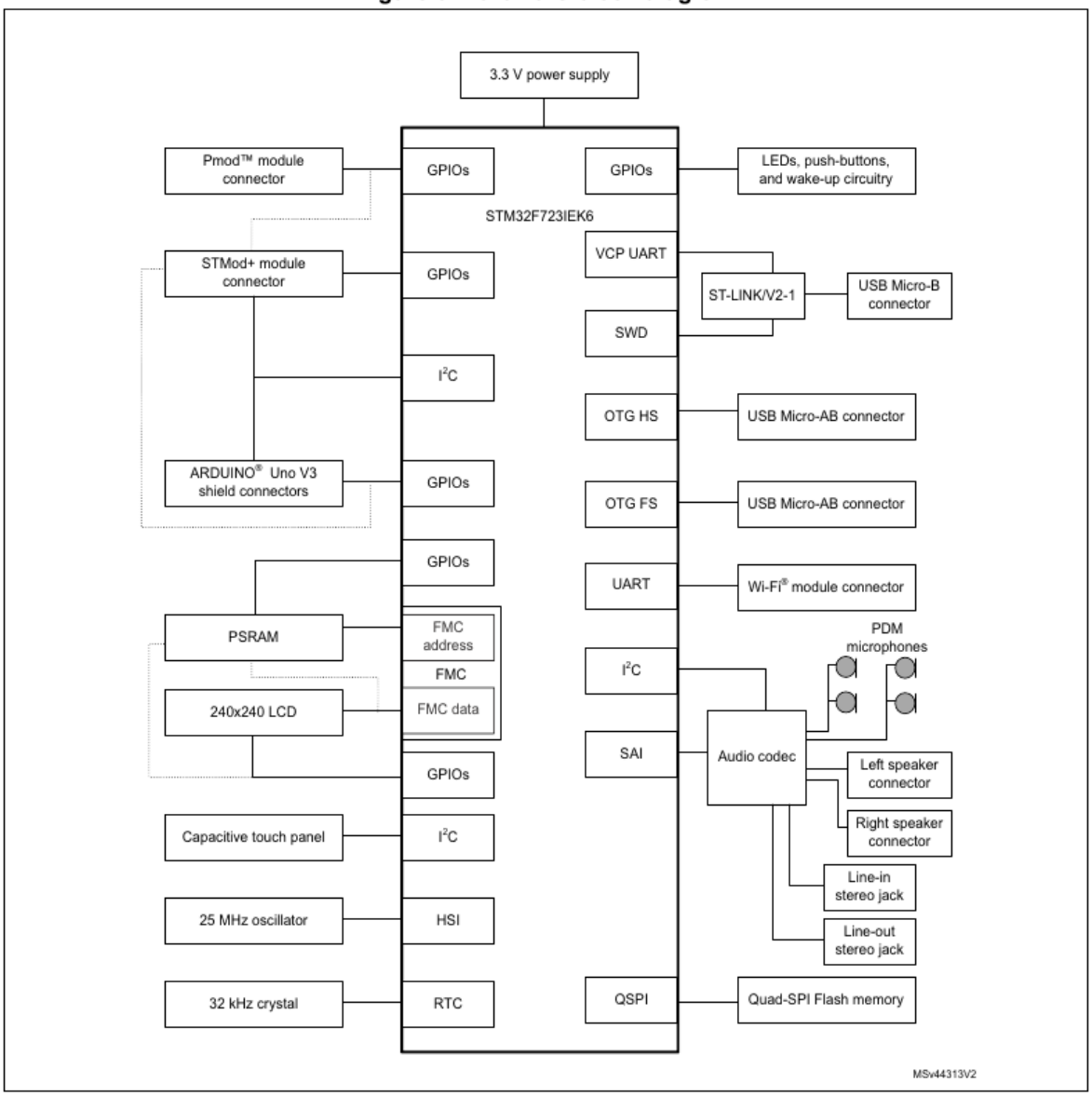


Figura 25: Diagrama de bloques de la placa **32F723EDISCOVERY**

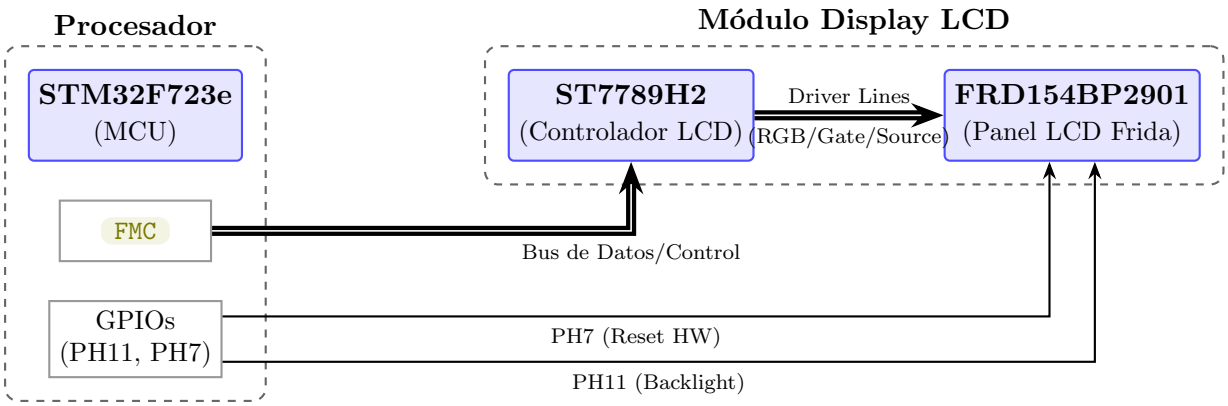


Figura 26: Diagrama de conexiones entre la MCU STM32F723, el controlador ST7789H2 y el panel LCD.

la documentación no es muy clara, si os fiáis de vuestro profesor, y no habéis abandonado la asignatura por dificultad (espero que no, si es así contactadme), el proceso no es excesivamente complicado. Habrá primero que activar el panel a través de los pines de conexión directa (básicamente, resetearlo), y luego activar el controlador `ST7789H2` utilizando comandos enviados por el `FMC`. En concreto, la secuencia necesaria de encendido es:

1. Encender la luz trasera del display desde el pin PH11 (esto es opcional, pero querremos ver la pantalla, no?).
2. Aplicar la secuencia de reset en `PH7`: `0` (esperar 5 ms) → `1` (esperar 10 ms) → `0` (esperar 20 ms) → `1` (esperar 10 ms).
3. Con el panel encendido, toca el turno al controlador, al que tenemos que mandar un comando de `SWRESET`.
   a) Mandar un comando de `SLEEP_IN` `ST7789H2 9.1.11` para asegurar que el panel está desactivado. La documentación obliga a esperar 5ms, esperamos 10ms para asegurar.
   b) Mandar un comando de `SWRESET` `ST7789H2 9.1.2` para resetear el panel. La documentación obliga a esperar 120ms, esperamos 200ms para asegurar.
   c) Mandar un comando de `SLEEP_OUT` `ST7789H2 9.1.12` para activar el panel. La documentación obliga a esperar 5ms, pero como menciona 120ms también, esperamos ese tiempo para asegurar.

Lo más interesante aquí es que estos comandos que debe recibir el panel LCD, se van a enviar a través del `FMC`. Es decir, que estamos delegando la configuración en este periférico versátil, en lugar de tener que hacerlo nosotros a través de los aproximadamente 30 pines que lo controlan. Y ahora, cómo funciona todo esto?

8.2.1. Funcionamiento del periférico FMC

El periférico `FMC` funciona, a ojos del programador, como una extensión de la memoria RAM del sistema. En lugar de requerir un protocolo de comunicación complejo con llamadas a funciones de transmisión bit a bit, el `FMC` mapea el controlador externo directamente en el mapa de memoria del microcontrolador (`rm0431 12.4`).

Esto significa que escribir o leer en el controlador de pantalla es tan sencillo como hacer una operación de escritura en una dirección de memoria en C. El propio hardware del `FMC` se encarga de traducir esa escritura en las señales eléctricas apropiadas (bus de datos, activación de `CS`, pulsos de `WR/RD`, etc.) a través de los pines GPIO asignados. En este caso, únicamente se mapean dos direcciones: `REG` (para indicar el registro o comando del periférico) y `RAM` (para indicar el valor que se escribe):

```
typedef struct
{
    volatile uint16_t REG;
    volatile uint16_t RAM;
} FMC_CONTROLLER_TypeDef;

#define FMC_BANK2           ((FMC_CONTROLLER_TypeDef *) FMC_BANK2_BASE_ADDR)

static inline void FMC_BANK2_WriteReg(uint8_t Reg)
{
```

```
    /* Write 16-bit Index, then write register */
    FMC_BANK2->REG = Reg;
    __DSB();
}

static inline void FMC_BANK2_WriteData(uint16_t Data)
{
    /* Write 16-bit Reg */
    FMC_BANK2->RAM = Data;
    __DSB();
}
```

Para comunicarse con el ST7789H2, el FMC utiliza una interfaz de tipo 8080/6800 paralela de tipo SRAM. Ésta contiene líneas que doblan para datos o comandos, y su información se discierne mediante una línea especial A0 (o Data/Command Line) (um2140 Table 14):

- Cuando la línea A0 está a nivel bajo (0), la transferencia se interpreta como un comando o instrucción para el controlador.
- Cuando la línea A0 está a nivel alto (1), la transferencia se interpreta como datos (parámetros del comando o colores de los píxeles).

Gracias al mapeo en memoria, en este caso en el banco 2 (BANK2) del FMC, diferenciar entre enviar un comando o un dato es tan simple como escribir en dos direcciones de memoria consecutivas dentro del rango asignado al FMC. La dirección base (REG) enviará la línea A0 a 0 (Comandos), mientras que la siguiente (RAM) enviará A0 a 1 (Datos).

8.2.2. Funcionamiento del controlador ST7789H2

El chip ST7789H2 es el "cerebro" interno del panel LCD. Su función principal es mantener la imagen visible en la pantalla refrescando continuamente los píxeles físicos, liberando por completo al **STM32F723IE**.

Para lograr esto, el ST7789H2 contiene una memoria RAM interna de 240×320 localizaciones de memoria, que se configura mediante secuencias de comandos y datos concretas a través del FMC. El flujo general de trabajo con este chip consta de tres etapas:

1. **Configuración e Inicialización:** Tras la secuencia de reset, se le envían comandos mediante el FMC para definir el formato de color (por ejemplo, RGB565 de 16 bits por píxel), orientación de la pantalla (lectura vertical u horizontal), etc.
2. **Definición de Ventana de Trabajo:** Antes de pintar, se envía un comando de dirección de columnas (CASET, ST7789H2 9.1.20) y de filas (RASET, ST7789H2 9.1.21). Esto le dice al ST7789H2 en qué sub-región o rectángulo de la pantalla vamos a escribir.
3. **Escritura de Píxeles:** Una vez delimitada la zona, se envía el comando de escritura en memoria (RAMWR, ST7789H2 9.1.22). A partir de ese momento, cada dato enviado por el FMC corresponderá al color de un píxel (se pueden enviar varios en ráfaga). El propio controlador incrementa automáticamente la dirección de memoria interna píxel a píxel, llenando la ventana definida de izquierda a derecha y de arriba a abajo.

Es decir, el procesador escribe los colores a cada píxel en la memoria mediante el `FMC`, el `ST7789H2` los almacena en su RAM interna y sus circuitos de control (drivers (no confundir con driver software)) convierten esos valores digitales en las tensiones físicas que configuran el display `FRD154BP2901`. Un diagrama mostrando este funcionamiento se observa en la Figura 27.

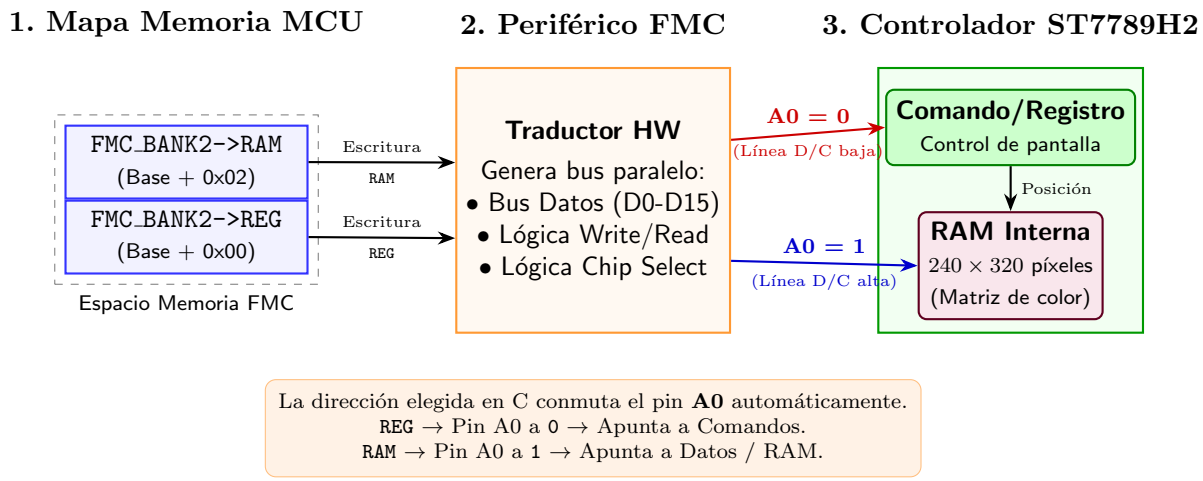


Figura 27: Mapeo de memoria a través del FMC hacia la lógica y RAM interna del ST7789H2

8.3. Desarrollo de la práctica

Para esta práctica, vamos a partir del código final de `main.c` al que queremos llegar. Básicamente, inicializaremos todos los elementos necesarios (el panel LCD, el periférico `FMC`, y el controlador `ST7789H2`), antes de dibujar algo sencillo en la pantalla. Se incluye además un pequeño código para poner píxeles aleatorios y ver que de verdad está funcionando:

```
// Definimos una variable global para la semilla (debe ser distinta de 0)
static uint32_t estado_rng = 123456789;

// Genera un entero sin signo de 32 bits (0 a 4,294,967,295)
uint32_t rand32(void) {
    uint32_t x = estado_rng;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return estado_rng = x;
}

int main(void)
{
    LCD_Init();
    FSMC_Init();
    ST7789H2_INIT();

    /* Clear the LCD */
    for (int i = 0; i < 240; i++)
        for (int j = 0; j < 240; j++)
            ST7789H2_WritePixel(i, j, LCD_COLOR_WHITE);


    for (int i = 50; i < 60; i++)
        for (int j = 50; j < 60; j++)
            ST7789H2_WritePixel(i, j, LCD_COLOR_RED);

```

```
29
30      while(1)
31      {
32          uint32_t rnd = rand32();
33          ST7789H2_WritePixel((rnd >> 24) & 0xff, (rnd >> 16) & 0xff, rnd & 0xffff
                );
34      }
35  }
```

Yendo por orden, lo primero es inicializar el panel LCD. Para ello hay que configurar los pines de conexión directa (de reset, interrupción (que no utilizaremos) y luz trasera). Posteriormente, aplicaremos la secuencia de inicialización del panel.

```
1   void LCD_Pins_Init() {
2       /* Initialize LCD special pins GPIOs */
3       /* LCD_RESET GPIO configuration */
4       RCC_AHB1ENR->bits.GPIOHEN = 1;
5       GPIO_CONFIG(GPIOH, 7, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
            GPIO_PUPD_NONE, 0);
6       /* LCD_TE GPIO configuration */
7       RCC_AHB1ENR->bits.GPIOCEN = 1;
8       GPIO_CONFIG(GPIOC, 8, GPIO_MODE_INPUT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
            GPIO_PUPD_NONE, 0);
9       /* LCD_BL_CTRL GPIO configuration */
10      RCC_AHB1ENR->bits.GPIOHEN = 1;
11      GPIO_CONFIG(GPIOH, 11, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP, GPIO_OSPEED_LOW,
            GPIO_PUPD_NONE, 0);
12  }
13
14  void LCD_Startup_Seq() {
15      /* Backlight enable */
16      GPIO_PIN_WRITE(GPIOH, 11, GPIO_STATE_ONE);
17
18      /* Apply hardware  */
19      //... completar escribir en el pin de reset la secuencia de activación
20      //... utiliza la función DELAY para las esperas
21  }
22
23  void LCD_Init() {
24      LCD_Pins_Init();
25      LCD_Startup_Seq();
26  }
```

Una vez iniciado el panel, lo siguiente es activar el `FMC`. La configuración es bastante complicada y ya está hecha en el código proporcionado de base. Se recomienda echar un vistazo. En resumen, lo que hacemos es activar primero todos los pines requeridos por el controlador, y luego configurar el modo de conexión con la memoria, así como las características de temporización del bus.

Finalmente, se configura el controlador del display, enviando comandos directamente desde el `FMC`. El código se encuentra en `ST7789h2.h`, y hay que completar algunas partes para hacerlo funcionar completamente. En concreto:

En la función `ST7789H2_INIT`, al principio, debemos inicializar el controlador, mandando los comandos adecuados:

```
1   /* Sleep In Command */
2   ST7789H2_WriteReg(ST7789H2_SLEEP_IN, (uint8_t*)NULL, 0);
3   /* Wait for 10ms */
4   DELAY(10);
```

```
5
6  //... completar mandar el comando de RESET, seguido de una pausa de 200ms
7  //... completar mandar el comando de SLEEP_OUT, seguido de una pausa de 120ms
```

También deberemos crear la función `DISPLAY_ON` para encender el display:

```
1  void ST7789H2_DisplayOn(void)
2  {
3      //... completar mandar el comando DISPLAY_ON
4      //... completar mandar el comando SLEEP_OUT
5  }
```

Con esto ya deberíamos tener el display funcionando! El código debería borrar el display a blanco (tardará un poco), pintar un cuadrado rojo, y luego empezar a dibujar cuadrados de colores aleatoriamente.

8.3.1. Creación de la Librería Gráfica (GFX Library)

Dibujar gráficos solo con `ST7789H2_WritePixel()` resulta tedioso. Para abstraer el dibujado, se diseñan librerías gráficas que faciliten el proceso.

Esta librería (implementada por ejemplo en dos archivos llamados `gfx.h` y `gfx.c`) debe apoyarse directamente sobre la función de bajo nivel `ST7789H2_WritePixel(x, y, color)` proporcionada por el driver del display. A continuación se proporciona un **ejemplo** de qué funciones podrían implementarse, si bien puedes elegir las que más te convengan.

Fichero de cabecera (`gfx.h`): Crea el fichero `gfx.h` con las definiciones necesarias y los prototipos de las funciones:

```
1  #ifndef GFX_H_
2  #define GFX_H_
3
4  #include <stdint.h>
5  #include <stdlib.h>
6  #include "ST7789h2.h"
7
8  #define GFX_WIDTH   240
9  #define GFX_HEIGHT  240
10
11 /* Primitivas basicas con verificacion de limites */
12 void GFX_DrawPixel(int16_t x, int16_t y, uint16_t color);
13 void GFX_DrawFastHLine(int16_t x, int16_t y, int16_t w, uint16_t color);
14 void GFX_DrawFastVLine(int16_t x, int16_t y, int16_t h, uint16_t color);
15
16 /* Lineas genericas */
17 void GFX_DrawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t color
       );
18
19 /* Cuadrados y Rectangulos */
20 void GFX_DrawRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);
21 void GFX_FillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color);
22 void GFX_DrawSquare(int16_t x, int16_t y, int16_t size, uint16_t color);
23 void GFX_FillSquare(int16_t x, int16_t y, int16_t size, uint16_t color);
24
25 /* Circulos */
26 void GFX_DrawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color);
```

```
void GFX_FillCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color);

/* Control de pantalla */
void GFX_FillScreen(uint16_t color);

#endif /* GFX_H_ */
```

Implementación (`gfx.c`): A continuación se describen los requisitos algorítmicos que debes codificar dentro de `gfx.c`:

1. **Trazado rápido y comprobación de límites (`GFX_DrawPixel`, `GFX_DrawFastHLine`, `GFX_DrawFastVLine`):** Implementa `GFX_DrawPixel` asegurando que no se dibujen coordenadas fuera del rango de la pantalla. Igualmente implementa funciones rápidas (un solo bucle) para dibujar líneas horizontales y verticales.

2. **Algoritmo de Línea de Bresenham (`GFX_DrawLine`):** Para dibujar líneas arbitrarias entre (x_0, y_0) y (x_1, y_1), debes implementar el algoritmo de *Bresenham*. Este algoritmo utiliza únicamente aritmética entera para calcular los píxeles intermedios:

```
void GFX_DrawLine(int16_t x0, int16_t y0, int16_t x1, int16_t y1, uint16_t
    color) {
    int16_t dx = abs(x1 - x0);
    int16_t dy = abs(y1 - y0);
    int16_t sx = (x0 < x1) ? 1 : -1;
    int16_t sy = (y0 < y1) ? 1 : -1;
    int16_t err = dx - dy;

    while (1) {
        GFX_DrawPixel(x0, y0, color);
        if (x0 == x1 && y0 == y1) break;
        int16_t e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 < dx)  { err += dx; y0 += sy; }
    }
}
```

3. **Dibujado de Rectángulos y Cuadrados:**
   - `GFX_DrawRect`: Dibuja solo el borde de un rectángulo mediante líneas horizontales y verticales. Reaprovecha las funciones anteriores.
   - `GFX_FillRect`: Rellena el área interna dibujando múltiples líneas horizontales.
   - `GFX_DrawSquare` y `GFX_FillSquare`: Reutilizan las funciones para un rectángulo con lados iguales.

4. **Algoritmo del Punto Medio para Círculos (`GFX_DrawCircle` y `GFX_FillCircle`):** Para dibujar el contorno de un círculo de radio r centrado en un punto (x_0, y_0), existe el denominado *algoritmo del punto medio de Bresenham* (este tío debió hacer muchas cosas por los gráficos 2D sencillos). La idea detrás del mismo es dibujar solo un arco de 45 grados, y utilizar la simetría en octantes del círculo (básicamente, un círculo se forma de 8 partes iguales, como una pizza). Para la versión rellena (`GFX_FillCircle`), sustituye los puntos individuales por líneas verticales que unan los límites simétricos superiores e inferiores:

```
void GFX_DrawCircle(int16_t x0, int16_t y0, int16_t r, uint16_t color) {
    int16_t f = 1 - r;
    int16_t ddF_x = 1;
```

```
    int16_t ddF_y = -2 * r;
    int16_t x = 0;
    int16_t y = r;

    GFX_DrawPixel(x0, y0 + r, color);
    GFX_DrawPixel(x0, y0 - r, color);
    GFX_DrawPixel(x0 + r, y0, color);
    GFX_DrawPixel(x0 - r, y0, color);

    while (x < y) {
        if (f >= 0) {
            y--;
            ddF_y += 2;
            f += ddF_y;
        }
        x++;
        ddF_x += 2;
        f += ddF_x;

        GFX_DrawPixel(x0 + x, y0 + y, color);
        GFX_DrawPixel(x0 - x, y0 + y, color);
        GFX_DrawPixel(x0 + x, y0 - y, color);
        GFX_DrawPixel(x0 - x, y0 - y, color);
        GFX_DrawPixel(x0 + y, y0 + x, color);
        GFX_DrawPixel(x0 - y, y0 + x, color);
        GFX_DrawPixel(x0 + y, y0 - x, color);
        GFX_DrawPixel(x0 - y, y0 - x, color);
    }
}
```

Comprobación en `main.c`: Añade pruebas en `main.c`, incluyendo tu librería `"gfx.h"` y ejecutando un test completo que dibuje líneas diagonales, rectángulos, cuadrados, círculos... de distintos colores.

8.4. Para hacer más

Explora la documentación del display para estudiar las maneras más rápidas de pintar la pantalla (por líneas en lugar de píxeles, por ejemplo). También puedes aumentar la velocidad del procesador cambiando su frecuencia de reloj de los 16MHz por defecto a más de 200MHz para conseguir mayor velocidad.

9. Práctica 8: Buses de Expansión - I2C y pantalla táctil

9.1. Objetivos

- Entender el funcionamiento básico del bus I2C.
- Programar un driver para la comunicación con la pantalla táctil a través de I2C.

9.2. Fundamentos teóricos

Si bien los procesadores cuentan con periféricos internos con funcionalidades muy amplias (temporización, controles de memoria, interrupciones, protocolos básicos de comunicación...), hay ciertas funciones que no pueden tener integradas. Por ejemplo, sería muy complicado tener un periférico de pantalla integrado, ya que cada pantalla es un mundo y habría que limitar los dispositivos soportados. Lo mismo ocurriría con audio, teclados, ratones, actuadores...

Es decir, para actuar con el mundo .exterior.al procesador, no podemos contar con periféricos específicos. En su lugar, se suelen ofrecer periféricos genéricos que, utilizando protocolos estandarizados, se pueden conectar con una gran cantidad de elementos externos. Así, el trabajo de controlar esos periféricos externos será ahora del programador *software*, mediante la escritura de los *drivers* necesarios.

En nuestro caso, vamos a utilizar el bus **I2C** para la conexión con el panel táctil. Este bus utiliza dos líneas, **SCL** para transmitir una señal de reloj, y **SDA** para transmitir los datos. Permite, a través de identificación mediante direcciones (enviando parejas dirección-dato por la línea **SDA**), la interconexión de múltiples dispositivos. Su extrema sencillez y flexibilidad han hecho que se haya popularizado ampliamente.

9.2.1. Conexiones en nuestra placa

Podemos ver el pinout completo de las conexiones procesador-pantalla táctil en `um2140 6.8`. El conector a la pantalla táctil tiene señales **SCL**, **SDA**, **INT**, **RESET**, además de varias conexiones para alimentación y tierra. Estas últimas son pasivas y no requieren de control por parte del procesador.

Las que sí requieren de control, se especifican en `um2140 Appendix A`. Ahí, podemos ver las conexiones que se especifican en la Tabla 1, con un esquema ilustrativo en la Figura 28.

Pin	Function	Connection
PA8	I2C3 SCL	CTP SCL
PH8	I2C3 SDA	CTP SDA
PH9	GPIO OUTPUT	CTP RST
PI9	GPIO EXTI9	CTP INT

Tabla 1: Conexiones para la pantalla. CTP: *Connector Touch Panel*. Los pines PA8 y PH8 no vienen completamente especificados en `um2140`, pero se incluyen aquí por completitud.

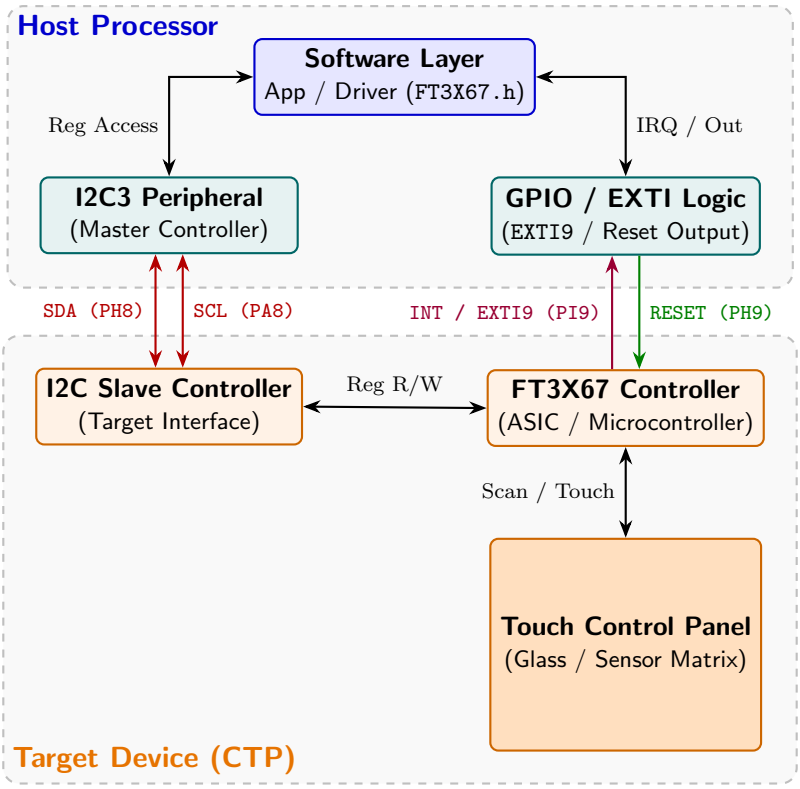


Figura 28: Esquema de comunicación entre el procesador principal, el controlador I2C y el ASIC del panel táctil.

9.2.2. Funcionamiento del periférico I2C

El periférico que controla el bus **I2C** se encuentra descrito en `rm0431 26`. Más allá de la configuración concreta de los registros internos, para controlar los parámetros de **I2C** como frecuencia, funcionamiento y demás, el periférico ofrece dos funciones principales:

1. Enviar un dato a un registro concreto de un dispositivo externo concreto.
2. Recibir un dato de un registro concreto de un dispositivo externo concreto.

Esto se hará más adelante mediante sendas funciones de escritura y lectura (driver) que utilizará el `I2C` para la comunicación con el dispositivo externo de panel táctil. Estas funciones, interactuando con los registros internos del controlador `I2C`, harán transparente para el usuario los detalles de transacción dentro del bus.

9.3. Desarrollo de la práctica

Llegados a este punto, sólo nos queda comunicarnos con el panel táctil. Haremos una aplicación donde el usuario "pintará" con el dedo en la pantalla. Para ello, partiremos de la práctica anterior, donde ya teníamos el controlador de la pantalla funcionando. En lugar de pintarse aleatoriamente, ahora controlaremos nosotros el pintado. Para ello, habrá que hacer lo siguiente:

- Configurar el bus I2C para poder comunicarnos con el panel táctil (CTP).

- Activar las interrupciones adecuadas para recibir eventos del CTP.
- Crear el driver que nos permita leer los eventos y transformarlos a coordenadas para pintar en la pantalla.

Vamos por orden:

9.3.1. Configuración del bus I2C

Lo primero que debemos hacer es activar los pines adecuados para la comunicación. En primer lugar, los pines **SCL** y **SDA** (PA8 y PH8) deben configurarse en modo alternativo con función 4 (esto los conecta al I2C3, ver `ds11853 Table 12`), modo open drain, sin pull up y velocidad alta. Además, hay que asegurarse de activar `GPIOAEN` y `GPIOBEN` en `AHB1ENR` (`rm0431 5.3.10`).

Por otro lado, debemos asegurarnos que el CTP no está en modo RESET, para ello, debemos activar el GPIO para el pin PH9 (Correspondiente a CTP RESET (Tabla 1)). Este pin debe configurarse en modo salida, push-pull sin pullup, y velocidad baja. A continuación debemos escribir un "1" en él para desactivar el RESET (que es en lógica inversa).

```
void I2C3_Pins_Init() {
    //Pin A8 tiene I2C_SCL
    // ... completar
    //Pin H8 tiene I2C_SDA
    // ... completar
    //Pin H9 para el reset del CTP
    // ... completar
}
```

Con esto los pines ya están activos, y nos toca configurar el periférico I2C. Las definiciones de los registros que contiene se pueden ver en `rm0431 26.9`, y la dirección base se puede encontrar en `rm0431 1.6.2`. Se ofrece a continuación el código para `bsp.h` con las definiciones y algunas funciones predefinidas creadas:

```
#define I2C3_BASE_ADDR 0x40005C00UL

typedef struct
{
    volatile uint32_t CR1;      /* Control register 1,            */
    volatile uint32_t CR2;      /* Control register 2,            */
    volatile uint32_t OAR1;     /* Own address 1 register,        */
    volatile uint32_t OAR2;     /* Own address 2 register,        */
    volatile uint32_t TIMINGR;  /* Timing register,               */
    volatile uint32_t TIMEOUTR; /* Timeout register,              */
    volatile uint32_t ISR;      /* Interrupt and status register, */
    volatile uint32_t ICR;      /* Interrupt clear register,      */
    volatile uint32_t PECR;     /* PEC register,                  */
    volatile uint32_t RXDR;     /* Receive data register,         */
    volatile uint32_t TXDR;     /* Transmit data register,        */
} I2C_TypeDef;

#define I2C3                ((I2C_TypeDef *) I2C3_BASE_ADDR)

static inline void I2C_Disable(I2C_TypeDef * I2Cx) {
    I2Cx->CR1 &= ~(0x1);
}

```

```
24 static inline void I2C_Enable(I2C_TypeDef * I2Cx) {
25     I2Cx->CR1 |=  (0x1);
26 }
27
28 static inline void I2C_Set_Timing(I2C_TypeDef * I2Cx, uint32_t presc, uint32_t
       scldel, uint32_t sdadel, uint32_t sclh, uint32_t scll) {
29     I2Cx->TIMINGR = (presc << 28) | (scldel << 20) | (sdadel << 16) | (sclh <<
           8) | (scll);
30 }
```

En lo que a nosotros respecta, debemos utilizar estas funciones para inicializar el periférico `I2C3` (`rm0431` 26). En `rm0431 26.4.5`, tenemos un diagrama que nos indica cómo proceder para configurar el controlador I2C. Como no necesitamos todas las funciones, lo haremos algo más sencillo.

Lo primero será activar el periférico `I2C3` desde `APB1ENR` (`rm0431 5.3.13`), para a continuación aplicar una secuencia de reseteo desde `APB1RST` (`rm0431 5.3.8`) consistente en escribir un 1 y luego un 0 sobre el bit de `I2C3`. Tras esto, ya podemos configurar el periférico. Para ello se inhabilita (ya desde dentro de su registro `I2C_CR1`), se configura el temporizador `I2C_TIMINGR`, y se vuelve a habilitar.

Para la temporización, hay numerosos campos. Tenemos que configurarlos teniendo en cuenta que la frecuencia base son 16MHz. La frecuencia objetivo que queremos son 400KHz. Hay muchas maneras de conseguirla, tenemos que tener en cuenta que la fórmula es la siguiente:

$$f_{obj} = \frac{f_{base}}{(PRESC + 1) * (SCLL + 1 + SCLH + 1)}$$

`SCLL` y `SCLH` indican el tiempo que la señal de reloj pasará, respectivamente, a baja (low) y alta (high). Podemos ponerlos iguales para conseguir una señal cuadrada. El valor de `PRESC` es un preescaler que nos divide la señal de reloj por si fuera muy alta. En nuestro caso, podemos dejar el prescaler a 0, y poner los otros dos registros a 19, para conseguir una onda cuadrada exacta a 400KHz. Los valores de `SCLDEL` y `SDADEL` controlan los desfases entre las subidas y bajadas de **SCL** y **SDA**. En general son valores bastante flexibles, aunque sus detalles son complejos y pueden explorarse más en `rm0431 26.9.5` y `rm0431 26.4.5`. Para nuestro caso, con 3 y 1 respectivamente debería funcionar.

```
1  void I2C3_Reg_Config() {
2      //Enable I2C3 and reset cycle
3      // ... completar
4
5      //Configure I2C
6      I2C_Disable(I2C3);
7      // Set Timing
8      // ... completar
9      I2C_Enable(I2C3);
10 }
```

9.3.2. Interrupciones del CTP

A continuación debemos activar las interrupciones del panel táctil. Como vimos en Tabla 1 tiene un pin dedicado para interrumpir al procesador, a través de la línea de interrupción externa `EXTI9`. Debemos configurar el procesador para atender esa petición. Recordemos que cada

línea de interrupción externa `EXTIx` puede conectarse a cualquiera de los puertos (PA, PB, ...) (rm0431 10). En este caso, como interrumpimos desde `PI9`, debemos configurar `EXTI9` para que atienda al puerto `I` en modo falling edge. Además, debemos indicar desde `SYSCFG` en el registro `EXTICR` (rm0431 7.2.3) que el puerto I (8) es el que se atiende en `EXTI9`, y activar `SYSCFG` desde el bit de enable de `APB2ENR` (rm0431 5.3.19). Finalmente, tenemos que activar la interrupción correspondiente en el `NVIC` (rm0431 9.1.2).

Hay que tener en cuenta que las interrupciones externas están agrupadas (en concreto, de la 5 a la 9 se atienden juntas (rm0431 9.1.2) a través de la línea de interrupción número 23 del procesador). Así que, en nuestra rutina de interrupción, tendremos que asegurarnos de que fue la interrupción externa 9 la que causó la interrupción a través del registro `PR` de `EXTI` (rm0431 10.9.6)

```
void TS_INT_Init(void) {
    // ... completar activar GPIO PI9 como entrada pull-up, velocidad rápida

    /* Enable and Map EXTI Line 9 to GPIOI (0b1000 = Port I) */
    // ... completar activar SYSCFGEN en APB2ENR
    // ... configurar int 9 para atenderse desde PI en EXTICR
    // ... activar interrupción EXTI9 en EXTI
    // ... habilitar modo falling edge y deshabilitar rising edge
    // ... activar interupción 23 (EXTI9_5) en NVIC
}

void EXTI9_5_IRQHandler(void) {
    // ... completar comprobar bit 9 de EXTI PR
    if () {
        // ... limpiar bit 9 de EXTI PR
        g_ts_interrupt_flag = 1;    // Notify main thread
    }
}
```

Con esto, ya tendremos las interrupciones disparando ante eventos del panel táctil. De hecho, puedes probar a ejecutar el código con un *breakpoint* en la interrupción, y deberías ver que se dispara al tocar la pantalla, aunque no recojamos dato alguno.

9.3.3. Driver del I2C

Nos falta leer los datos correspondientes a la posición, y hacer algo con ellos. Para ello necesitamos dos cosas:

- Un *driver* para el periférico `I2C` que nos permita leer/escribir a través del bus (programaremos ambas funciones, aunque sólo utilizaremos la primera).
- Otro *driver* que, utilizando el primero, se comunique con el panel táctil para leer la posición de la pulsación.

Lo primero de todo es crear el driver para `I2C` (rm0431 26). Vamos a empezar por unas cuantas funciones auxiliares para el control de los diversos registros internos. Analízalas bien, viendo los registros que utilizan y el manual, para comprender su funcionamiento:

```
static inline void I2C_Write(I2C_TypeDef *I2Cx, uint8_t data) {
    // Wait for TXDR to be empty (TXIS == 0)
    while (!(I2Cx->ISR & (1U << 1U)));
```

```
    // Load byte into Transmit Data Register
    I2Cx->TXDR = data;
}

static inline uint8_t I2C_Read(I2C_TypeDef *I2Cx) {
    // Wait for RXNE (Receive Buffer Not Empty flag)
    while (!(I2Cx->ISR & (1U << 2U)));
    // Read byte from Receive Data Register
    return (uint8_t)(I2Cx->RXDR & 0xFFU);
}

static inline void I2C_Wait_TC(I2C_TypeDef *I2Cx) {
    // Wait for TC (Transfer Complete flag)
    while (!(I2Cx->ISR & (1U << 6U)));
}

static inline void I2C_Stop(I2C_TypeDef *I2Cx) {
    // Generate STOP condition (STOP bit 14 in CR2)
    I2Cx->CR2 |= (1U << 14U);
    // Wait for STOPF (Stop detection flag bit 5 in ISR)
    while (!(I2Cx->ISR & (1U << 5U)));
    // Clear STOPF flag by writing to ICR (STOPCF bit 5 in ICR)
    I2Cx->ICR |= (1U << 5U);
}
```

Uno de los registros más importantes es `CR2` (`rm0431 26.9.2`). Nos va a permitir configurar la transferencia que queremos realizar (dirección destino, número de bytes a trasferir, si es escritura o lectura, y algunas funciones más que no utilizaremos). Estúdialo bien, e implementa la función `I2C_Start`:

```
typedef enum {
    I2C_WRITE = 0U,
    I2C_READ  = 1U
} I2C_Direction;

static inline void I2C_Start(I2C_TypeDef *I2Cx, uint8_t slave_addr,
    I2C_Direction direction, uint32_t len) {
    uint32_t cr2 = 0;

    // ... completar Set 7-bit Slave Address (SADD[7:1])
    // ... completar Set Transfer Length (NBYTES[23:16])
    // ... completar Set Direction (RD_WRN bit 10: 0 = Write, 1 = Read)
    // ... completar Generate START Condition (START bit 13)

    // Apply configuration to hardware
    I2Cx->CR2 = cr2;
}
```

Una vez tenemos estas funciones auxiliares, ya podemos programar la escritura y lectura a periféricos a través del bus I2C. Recordemos de **??** el funcionamiento general de I2C. Más allá de no necesitar controlar los ACK (ya que se generarán solos), debemos programar el resto de la lógica siguiendo los pasos allí indicados. Se proporciona el esqueleto a continuación:

```
/* Writes 1 byte to a specific register inside the sensor */
void I2C_Reg_Write(I2C_TypeDef *I2Cx, uint8_t slave_addr, uint8_t reg, uint8_t
    val) {
    //... completar Issue START + set up Address (Write Mode, 2 bytes)
    //... completar Write Target Register Address
    //... completar Write Value
    //... completar Wait for transfer complete (TC)
    //... completar issue STOP condition
```

```
}

/* Reads 'len' sequential bytes starting from 'reg' */
void I2C_Reg_Read_Seq(I2C_TypeDef *I2Cx, uint8_t slave_addr, uint8_t reg,
    uint8_t *dest, uint32_t len) {
    // ================= PHASE 1: WRITE REGISTER ADDRESS =================
    // ... completar Issue START + set up Address (Write Mode, 1 Byte)
    // ... completar Write target register address
    // ... completar Wait for Transfer complete (TC)

    // ================= PHASE 2: READ DATA PAYLOAD =================
    // ... completar Issue START + set up Address (Read Mode, 'len' Bytes)
    // ... completar Read 'len' bytes from the sensor
    // ... completar Wait for Transfer complete (TC)
    // ... completar issue STOP condition
}
```

9.3.4. Driver del CTP

Finalmente, y con las funciones `I2C` creadas, estamos listos para leer los registros del panel táctil con la información correspondiente a las coordenadas de la pulsación. La documentación del mapa de registros interno del panel, y sus valores es bastante escasa. Pero sí disponemos del driver oficial de ST en `https://github.com/STMicroelectronics/stm32-ft3x67/tree/main`. Consultando el fichero `ft3x67.h` podemos ver el mapa de registros, así como los diversos códigos de control y respuesta. Para simplificar el proceso y no hacer un driver tan complejo, únicamente vamos a fijarnos en los siguientes registros internos mostrados en la Tabla 2 y Tabla 3.

Addr	Register Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0x02	FT3X67_TD_STAT_REG	Reserved [7:4]				**Touch Points [3:0]** (0–2)			
0x03	FT3X67_P1_XH_REG	**Event [7:6]**		Res [5:4]		**P1_X[11:8]** (MSB)			
0x04	FT3X67_P1_XL_REG	**P1_X[7:0]** (LSB)							
0x05	FT3X67_P1_YH_REG	Reserved [7:4]				**P1_Y[11:8]** (MSB)			
0x06	FT3X67_P1_YL_REG	**P1_Y[7:0]** (LSB)							
0x07	FT3X67_P1_WEIGHT_REG	**P1_WEIGHT[7:0]**							
0x08	FT3X67_P1_MISC_REG	**P1_MISC[7:0]**							
0x09	FT3X67_P2_XH_REG	**Event [7:6]**		Res [5:4]		**P2_X[11:8]** (MSB)			
0x0A	FT3X67_P2_XL_REG	**P2_X[7:0]** (LSB)							
0x0B	FT3X67_P2_YH_REG	Reserved [7:4]				**P2_Y[11:8]** (MSB)			
0x0C	FT3X67_P2_YL_REG	**P2_Y[7:0]** (LSB)							
0x0D	FT3X67_P2_WEIGHT_REG	**P2_WEIGHT[7:0]**							
0x0E	FT3X67_P2_MISC_REG	**P2_MISC[7:0]**							

Tabla 2: Disposición de los registros en el controlador de panel táctil FT3X67

Como de momento sólo nos interesa leer una coordenada, podemos leer los primeros 5 registros y será suficiente. Deberemos crear un nuevo fichero llamado `FT3x67.h` que contenga la lógica para hacerlo:

Field / Mask	Bits	Name	Values / Description
TD_STAT_MASK	[3:0]	Active Touch Points	Number of active touch points detected (0 to 2).
TOUCH_EVT_FLAG	[7:6]	Event Flag	0x00 (00b): Press Down 0x01 (01b): Lift Up 0x02 (10b): Contact 0x03 (11b): No Event
POS_MSB_MASK	[3:0]	Position MSB	Upper 4 bits of X or Y coordinate (bits [11:8]).
XL / YL	[7:0]	Position LSB	Lower 8 bits of X or Y coordinate (bits [7:0]).
WEIGHT	[7:0]	Touch Weight	Pressure / weight value of touch contact.
MISC	[7:0]	Miscellaneous	Touch area / extra touch panel metrics.

Tabla 3: Definiciones y valores para los registros del FT3X67

```c
/*
* FT3X67.h
*
*  Created on: Aug 11, 2026
*      Author: dani
*/

#ifndef FT3X67_H_
#define FT3X67_H_

#define TS_I2C_7BIT_ADDR 0x38
#define FT_REG_TD_STATUS 0x02

typedef struct {
    uint16_t x;
    uint16_t y;
    uint8_t  touch_detected;
} TS_State_t;

/**
* @brief  Reads the current touch coordinates from the FT3X67.
* @param  ts: Pointer to TS_State_t structure to store parsed coordinates
* @return 1 if touch is valid, 0 otherwise
*/
uint8_t FT3X67_GetTouchCoordinates(TS_State_t *ts) {
    uint8_t buffer[5];

    /* Read 5 consecutive registers starting at TD_STATUS (0x02 to 0x06) */
    // ... completar leer utilizando I2C_Reg_Read_Seq 5 bytes

    uint8_t num_touches = buffer[0] & 0x0F;

    if (num_touches > 0 && num_touches <= 2) {
        // ... completar parsear coordenadas en ts->x y ts->y

        ts->touch_detected = 1;
        return 1;
    }

    ts->touch_detected = 0;
    return 0;
}
```

```
43
44 #endif /* FT3X67_H_ */
```

9.3.5. Integración

Finalmente, ya estamos listos para integrar todo en nuestro código `main.c`. Tras inicializar el I2C y el panel táctil con las funciones `I2C3_Init()` y `TS_INT_Init()` podremos entrar en un bucle infinito que espere el aviso de interrupción del panel táctil antes de leerlo:

```
1  // ... código de inicialización de la pantalla (práctica anterior)
2
3  I2C3_Init();
4  TS_INT_Init();
5
6  while (1) {
7      /* Check if the touchscreen interrupt fired */
8      if (g_ts_interrupt_flag) {
9          g_ts_interrupt_flag = 0; // Reset software flag
10
11         // ... completar obtener coordenada utilizando
12         // ... el driver FT3X67 y pintarla en la pantalla
13         // ... hay que rotarla pues las coordenadas
14         // ... del panel y la pantalla no coinciden
15     }
16 }
```

9.4. Para hacer más

Investiga el repositorio oficial `https://github.com/STMicroelectronics/stm32-ft3x67/tree/main` para el driver del panel táctil, e intenta implementar detección de gestos. También puedes extender la lógica de tu programa para que, en lugar de pintar sólo los puntos que tocas, vaya haciendo una línea entre ellos (verás que, de lo contrario, los puntos no son perfectamente contiguos).

10. Práctica 9: Persistencia - Memoria FLASH

10.1. Objetivos

- Entender el concepto de persistencia y para qué datos sirve.
- Entender el funcionamiento de la memoria `FLASH`.

10.2. Fundamentos teóricos

Todos damos por sentado que, en un ordenador al uso, podemos guardar datos que se mantendrán tras el reinicio. Sin embargo, en los dispositivos empotrados, muchas veces no ocurre lo mismo: Los programas se ejecutan en RAM y, al terminar, toda su información se ha borrado. Si, por ejemplo, queremos tener cierta configuración de usuario (como pueda ser la temperatura en un termostato, por ejemplo) hay que asegurarse de alguna manera que se mantiene.

Una forma de hacerlo es mediante memoria persistente `FLASH`. Este tipo de memoria tiene una peculiaridad, y es que, si bien consigue guardar datos incluso sin corriente, al contrario que la RAM, se desgasta con el uso. Adicionalmente, sólo se pueden escribir "0" y borrar a "1" (lo que se conoce como un ciclo de escritura-borrado). Por tanto, cada vez que la queremos reprogramar, desgastamos su estructura. El procesador **STM32F723IE**tiene una garantía mínima de un aguante de 10000 ciclos de este proceso. Aún así, se recomienda encarecidamente limitar estos números lo máximo posible. Para esta práctica, en concreto, intenta limitar el número de borrados de la `FLASH`. Si tienes cualquier duda, consulta a tu profesor.

En cuanto a las lecturas, no supone ningún problema, y de hecho nuestro programa se guarda en la `FLASH`, para que se ejecute desde ahí las veces que sea necesario, y sus valores volátiles en la `RAM`, evitando así ese desgaste continuo.

10.2.1. Organización de la flash

En nuestro procesador, la memoria `FLASH` se organiza en *sectores* (`rm0431 3.3.1`). Podemos escribir byte a byte en cada sector, sin embargo para resetearlo, deberemos resetear los sectores enteros. Este diseño hace que se desgaste menos la memoria, al caber generalmente un programa en únicamente unos pocos sectores, y no necesitar borrar el resto. Es lo mismo que ocurre con los discos SSD, que pueden ir repartiendo el desgaste entre sus sectores para durar más.

10.3. Desarrollo de la práctica

10.3.1. Bucle de control principal

En esta práctica vamos a hacer que la última coordenada pulsada en la pantalla táctil se guarde de manera persistente, de tal manera que, tras reiniciar (o incluso desconectar de la corriente) nuestro procesador, la podamos volver a ver tras un nuevo arranque. Condicionaremos el guardado con el pusado de un botón, para evitar que se desgaste en exceso. El esquema general es el siguiente:

```c
int main(void)
{
    // Inicializaciones de periféricos
    // para el display, panel, systick,...

    if (UserButton_IsPressed()) {
        // Borrar memoria persistente
    }

    // Attempt reading saved coordinates
    if (Flash_ReadTouch(&last_x, &last_y)) {
        // Si se encuentran datos en la memoria, cargarlos
        // dibujando un cuadrado verde en la posición guardada
    } else {
        // Si no se encuentran datos, dibujar un cuadrado amarillo
    }

    while (1) {
        // Lógica de actualización de la pantalla
        // Si hubo interrupción de pantalla
        // Guardar coordenada en memoria volátil (variable)
        // y pintar en pantalla

        if (UserButton_IsPressed()) {
            // Guardar coordenada en memoria persistente
        }
    }
}
```

10.3.2. Trabajando con la FLASH

En primer lugar, necesitamos completar nuestro `bsp.h` con el código necesario para exponer los registros de la `FLASH` (`rm0431 3`).

```c
#define FLASH_BASE_ADDR      0x40023C00UL

typedef struct {
    volatile uint32_t ACR;      // 0x00
    volatile uint32_t KEYR;     // 0x04
    volatile uint32_t OPTKEYR;  // 0x08
    volatile uint32_t SR;       // 0x0C
    volatile uint32_t CR;       // 0x10
    volatile uint32_t OPTCR;    // 0x14
    volatile uint32_t OPTCR1;   // 0x18
    volatile uint32_t OPTCR2;   // 0x1C
} FLASH_TypeDef;

#define FLASH               ((FLASH_TypeDef *) FLASH_BASE_ADDR)
```

Utilizar la `FLASH` no es tan fácil como escribir directamente en sus registros. Ya hemos mencionado en un par de ocasiones que la memoria se desgasta. No es de extrañar, por tanto, que haya un cierto nivel de protección para trabajar con ella. Aunque es accesible directamente para lectura (podemos leer desde la dirección `0x08000000` en adelante, y estaremos leyendo desde la `FLASH` (`rm0431 1.6.2`)), la escritura está protegida. Adicionalmente a evitar el desgaste, evitamos escribir accidentalmente en una región que suele contener únicamente código y constantes (Ver el archivo de linkado `STM32F723IEKX_FLASH.ld` para comprobarlo).

La `FLASH` está siempre activa, así que nos ahorraremos activar periféricos o relojes. Pero lo primero, antes de trabajar con ella, si queremos realizar algún cambio, es desbloquear el acceso que previene escrituras accidentales. Para ello hay que escribir una secuencia específica de datos en el registro `KEYR` (`rm0431 3.7.2`) tras comprobar que no esté bloqueda. Si por el contrario queremos bloquearla, esto se hace de manera más sencilla configurando el bit correspondiente en `CR` (`rm0431 3.7.5`)

```
/* Flash Keys */
#define FLASH_KEY1         0x45670123U
#define FLASH_KEY2         0xCDEF89ABU
static void inline Flash_Unlock(void) {
    // ... completar aplicar secuencia de desbloqueo en KEYR
}

static void inline Flash_Lock(void) {
    // ... completar bloquear en registro CR
}
```

Ciertas operaciones con la `FLASH` requerirán esperar a que el periférico deje de estar ocupado (no deja de ser una interfaz con la memoria, que no necesariamente opera a la misma velocidad que el procesador). Incluimos también la función que realiza una espera activa a que se libere el bit correspondiente en `SR` : (`rm0431 3.7.4`).

```
static void inline Flash_WaitNotBusy(void) {
    while (FLASH->SR & (1U << 16)); // Wait for BSY bit
}
```

Con estas funciones estamos listos para, en primer lugar, borrar un sector. Recordemos que es necesario borrar (poner todo a "1") antes de poder escribir (que grabará sólo los "0" correspondientes). Esto es debido a que la tecnología subyacente sólo permite este tipo de trabajo. Programamos la función de borrado desde el registro `CR` (`rm0431 3.7.5`). Hay previamente que solicitar el desbloqueo de flash, y con posterioridad bloquearla para evitar escrituras accidentales.

```
static void inline Flash_EraseSector(uint8_t sector) {
    Flash_Unlock();
    Flash_WaitNotBusy();

    // ... completar borrar los bits de sector SNB
    // ... completar programar el sector que queremos borrar
    // ... completar marcar el bit de borrado SER
    // ... completar iniciar el borrado con el bit STRT

    Flash_WaitNotBusy();
    // ... completar desactivar el borrado SER
    Flash_Lock();
}
```

NOTA: Ten cuidado con los sectores que borras, ya que en los primeros (0-3) se suele guardar el código del programa. Si lo borras, lo más seguro es que la aplicación se bloquee, ten en cuenta la estructura de sectores de la Figura 29.

Ahora, si queremos escribir, el proceso será ligeramente distinto. Por el momento, ya que no vamos a escribir demasiada información, vamos a ir byte a byte, pero la función se puede adaptar para escribir posteriormente en bloques, sin necesidad de bloquear y desbloquear con cada transferencia de información. De nuevo trabajamos con el registro `CR` para, en este caso,

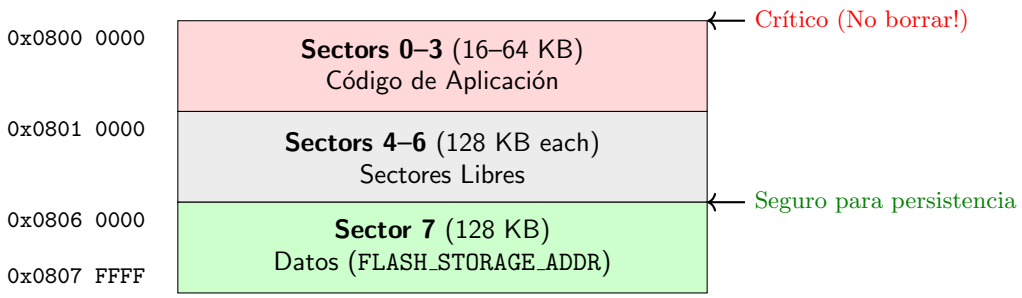


Figura 29: Organización de los sectores de Flash

habilitar la escritura. Observemos también la secuencia para escribir: lo vamos a hacer directamente escribiendo en la dirección de memoria requerida (de la misma forma que lo hacíamos en las prácticas iniciales). El controlador de la `FLASH` recibirá el dato y, tras ello, lo escribirá. Como puede tardar varios ciclos, forzamos mediante primitivas el vaciado de los *store buffers*, y hacemos una espera a que el periférico nos confirme que ha terminado:

```
static void inline Flash_WriteByte(uint32_t address, uint8_t data) {
    Flash_Unlock();

    // ... completar Ajustar el tamaño de transferencia PSIZE a '00'
    // ... habilitar la programación de FLASH con el bit PG a 1

    *(volatile uint8_t *)address = data;
    __asm volatile ("dsb 0xF" ::: "memory"); // Pipeline flush
    Flash_WaitNotBusy();          // Wait for BSY bit to clear

    // ... deshabilitar la programación de FLASH con el bit PG a 0


    Flash_Lock();
}
```

De la lectura no hay que preocuparse, pues podremos hacer lecturas directas a las direcciones de memoria atacadas.

10.3.3. Creando las estructuras para la persistencia

Con esto ya tenemos la `FLASH` preparada para trabajar con ella con todas las funciones auxiliares. Ahora, lo que nos falta es crear un nuevo conjunto de funciones que nos permitan escribir una estructura de datos (que queramos en este caso guardar) utilizándolas. Creamos un nuevo archivo (por ejemplo `persistence.h`) con el siguiente contenido:

```
/*
* persistence.h
*
*  Created on: Aug 27, 2026
*      Author: dani
*/

#ifndef PERSISTENCE_H_
#define PERSISTENCE_H_

#define FLASH_SECTOR_NUM     7U                    // Sector 7 (safe upper Flash
    sector)
#define FLASH_STORAGE_ADDR   0x08060000U           // Base address of Sector 7
```

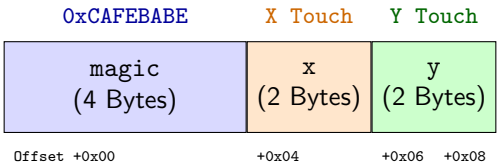


Figura 30: Organización de la estructura de guardado en memoria

```
#define FLASH_MAGIC_KEY     0xCAFEBABEU          // Marker to verify stored data
    validity

typedef struct {
    uint32_t magic;
    uint16_t x;
    uint16_t y;
} FlashTouchData;

/* Erases Sector 7 and writes the magic key + touch coordinates */
void Flash_SaveTouch(uint16_t x, uint16_t y) {
    FlashTouchData data = {
        .magic = FLASH_MAGIC_KEY,
        .x = x,
        .y = y
    };

    Flash_EraseSector(FLASH_SECTOR_NUM);

    uint8_t *pData = (uint8_t *)&data;
    for (uint32_t i = 0; i < sizeof(FlashTouchData); i++) {
        Flash_WriteByte(FLASH_STORAGE_ADDR + i, pData[i]);
    }
}
#endif /* PERSISTENCE_H_ */
```

Hay que fijarse en un detalle interesante: Aunque queremos guardar un conjunto de coordenadas (x, y), nuestra estructura (Figura 30) contiene un tercer campo llamado *magic* con un valor fijo. El guardar un valor como este es muy importante para poder saber si disponemos de valores válidos en las direcciones de memoria que consultemos. De lo contrario, estaríamos cargando valores aleatorios almacenados previamente por otros programas o usuarios. Programa ahora la función simétrica para realizar la lectura:

```
/* Reads coordinates from Flash if a valid magic key is present */
uint8_t Flash_ReadTouch(uint16_t *x, uint16_t *y) {
    // ... completar conseguir un puntero de tipo FlashTouchData
    // ... a la dirección base para poder leerla

    // ... si encontramos la MAGIC KEY, devolver 1
    // ... además de rellenar los valores x, y
    // ... de lo contrario, devolver 0 (Flash vacía)
}
```

Y... casi listo! Sólo nos falta completar el código de `main.c` anteriormente presentado, llamando ahora sí a estas funciones auxiliares que hemos creado para guardar y leer la `FLASH`. Comprueba que, tras guardar un punto, e incluso desconectando la placa de la corriente, se lee el dato guardado en memoria.

10.4. Para hacer más

Existe otra forma de persistencia en memoria mediante la `BKPSRAM` o memoria de *backup*. Es una zona de memoria que mantiene sus contenidos incluso tras resetear el procesador (eso sí, si se pierde la corriente, se borrarán igualmente). No es tan persistente como la `FLASH`, pero no sufre de desgaste. Investiga cómo podrías hacer la persistencia con esta memoria (`rm0431 4.1.3`).

CURIOSIDAD: Los antiguos juegos de videoconsolas como la *Game Boy* funcionaban con este tipo de memoria. Los cartuchos incluían una pila para mantener las partidas guardadas. Si se acababa la pila, se perdían los datos. Duraban unos cuantos años, pero debido a esa tecnología se han perdido muchos recuerdos.

11. Práctica 10: Multitarea Cooperativa

11.1. Objetivos

- Entender la importancia de la multitarea en sistemas empotrados y de tiempo real.
- Desarrollar un modelo cooperativo para la ejecución de diferentes tareas.

11.2. Fundamentos teóricos

Conforme aumenta la complejidad de un sistema, se hace necesaria la ejecución de varias tareas diferentes. Hasta ahora, nuestra solución ha sido tener un gran bucle principal que va ejecutando todo lo necesario. Si son múltiples acciones las que se deben llevar a cabo, se habrá programado una lógica que las vaya haciendo una tras otra en algún orden. Sin embargo, cuando aumenta la complejidad del sistema, esta aproximación acaba resultando inviable. En su lugar, se favorece el desarrolar algún tipo de *framework* que nos permita crear *tareas*, y las ejecute automáticamente cada cierto tiempo.

Especialmente en sistemas empotrados, esta temporalidad es muy importante (y en ocasiones crítica):

- Un generador que debe asegurar una frecuencia exacta de 50Hz.
- Un controlador de vuelo que debe muestrear una unidad de medición inercial a 400Hz.
- Un sistema de airbag que debe comprobar la velocidad y dispararse en menos de 1ms.

Para dar servicio a todos estos sistemas, y cualquier otro, se crean *planificadores*. Los planificadores no son algo que tenga dentro de sí el procesador (como podían ser los periféricos), sino que son responsabilidad del programador, que los programa aprovechando las capacidades del procesador. La principal característica que se explota para ello es la capacidad de temporización del procesador, ya sea a través de los `TIM` `rm0431 18,19,20`, a través del `RTC` `rm0431 25`, o a través (y lo más común) del *ticker* del sistema `SYSTICK` `pm0253 4.4`.

Con estos, se introducen interrupciones periódicas que paran las tareas en curso, y dan paso a que un planificador pueda organizar las sigiuentes tareas a ejecutar.

11.3. Desarrollo de la práctica

Lo primero es entender qué perseguimos, y podemos verlo en la Figura 31. Tener unas tareas (por ejemplo 1, 2 y 3) que se ejecuten, respectivamente, en intervalos diferentes (250, 500 y 1000ms). Para medir el tiempo, utilizaremos el `SYSTICK`, configurado precisamente para que avise cada milisegundo. Por supuesto, si se quiere una granularidad diferente de tiempos, se puede configurar con otros intervalos diferentes.

Como vemos, queda cierto tiempo libre entre la ejecución de tareas. Una cuestión importante de indicar es que, por mucho que consideremos este tipo de sistemas para "tiempo real", tenemos que asegurar siempre que las tareas tardan menos en ejecutarse, del tiempo disponible para la ejecución. De lo contrario, y aunque quizá sea obvio, no dará tiempo a completar todas.

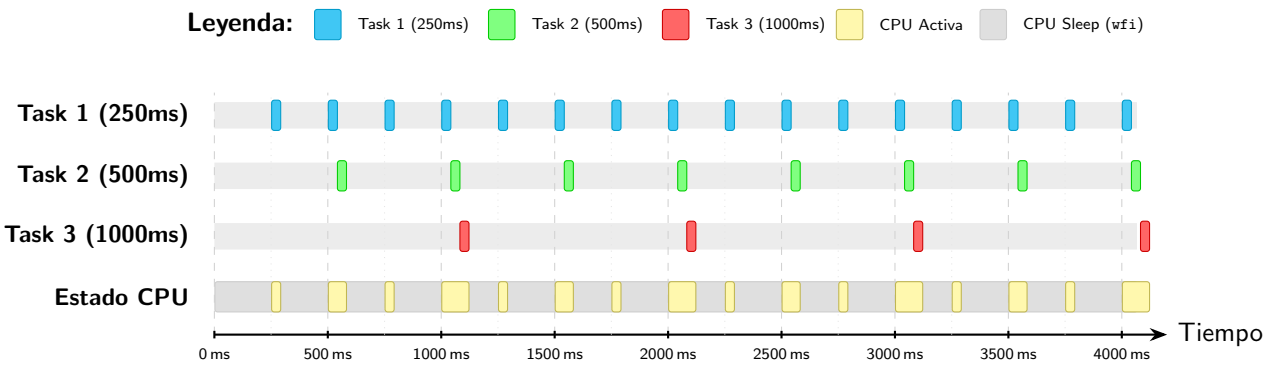


Figura 31: Diagrama de planificación de tareas.

Generalmente, se planifican de esta forma tareas muy críticas, condenando tareas no críticas a poder potencialmente perder "turnos". No es el objetivo en cualquier caso de esta práctica, ya que nuestras tareas serán muy sencillas, pero hay que tenerlo en cuenta.

Nuestras tareas simplemente parpadearán los 3 LEDs disponibles en la placa (El LED rojo en `PA7`, el azul el `PA5`, y el verde en `PB1`, recuerda que deberás inicializar los `GPIO` correspondientes).

Nuestro código será extremadamente simple (al menos el principal, el resto quizá sea debatible). La estructura es la siguiente:

```
1  void SysTick_UserCallback() {
2      scheduler();
3  }
4
5  int main(void) {
6      SysTick_Init();
7      GPIO_Init();
8      scheduler_init();
9
10     create_task(Task_BlinkBlue, 250);
11     create_task(Task_BlinkGreen, 500);
12     create_task(Task_BlinkRed, 1000);
13
14     // Scheduler
15     while (1) {
16         // Low-power sleep: Wait for the next SysTick interrupt before executing
               the scheduler loop again
17         __asm volatile ("wfi");
18         dispacher();
19     }
20
21     return 0;
22 }
```

Disponemos de una función `SysTick_UserCallback` que se llamará automáticamente en cada *tick* del `SYSTICK`, y organizará las tareas a ejecutar. Por otro lado, una función `main` que creará las tareas, y ejecutará el *despachador* para irlas ejecutando según estén listas. El diagrama general de ejecución se puede ver en la Figura 32.

Es decir, la función del *scheduler* es la de organizar qué tareas se pueden ejecutar, si es que ya les toca su "hora". Por su parte, el *dispatcher* se encargará de lanzarlas a ejecución. La lógica que pongamos dentro de cada uno será lo que determine qué tareas tienen más o menos

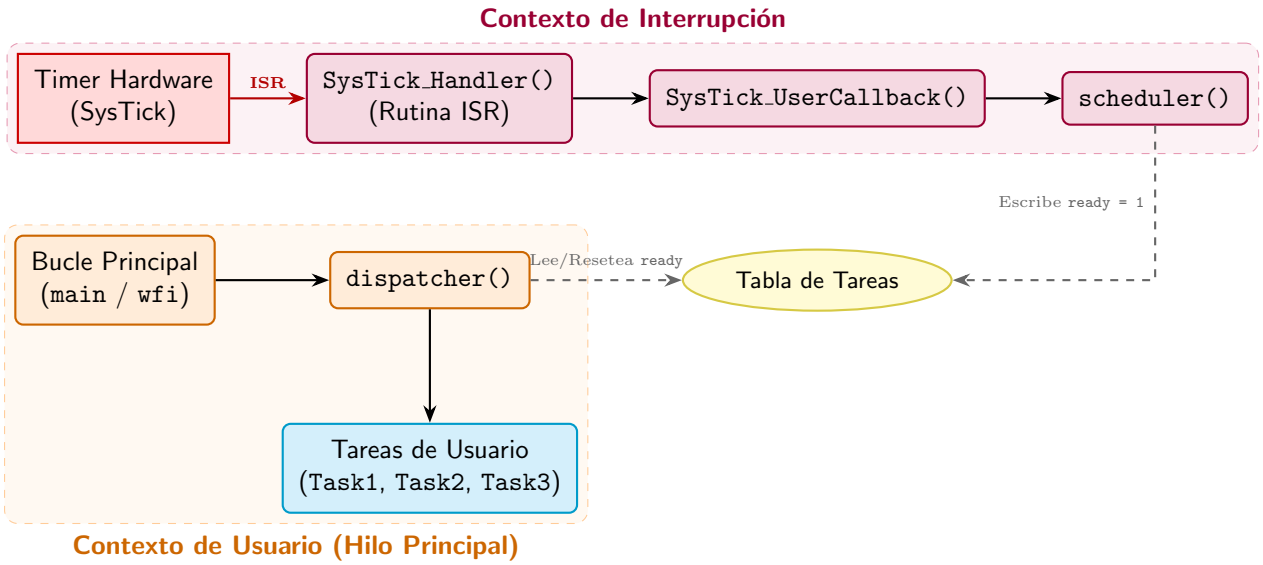


Figura 32: Arquitectura de llamadas a funciones e interacción de datos entre ISR e Hilo Principal.

prioridad. El *ticker* de sistema será el que vaya dictando cuándo se ejecutan estas dos funciones, ya que llamará al *scheduler* en cada *tick*, y a la vez despertará al procesador en caso de estar esperando en la instrucción `__asm volatile ("wfi");` (*wait for interrupt*).

Lo primero que necesitamos es modificar ligeramente el fichero `bsp.h` (separándolo en `bsp.h` y `bsp.c`) para incluir la llamada a la función `SysTick_UserCallback`. Así tendremos el aviso de que ha habido un *tick*. (No se puede hacer todo en `bsp.h` ya que saldrán errores de redefinición de funciones y/o variables)

```
//en bsp.h, quitamos la implementacion del Handler, y
//declaramos la variable msTicks como extern volatile
//para indicar que se define en el bsp.c
extern volatile uint32_t msTicks;

void SysTick_UserCallback(void);
void SysTick_Handler(void);


//código para bsp.c
#include "bsp.h"

volatile uint32_t msTicks = 0;

__attribute__((weak)) void SysTick_UserCallback(void) {
    // Default empty implementation
}

void SysTick_Handler(void) {
    msTicks++;
    SysTick_UserCallback();
}
```

Importante notar dos cosas: La variable msTicks que se define como externa en el `bsp.h` para que otros archivos que lo importen (como `main.c`) no lo redefinan, creando así conflictos en compilación. Por otro lado, la definición de `SysTick_UserCallback` como `__attribute__((weak))`, lo cual permite:

1. Tener ya la llamada hecha desde `bsp.c`.
2. Poder redefinir la función desde otro archivo (en este caso `main.c`) para que se llame a esa en su lugar.

Una vez modificado ya podemos atacar a `main.c`. Lo primero es definir las estructuras de tarea. Una tarea consta de una función a la que llamar, y datos sobre cada cuánto se tiene que llamar, y hace cuánto se ha llamado por última vez, para que el planificador pueda gestionar su ejecución:

```
#define MAX_TASKS 10

// Task structure defining what a task is
typedef struct {
    void (*task_function)(void); // Pointer to the task function
    uint32_t period_ms;          // How often the task should execute (in ms)
    uint32_t last_run;           // The sys_ticks timestamp when it last
        executed
    uint8_t ready;               // True if the task should run
} Task_TypeDef;

// Tasks in our program
static Task_TypeDef tasks[MAX_TASKS] = {0};
```

Las tareas en este caso son bien sencillas, simplemente parpadean unos LEDs:

```
void Task_BlinkGreen(void) {
    GPIO_PIN_TOGGLE(GPIOB, 1);
}

void Task_BlinkBlue(void) {
    GPIO_PIN_TOGGLE(GPIOA, 5);
}

void Task_BlinkRed(void) {
    GPIO_PIN_TOGGLE(GPIOA, 7);
}
```

Ahora toca crear las funciones para gestionar tareas. Observa que `create_task` recibe una función como argumento, que se guarda internamente en su estructura

```
uint32_t create_task( void (*pfunction)(void), uint32_t period ) {
    uint32_t id;

    // ... completar buscar una entrada libre en la tabla de tareas
    id =

    // ... completar inicializar los campos de la tarea

    return id;
}

void delete_task (uint32_t id ) {
    tasks[id].task_function = 0;
    tasks[id].period_ms = 0;
    tasks[id].last_run = 0;
    tasks[id].ready = 0;
}

void scheduler_init (void) {
```

```
    uint32_t id;
    for( id=0; id<MAX_TASKS; id++ )
        delete_task( id );
}
```

Con las tareas creadas, ya sólo nos queda crear el planificador y el despachador. Para el planificador, basta recorrer la lista de tareas y, si ya le toca ejecutarse (comprueba que desde la última vez que se ejecutó, ya ha pasado el tiempo suficiente), la marcamos como *ready*:

```
void scheduler (void) {
    uint32_t tick_actual = GetTick();
    for (uint32_t id = 0; id < MAX_TASKS; id++)
        if (tasks[id].task_function) //if there is a task here
           {
            // ... completar: si le toca ejecutar, marcar como ready
           }
}
```

El dispatcher, simplemente lo que hará será ejecutar las tareas que estén listas:

```
void dispacher (void) {
    uint32_t id;
    for (id = 0; id < MAX_TASKS; id++)
        if (tasks[id].ready) {
            // así se ejecuta un puntero a función
            // por si alguna vez te lo habías preguntado
            // (probablemente no, pero a que mola?)
            (*tasks[id].task_function)();
            tasks[id].ready = 0;
        }
}
```

Y listo! Ya solo nos falta conectar el *callback* del `SYSTICK` para que llame a nuestro *scheduler* cada vez que se dispare:

```
void SysTick_UserCallback() {
    scheduler();
}
```

Si hemos hecho bien todo y lo ejecutamos, deberíamos ver parpadear los 3 leds en diferentes frecuencias. Por detrás, lo que está ocurriendo es que 1000 veces por segundo se está disparando el *scheduler*, y a continuación el *dispatcher* al despertar el procesador de su espera. En caso de encontrar funciones para ejecutar, las lanza. Si no, se vuelve a dormir. Si una función tarda más de un hueco (o *timeslot*) en ejecutarse, simplemente el *dispatcher* no se lanzará en ese ciclo. Podemos ver este detalle en la Figura 33.

11.3.1. Fragilidad de la aproximación

Añade una tarea que no funcione "bien" (por ejemplo que entre en un bucle de espera con `Delay_ms`). ¿Qué pasa con la ejecución del resto? Comprueba también que pasa si esta tarea está la primera o la última. ¿Cómo piensas que podría solucionarse este problema?

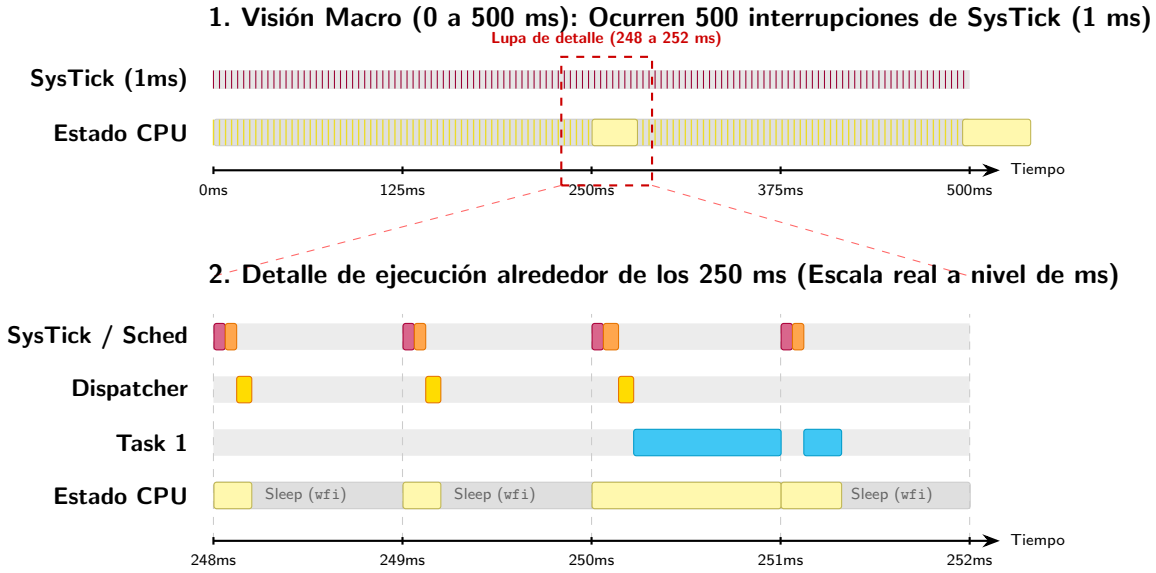


Figura 33: Detalle de ejecución de los diferentes elementos de nuestro planificador, con una tarea que tarda más de un *slot*

11.4. Para hacer más

En sistemas complejos, y para evitar bloqueos, se suelen introducir prioridades a las tareas, a fin de que las tareas más prioritarias (del sistema) puedan pasar primero. Incluye un sistema de prioridades a las tareas, donde tareas con una prioridad más alta se ejecuten siempre primero. (**NOTA:** al ser un planificador *cooperativo*, podrían bloquearse igualmente si una tarea no cede su turno, no te preocupes si ocurre).

12. Práctica 11: Comunicación remota con *ThingsBoard*

12.1. Objetivos

- Entender y configurar la plataforma de *ThingsBoard*.
- Programar el STM32F723 para comunicarse con *ThingsBoard*, enviando y recibiendo datos.
- Entender los protocolos de comunicación como MQTT.

12.2. Fundamentos teóricos

En general, cualquier plataforma empotrada hoy en día tiene cierta conectividad con la red. Desde electrodomésticos inteligentes hasta estaciones de monitorización atmosférica, pasando por bombillas que pueden cambiarse de color desde el móvil, todos estos dispositivos pueden enviar y recibir información desde internet.

Para ello existen numerosos protocolos. Uno de los más extendidos, por su ligereza y versatilidad, es **MQTT**. **MQTT** es un protocolo ligero para el intercambio de mensajes, diseñado para la comunicación en redes con ancho de banda limitado, y por dispositivos con capacidad de computación reducida. Debido a su simplicidad y escalabilidad, MQTT es de particular interés para aplicaciones de Internet de las cosas (IoT).

MQTT utiliza un formato de mensaje binario (Figura 34), que reduce enormemente la cantidad de información transmitida frente a otros protocolos basados en texto como **HTTP**.

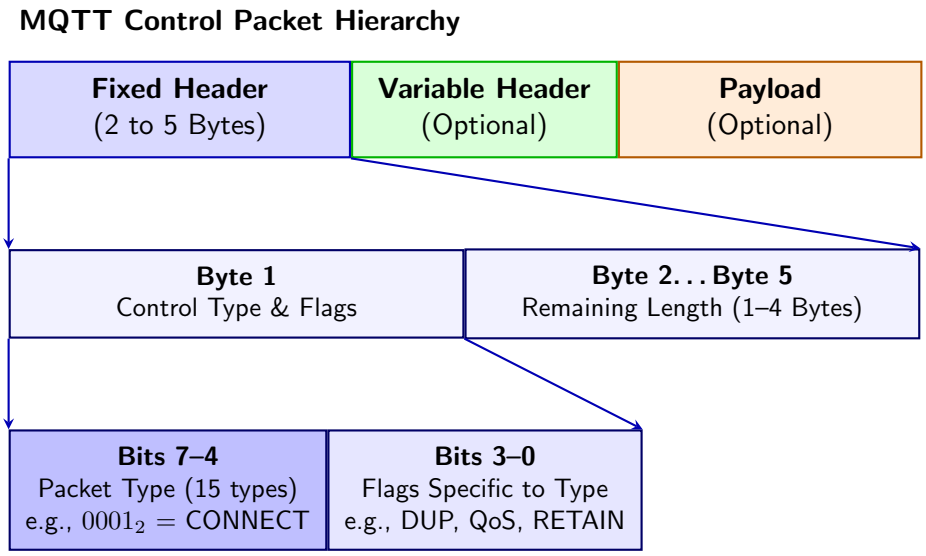


Figura 34: Formato de los paquetes **MQTT**

En **MQTT** se distinguen los *clientes* y los *brokers*. Un cliente puede enviar mensajes al broker (conteniendo generalmente actualizaciones de valores etiquetados), y el broker reenvía estas actualizaciones a otros clientes que estén suscritos a dichos valores etiquetados. Las etiquetas se indican categorizadas, con cada categoría separada por "/". Así, se permiten suscripciones a categorías completas, por ejemplo (luego lo veremos en más detalle al hacer la práctica). Un ejemplo sencillo de esta estructura puede verse en Figura 35

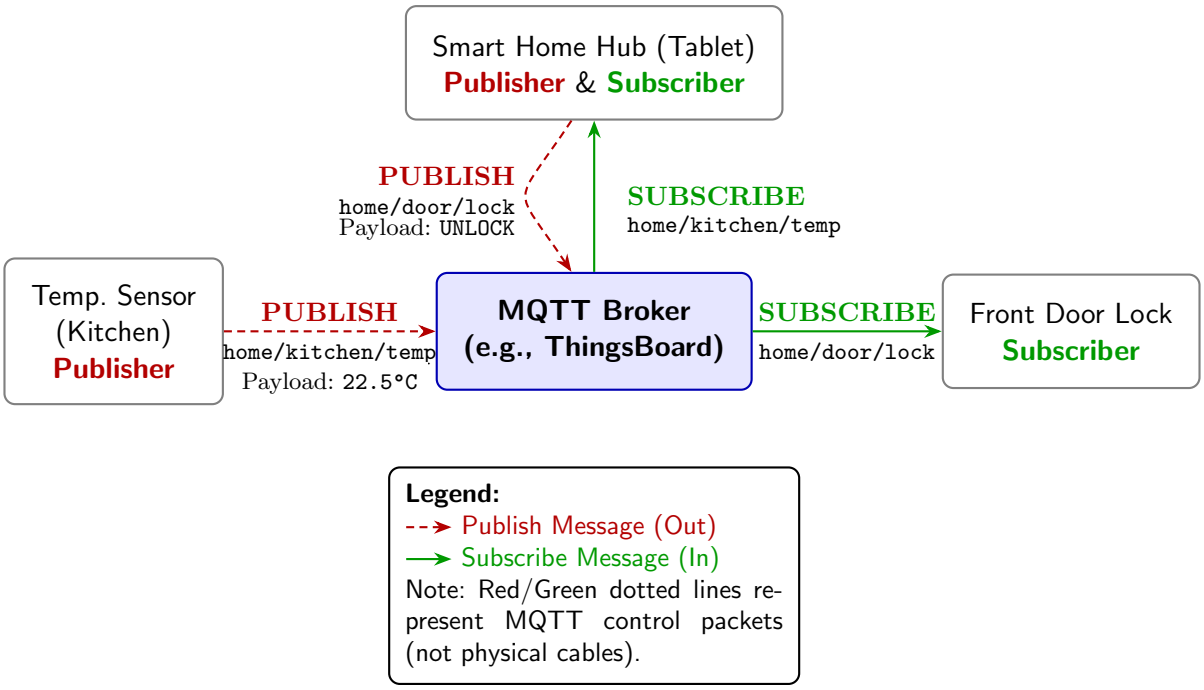


Figura 35: Generic MQTT diagram

12.2.1. El módulo Wifi ESP-01S

Nuestro procesador no tiene manera de realizar comunicación inalámbrica, por tanto se utiliza un módulo externo `ESP-01S` conectado al conector `CN14`. Este módulo permite, a través de una comunicación `UART` desde el procesador **STM32F723IE**, establecer una conexión WiFi y enviar o recibir datos a través de protocolos como `HTTP` y `TCP`, que serán vistos desde nuestra aplicación a través de la `UART`.

Nosotros lo veremos solo como usarios y con un conjunto reducido de comandos. Puedes consultar la documentación completa en `https://espressif-docs.readthedocs-hosted.com/projects/esp-at/en/release-v2.2.0.0_esp8266/AT_Command_Set/index.html`.

12.3. Desarrollo de la práctica

Para realizar la práctica, vamos a hacer uso de un *broker* llamado **ThingsBoard** (`https://thingsboard.io/`). Es una plataforma online que ofrece, de manera gratuita, un *broker* para hasta 5 dispositivos. Como solo tenemos uno, será suficiente.

Para nuestro cliente, utilizaremos las librerías de Eclipse Paho MQTT (`https://github.com/eclipse-paho/paho.mqtt.embedded-c`), una implementación ligera de MQTT para dispositivos empotrados.

12.3.1. Creando cuenta y dispositivo en *ThingsBoard*

Lo primero de todo es crear una cuenta gratuita en *Thingsboard Cloud Europe* (`https://eu.thingsboard.cloud/signup`). Veremos algo similar a la Figura 36 al iniciar.

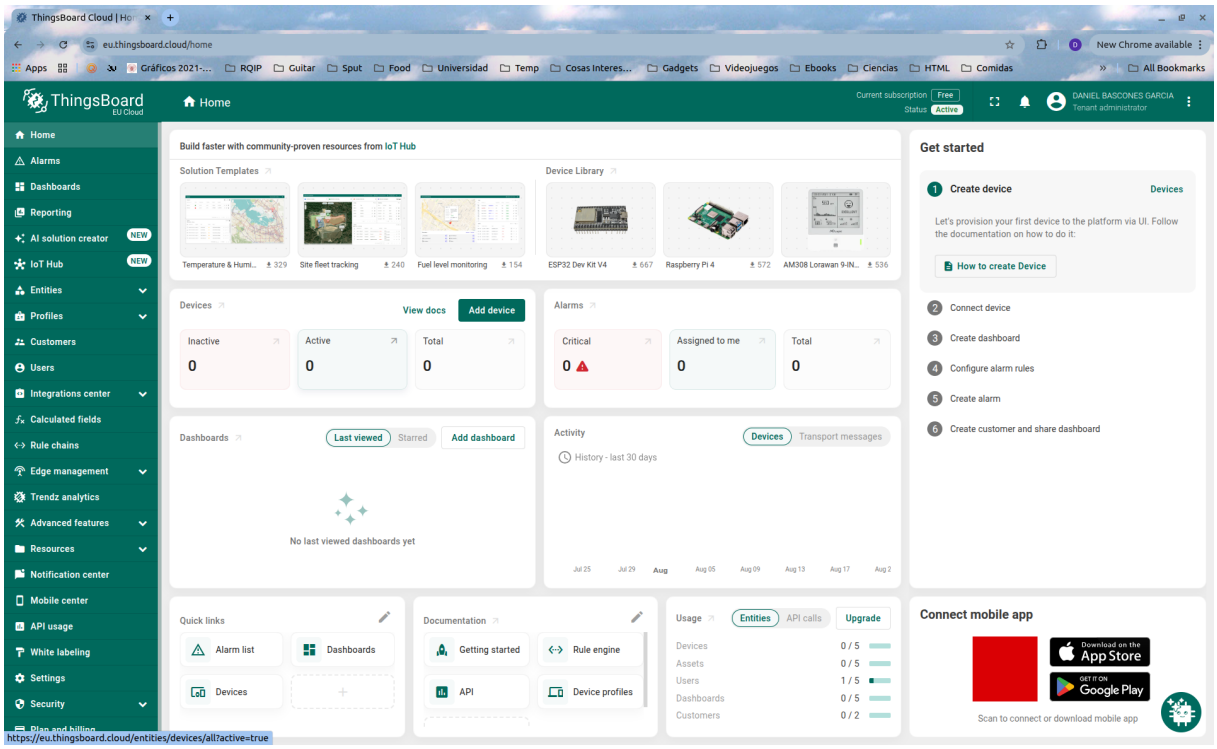


Figura 36: Pantalla inicial de *ThingsBoard* tras iniciar sesión

A continuación, tendremos que crear un dispositivo (desde el panel "Devices") (que representará nuestra placa **32F723EDISCOVERY**) como en la Figura 37.

Para conectarnos posteriormente, necesitaremos identificarnos (y demostrar, desde el dispositivo, que tenemos derecho a enviar/recibir información). Aunque hay varias maneras de hacerlo, nosotros lo haremos con un *token* de acceso. Para ello, entra en la configuración del dispositivo y busca el *token* dentro de *credentials* (Figura 38). Anótalo bien, lo necesitarás.

12.3.2. Programando nuestro dispositivo para conectar

En esta práctica no vas a tener que programar demasiadas cosas, ya que la complejidad de las librerías es elevada. La base del código, ya funcional, está hecha, y tiene esta pinta:

```
thingsboard_data_control/
├── Src/ ..... – Source code directory
│   └── main.c ..... – Core application entry point
└── Inc/ ..... – Include files
    ├── bsp.c ..... – Board Support Package Code
    ├── bsp.h ..... – Board Support Package Header
    ├── ESP01S_config_template.h ..... – Template to fill with config constants
    ├── ESP01S.c ..... – ESP01S Controller Code
    ├── ESP01S.h ..... – ESP01S Controller Header
    ├── MQTT* ..... – Eclipse Paho Embedded MQTT Library
    ├── RingBuffer.c ..... – Helper Receiver Buffer Code
    ├── RingBuffer.h ..... – Helper Receiver Buffer Header
    └── StackTrace.h ..... – Eclipse Paho Embedded MQTT Library
```

Lo único que falta es renombrar el archivo `ESP01S_config_template.h` por `ESP01S_config.h`,

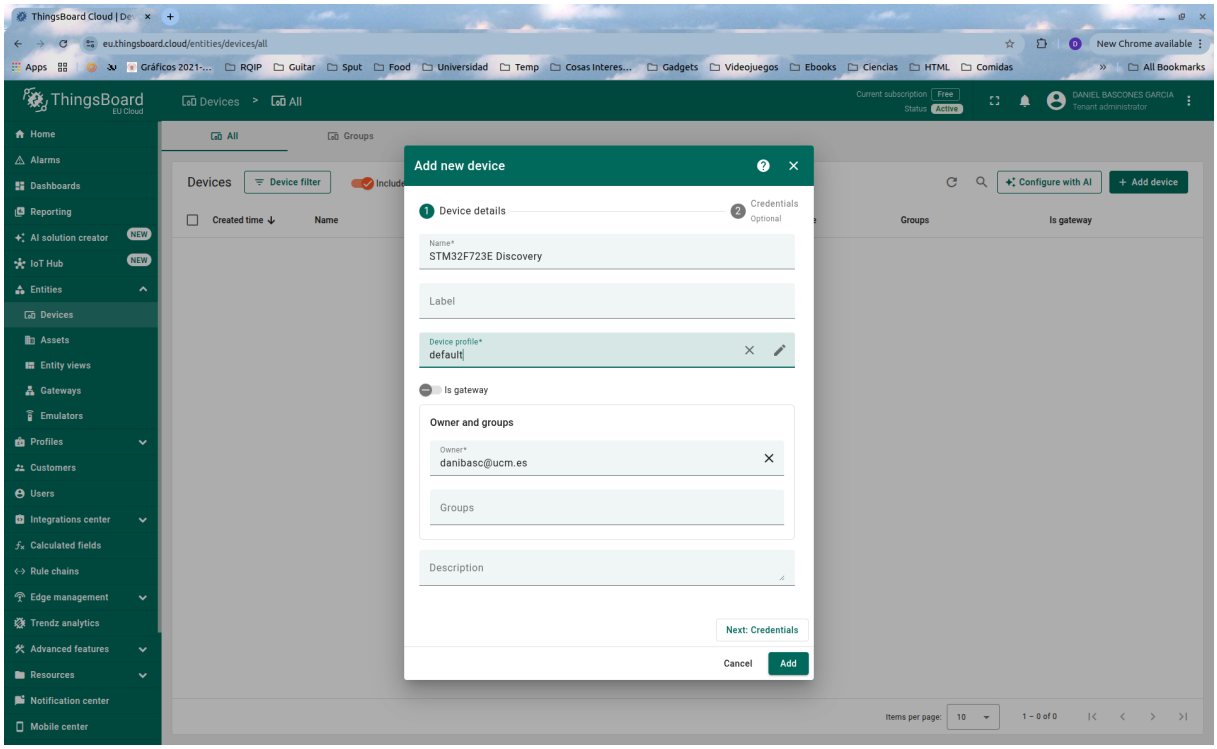


Figura 37: Pantalla de añadir un nuevo dispositivo

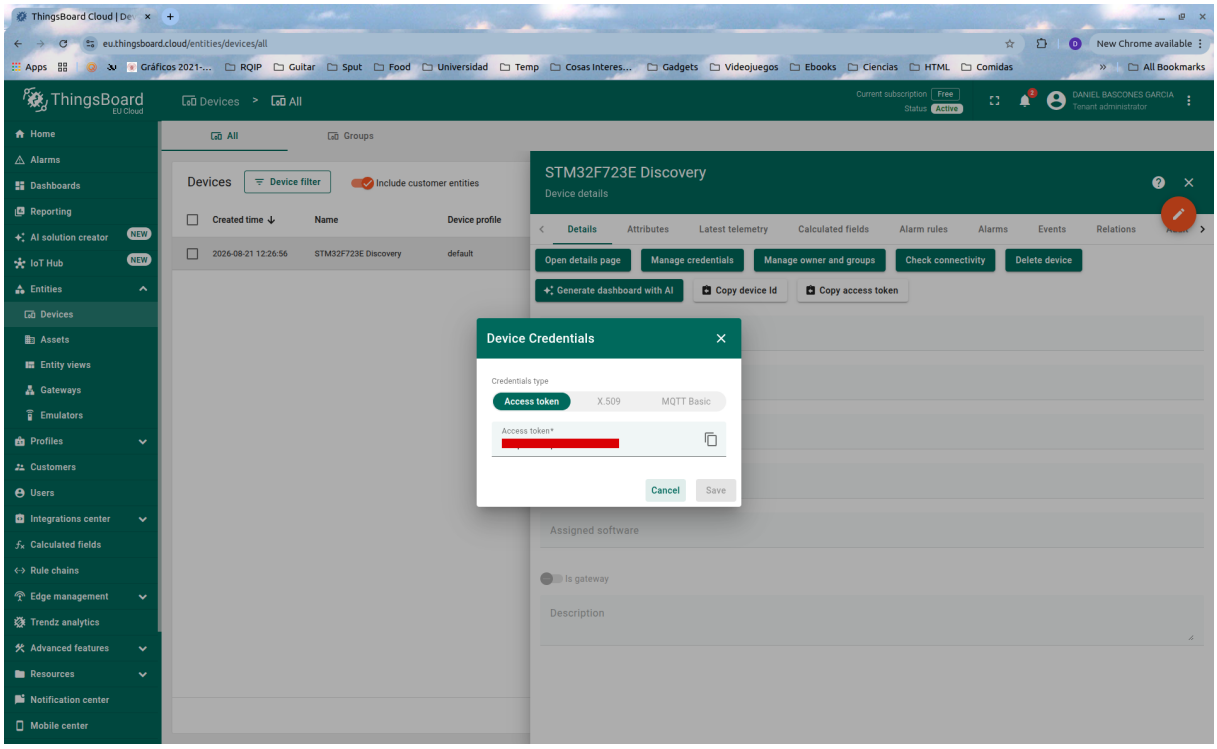


Figura 38: Credenciales de nuestro dispositivo

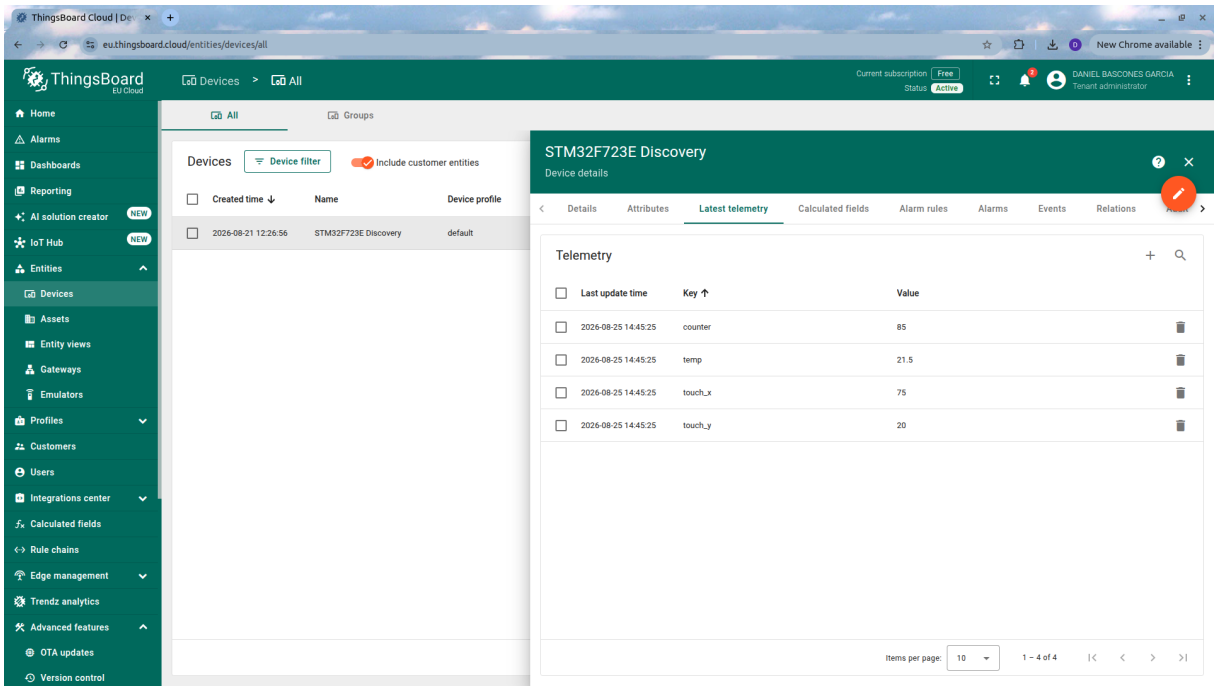


Figura 39: Telemetría vista desde thingsboard

y configurar las credenciales del punto WiFi a utilizar (puede ser vuestro teléfono en modo AP) y el *token* que hemos obtenido anteriormente.

Al lanzar ahora la aplicación, deberíamos ver por el terminal (recuerda abrirlo con `picocom`) los siguientes mensajes de depuración):

```
[ESP] Testing ESP01S AT Communication...
[ESP] Disabling Echo...
[ESP] Setting Station Mode...
[ESP] Disabling MUX (Single Conn Mode)...
[ESP] Enabling Transparent Passthrough Mode...
[ESP] Connecting to SSID: WIFI_SSID ..
[MQTT] Connecting to eu.thingsboard.cloud:1883...
[MQTT] Entering passthrough streaming...
[MQTT RX] CONNACK 0x00: Connection Accepted!
[MQTT RX] SUBACK Received! Granted QoS: 0
[MQTT] Connected & Subscribed successfully!
[SYSTEM] Ready! Streaming over MQTT...
[MQTT TX] {"temp":20.50,"counter":1,"touch_x":15,"touch_y":20}
[MQTT TX] {"temp":21.00,"counter":2,"touch_x":30,"touch_y":40}
[MQTT TX] {"temp":21.50,"counter":3,"touch_x":45,"touch_y":60}
[MQTT TX] {"temp":22.00,"counter":4,"touch_x":60,"touch_y":80}
[MQTT TX] {"temp":22.50,"counter":5,"touch_x":75,"touch_y":100}
```

Con esto, hemos establecido la conexión al *broker* de **MQTT** de *ThingsBoard*. Algunas cosas a notar es que, debido a la simplicidad de esta implementación (si echas un vistazo al código, verás que se implementa un cliente "a mano"), no tenemos control de errores ni QoS superior a 0. Es decir, que si algún mensaje se pierde, se ha perdido y no se recupera.

De momento, en cualquier caso, estamos enviando mensajes al servidor con varias variables (en este caso, creadas de manera sintética). Si entramos en *ThingsBoard*, dentro de la gestión del dispositivo, deberíamos ver ya las variables de telemetría como en la **??**.

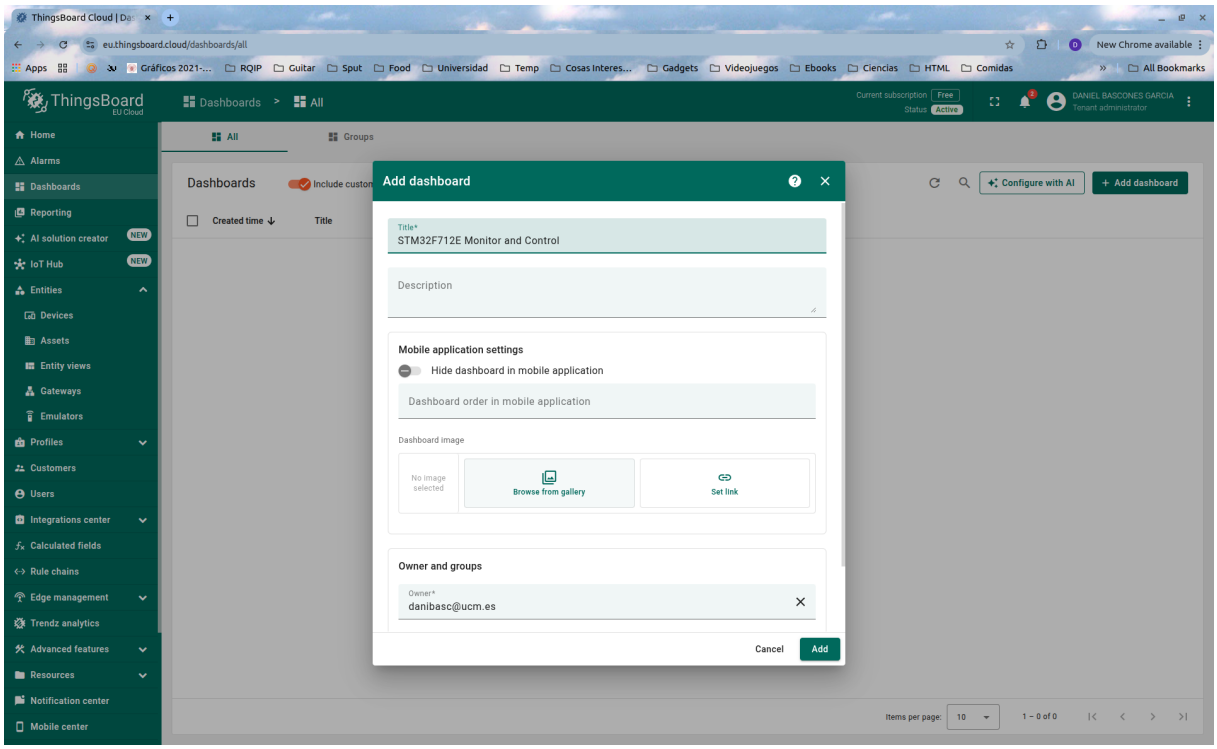


Figura 40: Creación de un panel de control

12.3.3. Visualización en *ThingsBoard*

ThingsBoard tiene un sistema de visualización muy potente para crear gráficas, así que vamos a aprovecharlo, para ver bien estos datos, en lugar de en una lista numérica. Lo primero para poder utilizarlo es crear un *Dashboard* como en la Figura 40.

Ahora podemos añadir fácilmente elementos a nuestro panel desde el modo edición ¿añadir widget. Por ejemplo una gráfica que muestre la temperatura como en la Figura 41. Asegúrate muy bien de que el nombre de la variable visualizada es el mismo que el que da nuestro dispositivo. A veces por defecto sale solo "temp" y no funcionará si es así.

También puedes añadir Widgets muy interesantes creados por la comunidad. Desde el mismo botón de Add Widget, pulsa en la pestaña de IoT Hub y busca "scatter plot". Puedes instalarlo desde ahí como muestra la Figura 42. Aprovecha para configurarlo mostrando `touch_x` y `touch_y`, las variables sintéticas que se están enviando.

Tras haber añadido ambos widgets, deberías tener un panel similar a este, que se va actualizando automáticamente con los valores que llegan desde **32F723EDISCOVERY**.

¿Y... cómo se está haciendo toda esta magia detrás de las cámaras? Puedes analizar, viendo el código de `main.c` que cada cierto tiempo se llama a una función de envío de telemetría. Esta misma se reproduce a continuación:

```
1 /*
2 * Generic publish telemetry over MQTT
3 */
4 uint8_t ESP01S_SendTelemetry_MQTT(ESP01S_TypeDef * esp01s, float temp, uint32_t
      counter, uint16_t touch_x, uint16_t touch_y) {
5     uint8_t buf[256];
6     char payload[256];
```

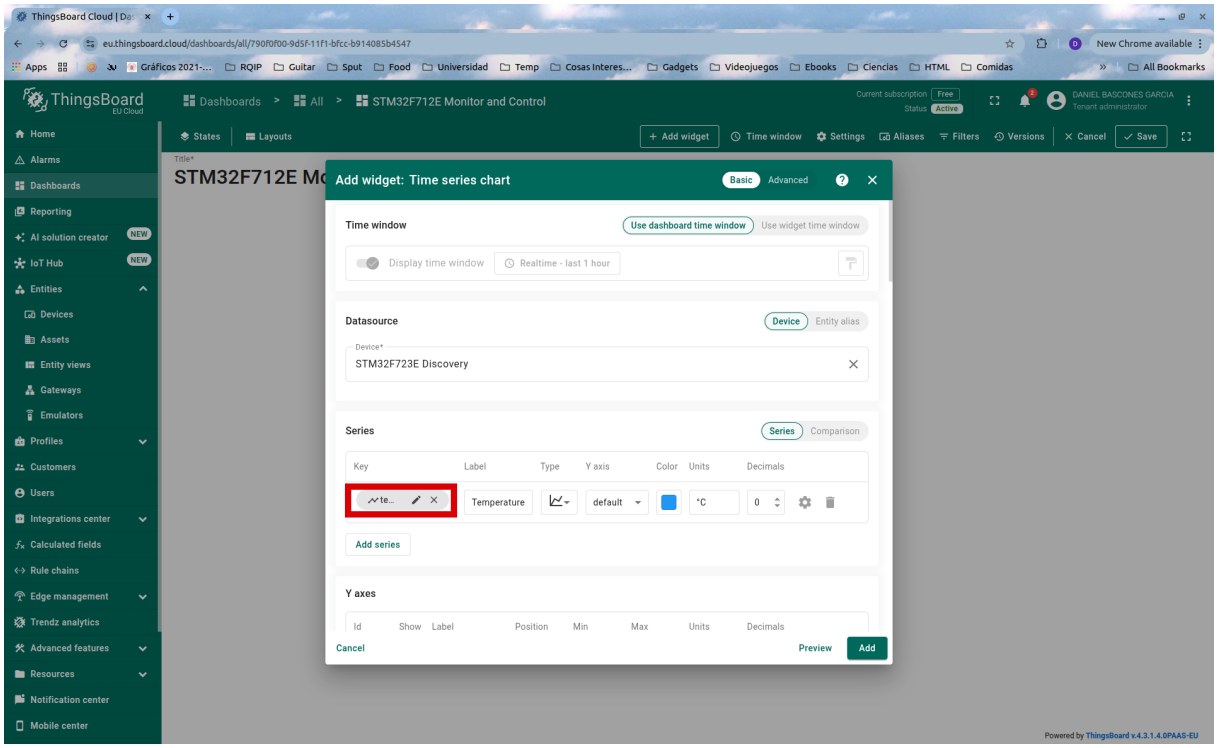


Figura 41: Añadiendo una gráfica que muestre la variable “temperature”

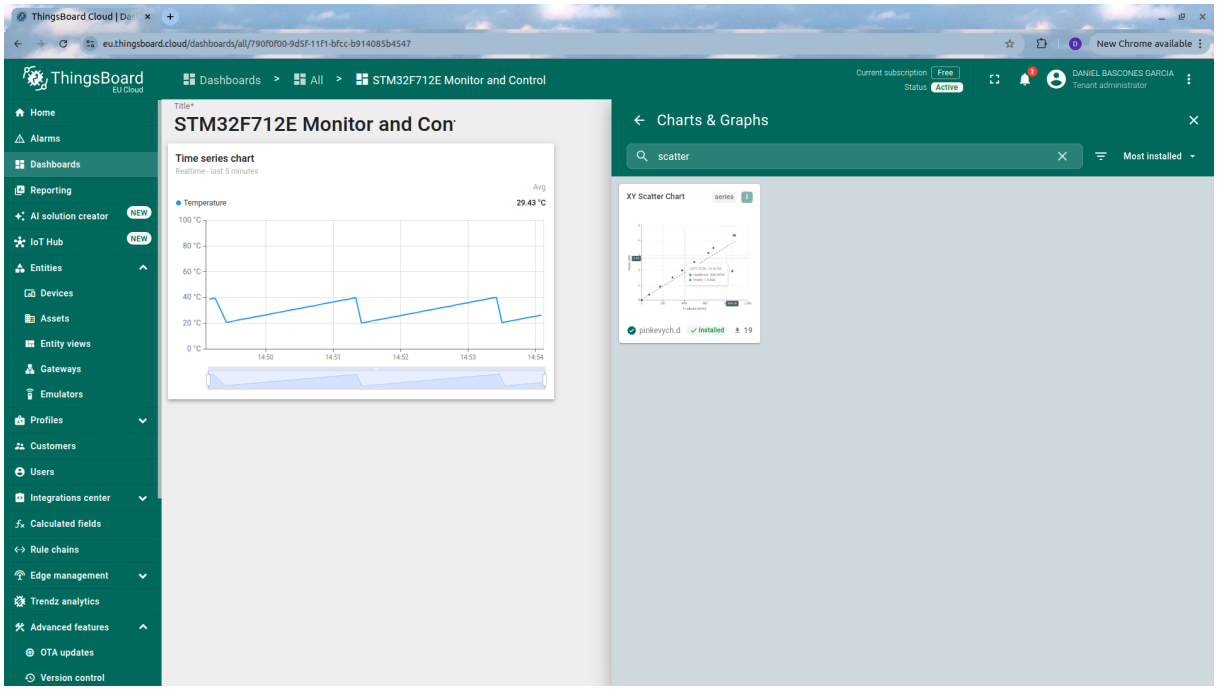


Figura 42: Instalando nuevos widgets

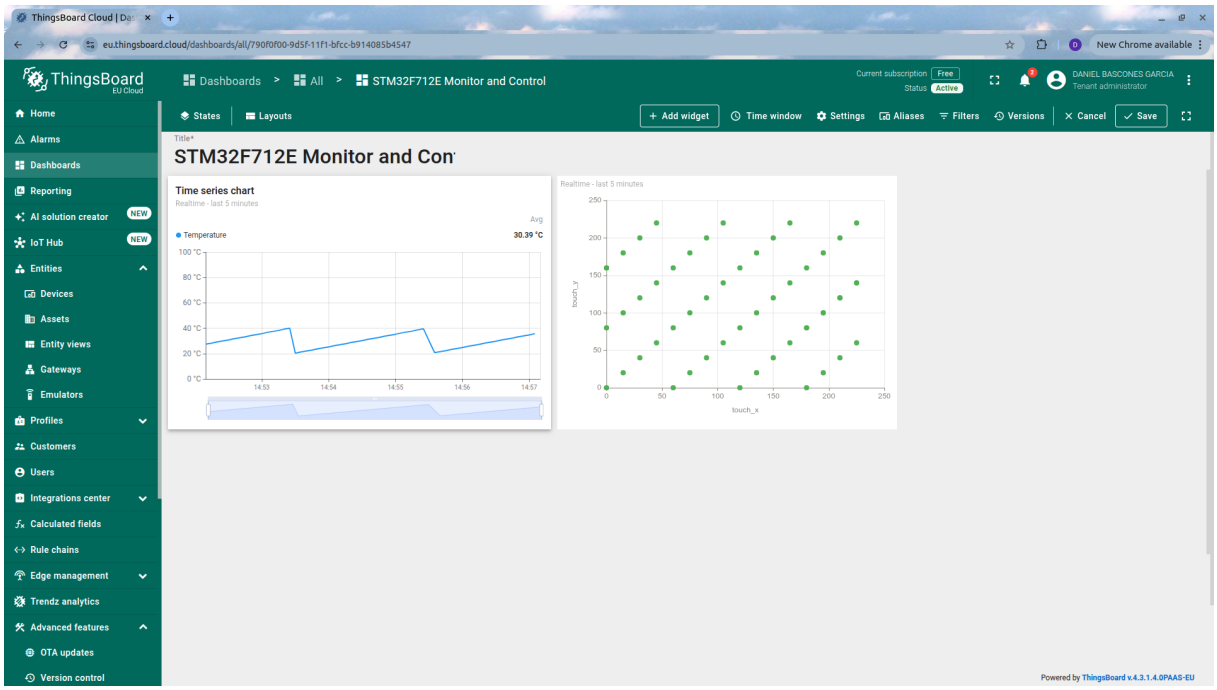


Figura 43: Panel finalizado

```
7
8       int temp_int = (int)temp;
9       int temp_frac = (int)(abs((int)(temp * 100.0f)) % 100);
10
11      snprintf(payload, sizeof(payload),
12              "{\"temp\":%d.%02d,\"counter\":%lu,\"touch_x\":%u,\"touch_y\":%u}",
13              temp_int, temp_frac, (unsigned long)counter, touch_x, touch_y);
14
15      MQTTString topicString = MQTTString_initializer;
16      topicString.cstring = "v1/devices/me/telemetry";
17
18      int len = MQTTSerialize_publish(buf, sizeof(buf), 0, 0, 0, 0,
19                                      topicString, (unsigned char *)payload,
                                           strlen(payload));
20
21      UART_WriteBytes(esp01s->UARTx, buf, len);
22      Debug_Printf("[MQTT TX] %s\r\n", payload);
23      return 1;
24  }
```

Puedes ver que primero se construye el *payload MQTT* o carga, con las variables a enviar. A continuación, se declara el *topic MQTT* o identificador donde enviar estas variables (por ejemplo, donde otros clientes podrían estar suscritos para escuchar cambios). El mensaje se forma con la librería incluida, mediante la función `MQTTSerialize_publish`, y se envía a través de la `UART` que comunica con el módulo `ESP01S`, que está configurada en modo *passthrough* para, básicamente, transformar nuestra `UART` en un canal `TCP` con el servidor.

Es en esta función en la que te tendrías que fijar si quisieras enviar cualquier otro tipo de dato, cambiando el *topic* y el *payload*.

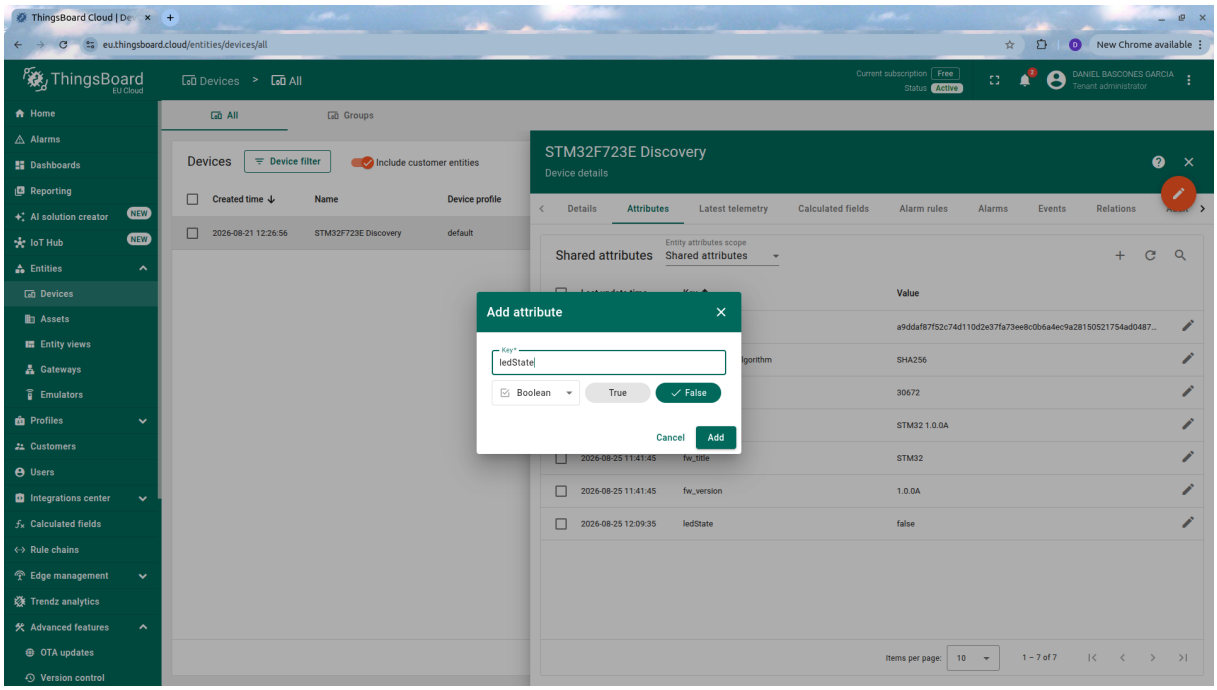


Figura 44: Atributo compartido para nuestro dispositivo

12.3.4. Recibiendo comandos desde *ThingsBoard*

.

Algo muy útil es la posibilidad no sólo de recibir información en *ThingsBoard*, sino también de enviar datos desde la misma plataforma, a fin de que los dispositivos se puedan configurar o accionar remotamente. En este caso, controlaremos el LED de la placa con un interruptor remoto. Para ello vamos a la configuración del dispositivo, y desde la pestaá "Attributes", añadimos un nuevo atributo compartido (*shared*) llamado `ledState`, como en la Figura 44.

Ahora tenemos que añadir un botón que lo pueda gestionar. Añadimos a nuestro panel un widget "single switch", y lo configuramos como muestra la Figura 45. Básicamente lo que estamos haciendo es añadir un botón que cambiará este valor asociado a nuestro dispositivo.

ThingsBoard, como *broker* que es, avisará al dispositivo cada vez que cambie este valor, ya que es un atributo compartido que debería actualizar. Si hacemos click, veremos que el LED ya cambia de color, pues la función de procesamiento de mensajes ya está hecha en la forma de `ESP01S_ProcessMQTT`, que se llama repetidamente desde el bucle de `main.c`:

Como es bastante larga, no la ponemos completa, pero consta de dos partes: en una primera, se extrae el paquete MQTT en crudo de la UART de entrada, y en una segunda, se parsea el paquete para extraer la información necesaria (básicamente, qué se ha actualizado):

```
1 // Paho Deserializer
2 unsigned char dup, retained;
3 int qos, payloadlen;
4 unsigned char* payload;
5 MQTTString receivedTopic;
6
7 if (MQTTDeserialize_publish(&dup, &qos, &retained, NULL, &receivedTopic,
8                              &payload, &payloadlen, packet, copy_len) == 1)
9 {
```

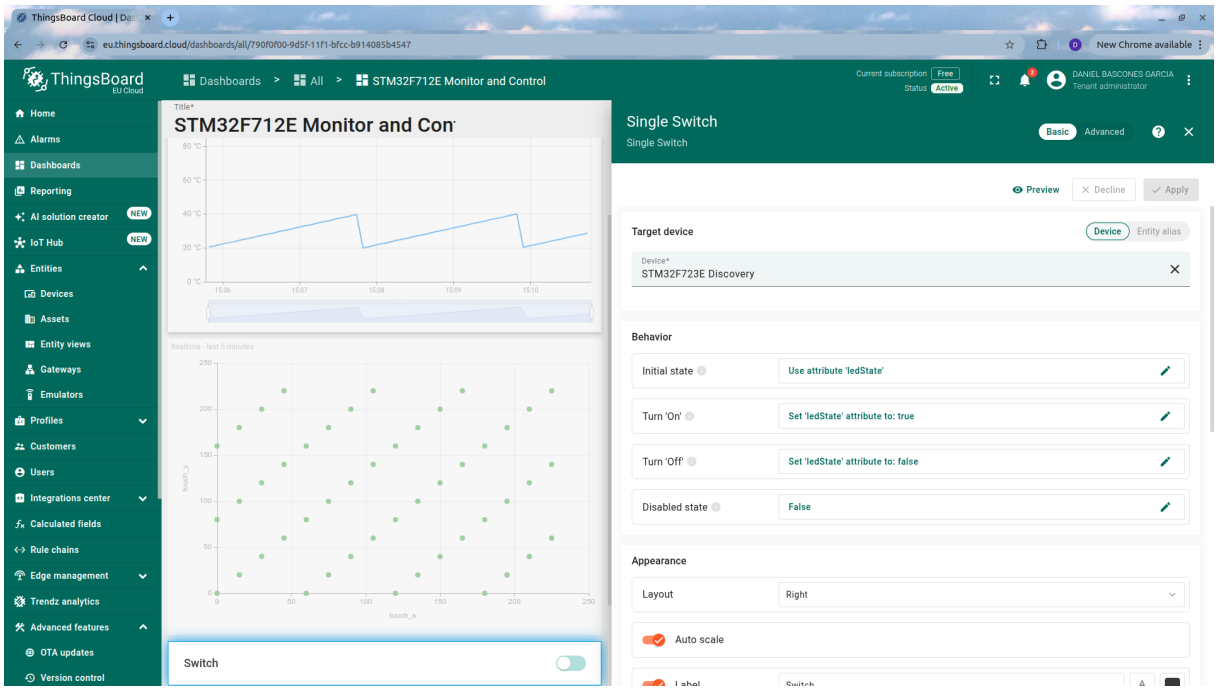


Figura 45: Atributo compartido para nuestro dispositivo

```
10      Debug_Printf("[MQTT RX] %s\r\n", receivedTopic.lenstring.data);
11
12      char json_str[128] = {0};
13      uint16_t str_len = (payloadlen < sizeof(json_str) - 1) ? payloadlen : sizeof
            (json_str) - 1;
14      memcpy(json_str, payload, str_len);
15
16      // LED command processing
17      if (strstr(json_str, "\"ledState\":true"))         GPIO_PIN_WRITE(GPIOA, 7,
            GPIO_STATE_ONE);
18      else if (strstr(json_str, "\"ledState\":false"))    GPIO_PIN_WRITE(GPIOA, 7,
            GPIO_STATE_ZERO);
19
20  }
```

Como se puede observar, si se recibe un mensaje conteniendo la cadena `"ledState":true` o `"ledState":false`, el LED cambiará su valor al recibido.

NOTA: En realidad, la forma canónica de hacer esto es utilizando la clase `MQTTClient` y registrando *callbacks* para atributos concretos recibidas, que llamarán a funciones que hagan la funcionalidad (valga la redundancia) asociada a dichos atributos. Sin embargo, la librería completa da algunos problemas de stack overflow. Si alguien se anima a optimizar esta práctica para que funcione (de manera consistente) con esa clase, es un buen proyecto final.

12.4. Para hacer más

Ahora que ya tienes la placa conectada con un servidor remoto, y puedes mandar datos y recibir comandos desde ella, puedes explorar las opciones de *ThingsBoard* en general o de los *dashboard* más concretamente, o hacer por ejemplo que los datos que se envían sean reales, en lugar de sintéticos.

13. Práctica 12: Actualización remota (OTA) utilizando *ThingsBoard*

13.1. Objetivos

- Entender la técnica de actualizaciones A/B.
- Preparar una aplicación para que sea actualizable.
- Comprender el funcionamiento de los bits de opción de arranque del procesador.

13.2. Fundamentos teóricos

Aunque los sistemas empotrados suelen tener funciones muy específicas y constantes a lo largo del tiempo, su complejidad a día de hoy hace que, en ocasiones, se cuele algún bug, o haga falta realizar algún cambio en los programas que los gobiernan.

Cuando desarrollamos desde nuestro ordenador, con un cable conectado, este proceso es muy sencillo: con un depurador (bien externo o integrado en la placa) podemos cambiar el código fácilmente. Sin embargo, al desplegar las soluciones en el mundo real, perdemos el acceso directo. Únicamente el propio controlador, a través de las interfaces que lo conecten al mundo exterior, es capaz de recibir información y, potencialmente, actualizarse.

Existen numerosas maneras de realizar estas actualizaciones remotas, aunque todas tienen algo en común: El código que actualiza la aplicación, no puede actualizarse a sí mismo, o de lo contrario se puede corromper mientras se actualiza. Para salvar esta barrera, existen numerosas técnicas, por ejemplo utilizando *bootloaders*, utilizando *launchers* con la única función de actualizar, haciendo que la propia app se actualice...

En nuestro caso vamos a utilizar una de las aproximaciones más robustas ante fallos, conocida como las actualizaciones A/B. En esta aproximación, se divide el espacio de memoria en dos zonas (A y B), de tal manera que la aplicación pueda residir en ambas. Cuando la app aranca desde A, en caso de encontrar una actualización la hará en B, y viceversa. Los bits de arranque del sistema se configuran, en cada caso, para arrancar la app más nueva. Añadiendo pasos de comprobación de integridad de los datos, se asegura, antes de cambiar el arranque, que la aplicación descargada no tiene fallos. Con todo esto, las probabilidades de *brickear* (o dejar inutilizable) el dispositivo se reducen enormemente. Este flujo general puede verse en Figura 46.

13.2.1. El problema de la compilación

Al compilar una aplicación para un sistema empotrado, normalmente se prepara con la idea de que siempre va a arrancar desde el mismo espacio en la memoria. De hecho, el código, en general no es de tipo PIC (position independent code), sino que ciertas direcciones de memoria están fijadas y no pueden cambiar. Al aplicar esta división A/B, esto sería un problema: la aplicación arranca desde dos direcciones de memoria diferentes. Es por ello que, en este tipo de aplicaciones, hay varias cosas a tener en cuenta:

- La aplicación debe ser consciente de cuál es su dirección de arranque (A o B).

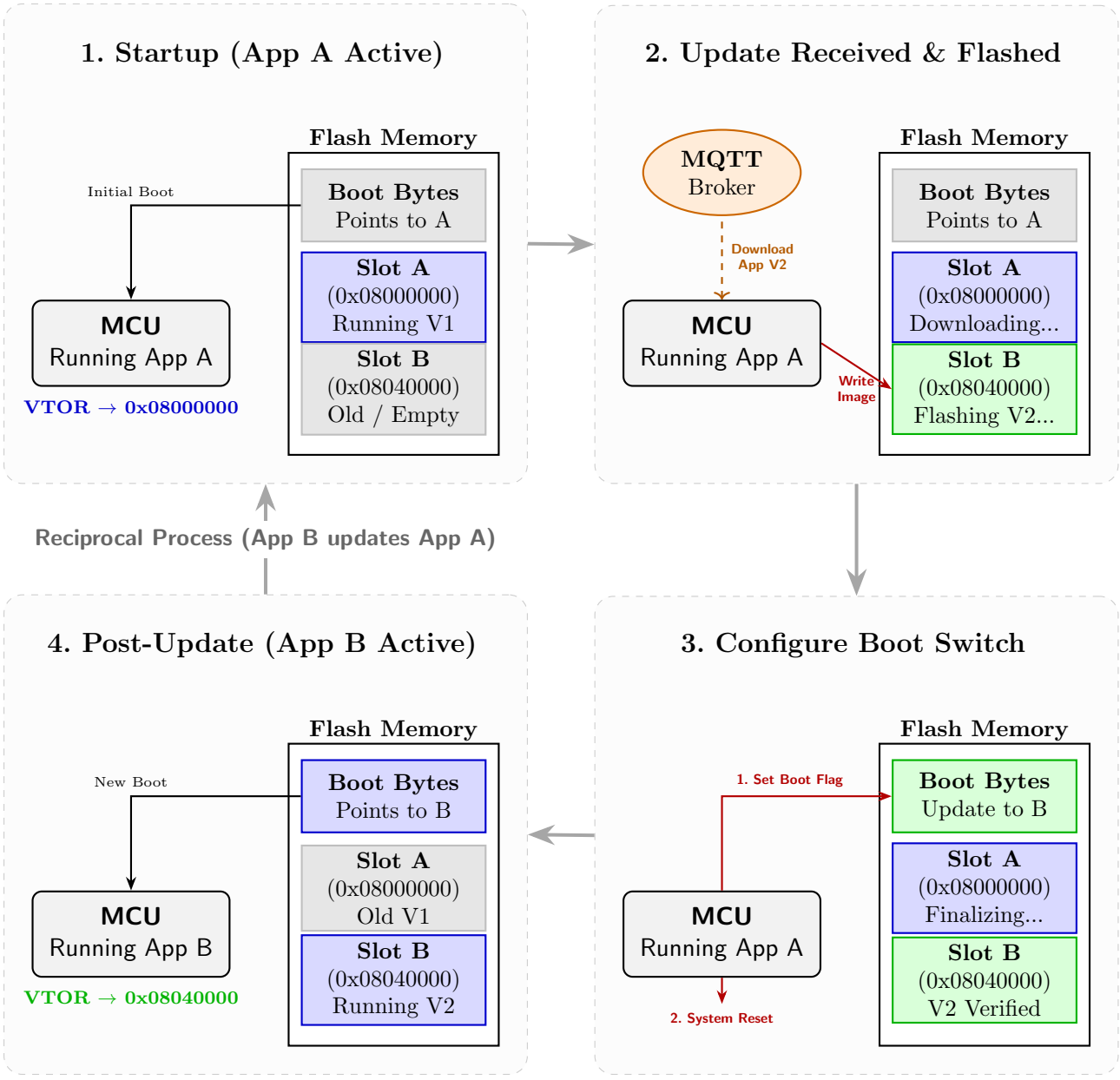


Figura 46: Diagrama de actualización A/B

- Se deben realizar dos compilaciones diferentes (A y B), cada una atacando una de las dos regiones de memoria.
- Cada compilación sólo puede residir en su zona de compilación predeterminada, con lo cual el sistema de actualizaciones deberá tenerlo en cuenta.

13.3. Desarrollo de la práctica

13.3.1. Preparación del código

El código ya está preparado para descargar la actualización OTA de *ThingsBoard* (configuraremos la interfaz web más adelante). Explicamos aquí con detalle los cambios respecto a la anterior práctica.

En la función `ESP01S_ProcessMQTT` tenemos los primeros cambios. En primer lugar, observamos que, además de comprobar en los mensajes de entrada las cadenas de actualización de `ledState`, ahora se llama a la función `OTA_ProcessJSON`. Está función (definida en los nuevos archivos `ota.h` y `ota.c`) se encarga de parsear el aviso de nuevo firmware disponible desde el servidor. Adicionalmente, se añade una comprobación directa para mensajes recibidos con el propio firmare (a través del *topic* **MQTT** "v2/fw/response").

```
if (MQTTDeserialize_publish(&dup, &qos, &retained, NULL, &receivedTopic,
                            &payload, &payloadlen, packet, copy_len) == 1)
{
    //Debug_Printf("[MQTT RX] %s\r\n", receivedTopic.lenstring.data);
    // 1. Handle Binary Firmware Response Chunk
    if (memcmp(receivedTopic.lenstring.data, "v2/fw/response", 14) == 0 &&
        OTA_getState() == OTA_STATE_DOWNLOADING) {
        OTA_WriteChunkAndProcess(payload, payloadlen);
    }
    // 2. Handle JSON Attributes (Runtime Notification or Startup Request)
    else {
        char json_str[128] = {0};
        uint16_t str_len = (payloadlen < sizeof(json_str) - 1) ? payloadlen :
            sizeof(json_str) - 1;
        memcpy(json_str, payload, str_len);

        OTA_ProcessJSON(json_str);

        // LED command processing
        if (strstr(json_str, "\"ledState\":true"))        GPIO_PIN_WRITE(GPIOA,
            7, GPIO_STATE_ONE);
        else if (strstr(json_str, "\"ledState\":false"))   GPIO_PIN_WRITE(GPIOA,
             7, GPIO_STATE_ZERO);
    }
}
```

Para estar en el modo de descarga de firmware, primero este debe ser solicitado. La función `OTA_ProcessJSON` se encarga de extraer la versión y tamaño del firmware anunciado desde el servidor. La versión desde el servidor tendrá la forma `1.0.0_A` o `1.0.0_B`, según el *slot* que ocupe. Esto sirve para identificar dos cosas: en primer lugar, si la versión es superior a la actual (definida en `ota.h` como `#define CURRENT_FW_VERSION "1.0.0"`), y si es la adecuada para el *slot* libre. En tal caso, se procede a borrar los sectores adecuados de la `FLASH`, y a solicitar los *chunks* o trozos de actualización uno tras otro (ya que, en general, las actualizaciones son demasiado grandes como para descargarlas de golpe).

```
void OTA_ProcessJSON(const char *json) {
    char *ver_ptr = strstr(json, "\"fw_version\":");
    char *size_ptr = strstr(json, "\"fw_size\":");

    if (ver_ptr && size_ptr && ota_ctx.state == OTA_STATE_IDLE) {
        char new_version[32] = {0};
        sscanf(ver_ptr, "\"fw_version\":\"%31[^\"]\"", new_version);
        uint32_t new_size = (uint32_t)atoi(size_ptr + 10);

        // Determine what binary suffix this device requires
        const char *required_suffix = ((uint32_t)&_app_start ==
            FLASH_SLOT_A_ADDR) ? "B" : "A";

        // Reject the update if the incoming version tag doesn't match the
            inactive slot
        if (strstr(new_version, required_suffix) == NULL) {
            Debug_Printf("[OTA] Rejected update %s! Device is running in Slot %c
                 and requires %s binary.\r\n",
                    new_version,
                    ((uint32_t)&_app_start == FLASH_SLOT_A_ADDR) ? 'A' : 'B'
                        ,
                    required_suffix);
            return;
        }

        // Verify if version differs from running app
        if (strcmp(new_version, CURRENT_FW_VERSION) != 0 && new_size > 0) {
            Debug_Printf("[OTA] New Version Found: %s (Size: %lu bytes)\r\n",
                new_version, new_size);

            ota_ctx.state = OTA_STATE_DOWNLOADING;
            ota_ctx.total_size = new_size;
            ota_ctx.bytes_downloaded = 0;
            ota_ctx.current_chunk = 0;
            strncpy(ota_ctx.target_version, new_version, sizeof(ota_ctx.
                target_version));

            OTA_EraseTargetSlot();
            OTA_RequestNextChunk();
        }
    }
}
```

Estos *chunks* se van procesando en `OTA_WriteChunkAndProcess`. Esta función va situando el puntero de escritura de la `FLASH` para ir poco a poco escribiendo el nuevo firmware, hasta completar la descarga.

```
void OTA_WriteChunkAndProcess(uint8_t *data, uint16_t length) {
    OTA_WriteChunk(ota_ctx.bytes_downloaded, data, length);
    ota_ctx.bytes_downloaded += length;
    ota_ctx.current_chunk++;

    Debug_Printf("[OTA RX] Chunk %u (%d bytes) -> Total: %lu / %lu\r\n",
                ota_ctx.current_chunk, length, ota_ctx.bytes_downloaded, ota_ctx
                    .total_size);

    if (ota_ctx.bytes_downloaded < ota_ctx.total_size) {
        OTA_RequestNextChunk();
    } else {
        Debug_Printf("[OTA SUCCESS] Download Finished! Rebooting...\r\n");
        ota_ctx.state = OTA_STATE_COMPLETE;
        OTA_SwapBootSlotAndReset();
```

```
    }
}
```

Cuando la descarga está completa, se lanza `OTA_SwapBootSlotAndReset`. Esta función tiene la importante función de calcular la nueva dirección de arranque y grabarla en los bits de opción (rm0431 3.4.1), para que la siguiente vez que el procesador comience, lo haga desde el nuevo *slot*. Los bits de opción de la FLASH permiten precisamente configurar ciertas opciones de arranque, que son tenidas en cuenta antes de que comience a ejecutar cualquier código. Así, no es la aplicación quién decide desde qué dirección se arranca, sino el propio procesador.

En nusetro procesador **STM32F723IE**, existen dos posibles direcciones de arranque rm0431 3.7.7 . El hecho de existir dos es para dar la opción de, al resetear, elegir cuál sirve de arranque con la pulsación (o no) de un botón. Normalmente, lo que se hace es colocar un *bootloader* en una de las direcciones iniciales, y nuestra aplicación en la otra. Esto hará que, si por algún motivo nuestra aplicación está mal escrita y no tiene comunicación con el depurador, se pueda recuperar la misma cargando el *bootloader*. En nuestra placa, al tener un depurador integrado, no tenemos que preocuparnos de ningún problema, ya que el depurador está conectado directamente por un puerto `SWI` (Serial Wire Interface) que puede tomar el control total del procesador aunque se quede "pillado".

En resumen, lo que haremos será ajustar la dirección `BOOT_ADD0` en OPTCR1 para apuntar a la aplicación que queramos arrancar. Como estos bits son delicados, no se puede escribir directamente en ellos, y hay que aplicar una secuencia de desbloqueo a través de los registros CR (rm0431 3.7.5), KEYR (rm0431 3.7.2), OPTCR (rm0431 3.7.6) y OPTKEYR (rm0431 3.7.3). Tras hacerlo, forzamos un volcado de la memoria mediante la primitiva `__DSB()`, y reseteamos el sistema desde el AIRCR del SCB (pm0253 4.3.5).

```
void OTA_SwapBootSlotAndReset(void)
{
    uint32_t target_addr = OTA_GetTargetAddress();
    // BOOT_ADD0 expects the flash address divided by 16KB (shifted right by 14
        bits)
    uint16_t boot_val = (uint16_t)(target_addr >> 14);

    Debug_Printf("[OTA] Swapping BOOT_ADD0 to 0x%08LU (val: 0x%04X) and
        resetting...\r\n",
                 target_addr, boot_val);
    Delay_ms(500);

    Flash_Unlock(); // Uses bsp.h helper
    Flash_Unlock_Opt();
    Flash_WaitNotBusy(); // Uses bsp.h helper

    //update target boot address
    Flash_Update_Opt(boot_val);

    Flash_WaitNotBusy();
    Flash_Lock_Opt();
    Flash_Lock(); // Uses bsp.h helper

    // Force system reset after Data Syncrhonization Barrier
    // (to flush pending mem stores)
    __DSB();
    SCB->AIRCR = (0x05FA0000U) | (1U << 2); // VECTKEY (0x05FA) + SYSRESETREQ (
        Bit 2)

    while (1); // Wait for hardware reset
}
```

Y... listo! Tras este proceso el procesador comenzará a ejecutar en el *slot* correspondiente. Al arrancar volverá a recibir la información del *firmware* disponible y, viendo que ya está actualizado, continuará su funcionamiento de manera normal. Podemos ver un resumen gráfico de todo el proceso en la Figura 47.

13.3.2. Preparación de la compilación

Ahora viene lo interesante. Tenemos el código ya casi listo, pero faltan dos cosas: Hacer consciente a la aplicación de quién es (A o B + versión), y compilarla de dos maneras diferentes, además de generar un archivo binario que poder subir a *ThingsBoard*. Vamos a empezar por lo segundo. Para compilarla de dos formas distintas, necesitamos dos *Linker Script* diferentes. Al *Linker Script* por defecto, debemos hacerle dos variantes:

```
//// ARCHIVO STM32F723IEKX_FLASH_A.ld ////

/* Memories definition */
MEMORY
{
    RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 256K
    FLASH    (rx)    : ORIGIN = 0x8000000,   LENGTH = 256K
}

/* Export FLASH start address to C code */
PROVIDE(_app_start = ORIGIN(FLASH));
```

```
//// ARCHIVO STM32F723IEKX_FLASH_B.ld ////

/* Memories definition */
MEMORY
{
    RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 256K
    FLASH    (rx)    : ORIGIN = 0x8040000,   LENGTH = 256K
}

/* Export FLASH start address to C code */
PROVIDE(_app_start = ORIGIN(FLASH));
```

Con esto, conseguimos que la base de la `FLASH` se sitúe en dos diferentes, o bien `0x08000000` o `0x08040000`. Es importante destacar que, al hacer esta partición, contamos con solo 256KB de espacio para nuestra app (al necesitar dos huecos) en lugar de los 512KB que teníamos antes. Otras aproximaciones a las actualizaciones, como la de incluir un pequeño actualizador en lugar de ser la aplicación completa la que actualiza, dejan más espacio libre. En cualquier caso, para nuestros propósitos y tamaños de librería, es suficiente (el binario final ocupa unos 30KB).

Además de ajustar la dirección inicial, exportamos un símbolo `_app_start` que podemos usar desde el código y nos indica la dirección de comienzo con la que hemos compilado el binario, para que desde ejecución seamos capaces de identificarla. Por otro lado, en `ota.h` tenemos las definiciones para poder hacer las actualizaciones:

```
#define CURRENT_FW_VERSION "1.0.0"
#define FLASH_SLOT_A_ADDR   0x08000000U
#define FLASH_SLOT_B_ADDR   0x08040000U
```

Así, en diferentes puntos del código, podemos identificar versión y *slot* para tomar diferentes decisiones:

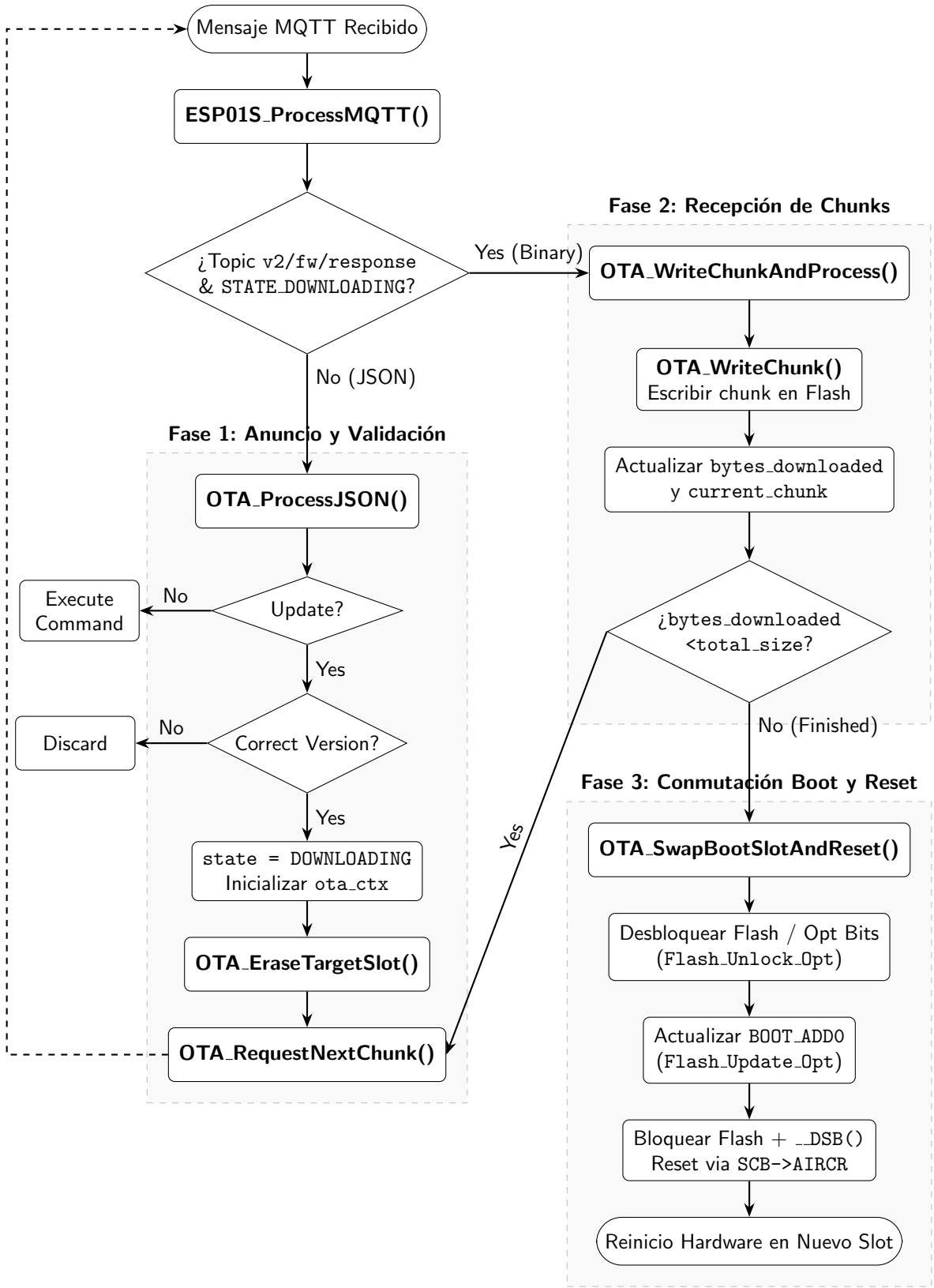


Figura 47: Diagrama de flujo de recepción OTA

```
//// en main.c
Debug_Printf("[BOOT] Running App v%s from Address: 0x%08X\r\n",
    CURRENT_FW_VERSION, (unsigned int)(uint32_t)&_app_start);

//// en ota.c -> OTA_ProcessJSON
// Determine what binary suffix this device requires
const char *required_suffix = ((uint32_t)&_app_start == FLASH_SLOT_A_ADDR) ? "B"
    : "A";

// Reject the update if the incoming version tag doesn't match the inactive slot
if (strstr(new_version, required_suffix) == NULL) {
    Debug_Printf("[OTA] Rejected update %s! Device is running in Slot %c and
        requires %s binary.\r\n",
                new_version,
                ((uint32_t)&_app_start == FLASH_SLOT_A_ADDR) ? 'A' : 'B',
                required_suffix);
    return;
}

//// en ota.c -> OTA_EraseTargetSector
uint32_t target_addr = OTA_GetTargetAddress();

if (target_addr == FLASH_SLOT_B_ADDR) {
    Debug_Printf("[OTA] Erasing Slot B (0x08040000)...\r\n");
    Flash_EraseSector(6);
    Flash_EraseSector(7);
} else {
    Debug_Printf("[OTA] Erasing Slot A (0x08000000)...\r\n");
    Flash_EraseSector(0);
    // ...

//// en ota.c -> OTA_SwapBootSlotAndReset
uint32_t target_addr = OTA_GetTargetAddress();
// BOOT_ADD0 expects the flash address divided by 16KB (shifted right by 14 bits
    )
uint16_t boot_val = (uint16_t)(target_addr >> 14);

Debug_Printf("[OTA] Swapping BOOT_ADD0 to 0x%08LU (val: 0x%04X) and resetting
    ...\r\n",
            target_addr, boot_val);
```

Para preparar estas dos compilaciones, debemos añadir los dos *linker script* al proyecto, y a continuación crear dos nuevas configuraciones de *build* dando click derecho al proyecto, `build configurations >manage >new`, tras lo que veremos un diálogo como el de la Figura 48. Hay que crear 2, una llamada "SLOT A" y otra "SLOT B".

A continuación, en las propiedades del proyecto, entramos a `C/C++ Build >Settings >MCU/MPU GCC Linker`. Tenemos que seleccionar arriba la configuración adecuada (A o B) y cambiar el nombre del *linker script* al que acaba en `_A` o `_B` que hemos creado anteriormente. Esto se muestra en la Figura 49

También debemos activar la opción de generar binario desde la misma configuración del proyecto, en `C/C++ Build >Settings >MCU/MPU Settings` (Figura 50).

Ahora ya podemos compilar los dos binarios diferentes. Para ello basta darle al martillo de arriba, en la configuración "SLOT A" o "SLOT B". Veremos aparecer en nuestro proyecto dos carpetas, con sendos nombres, que contendrán los archivos compilados de cada una de las versiones. Para depurar, tendremos que ir a `Debug >Debug Configurations` y elegir el binario

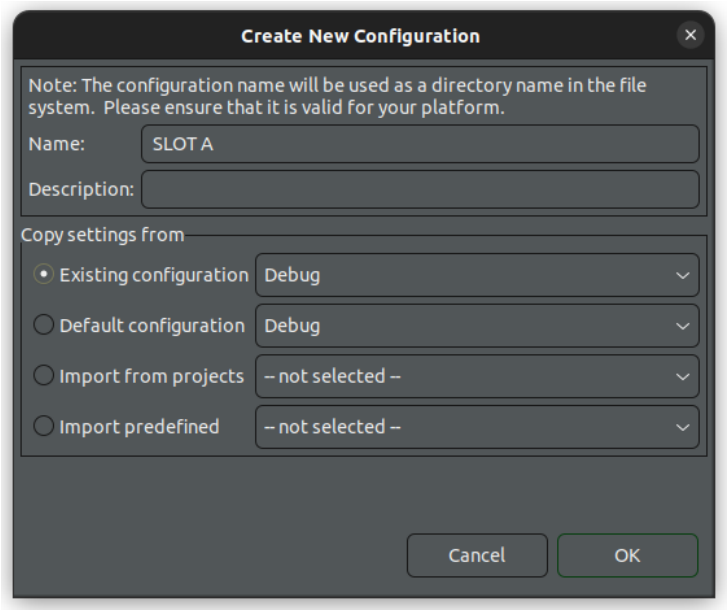


Figura 48: Creando una nueva configuración de *build*

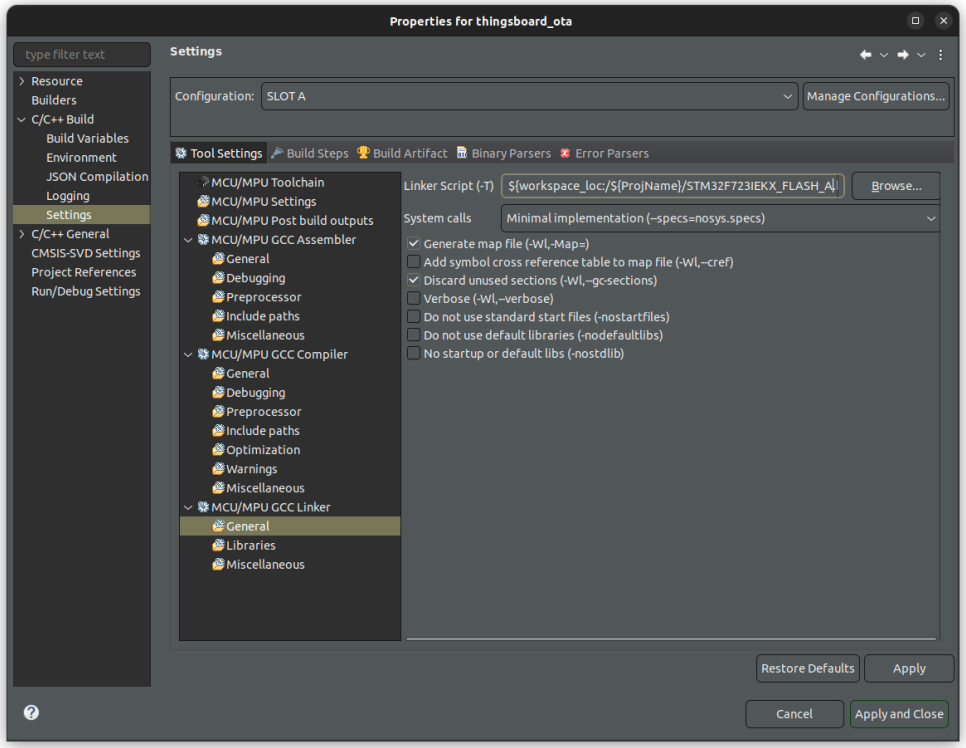


Figura 49: Configuración de linker script

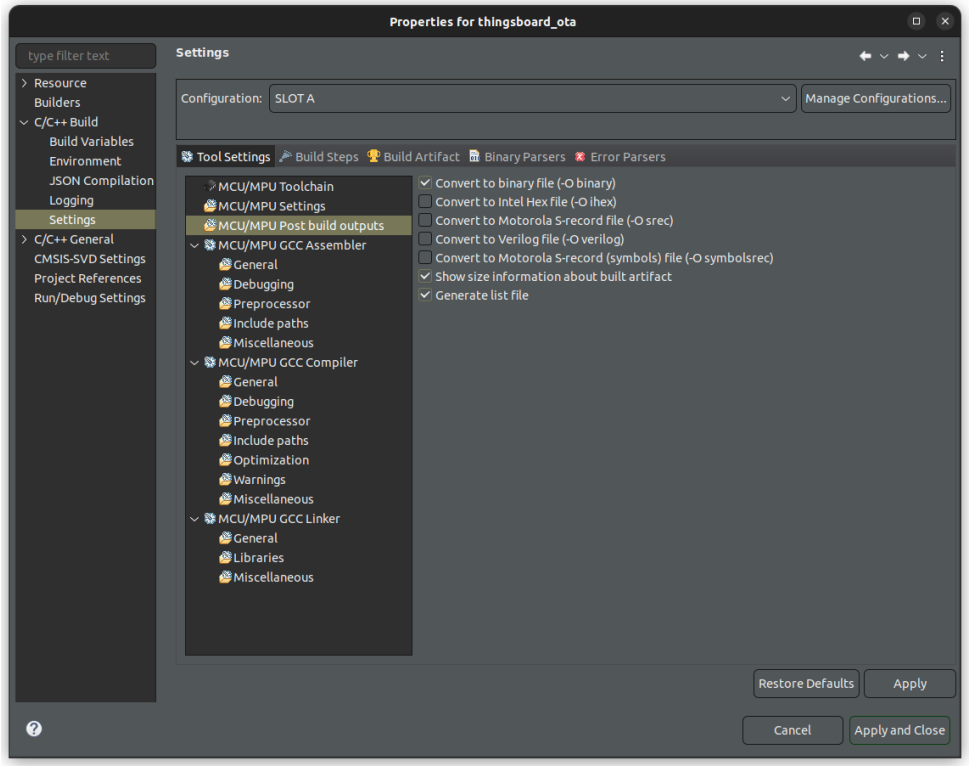


Figura 50: Configuración para generar archivo binario

que queremos depurar como muestra la Figura 51.

Al arrancar, deberíamos ver por terminal el siguiente mensaje, indicando que todo ha ido bien:

```
[BOOT] Running App v1.0.0 from Address: 0x08000000
[ESP] Testing ESP01S AT Communication...
...
```

13.3.3. Configuración en *Thingsboard*

Tras tener preparadas las dos compilaciones, podemos subirlas a *ThingsBoard*. Para ello, hay que ir a la sección `Advanced Features >OTA Updates` y añadir dos nuevos paquetes de Firmware. Tendremos que indicar un nombre y versión, además de asociarlo con un perfil (por defecto en nuestro caso), y subir el binario. Todo el proceso se muestra en la Figura 52

Podemos subir diferentes versiones, y seleccionar en cada momento la que nuestro dispositivo debería tener. En este caso, lo hacemos desde el apartado de `Entities >Devices`, donde podremos seleccionar, para nuestro dispositivo (ver Figura 53), el firmware que queramos y será anunciado mediante **MQTT**.

Por ejemplo, podemos seleccionar el firmware "A" (`Devices >Details >Assigned Firmware`) y arrancar la aplicación, desde STM32CubeIDE, también en el slot A. Si abrimos una terminal para ver los mensajes de depuración, deberíamos observar lo siguiente:

```
[BOOT] Running App v1.0.0 from Address: 0x08000000
[ESP] Testing ESP01S AT Communication...
[ESP] Disabling Echo...
```

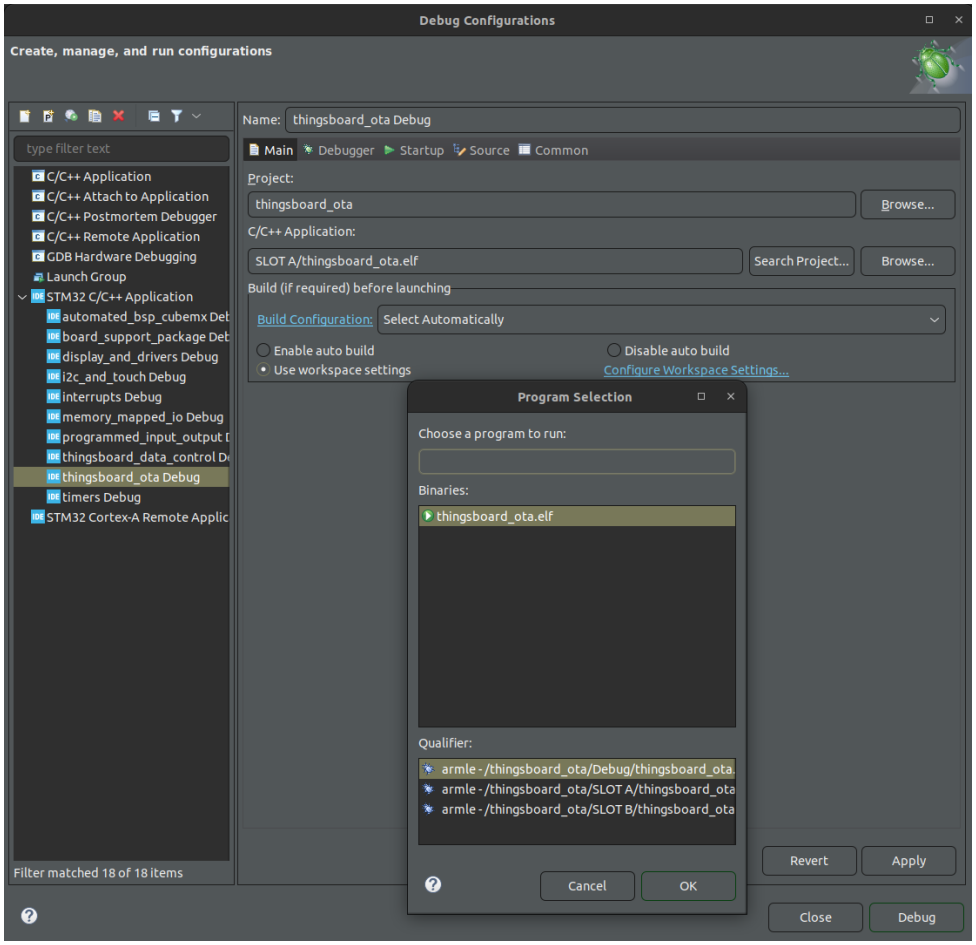


Figura 51: Configuración para cambiar el binario de depuración

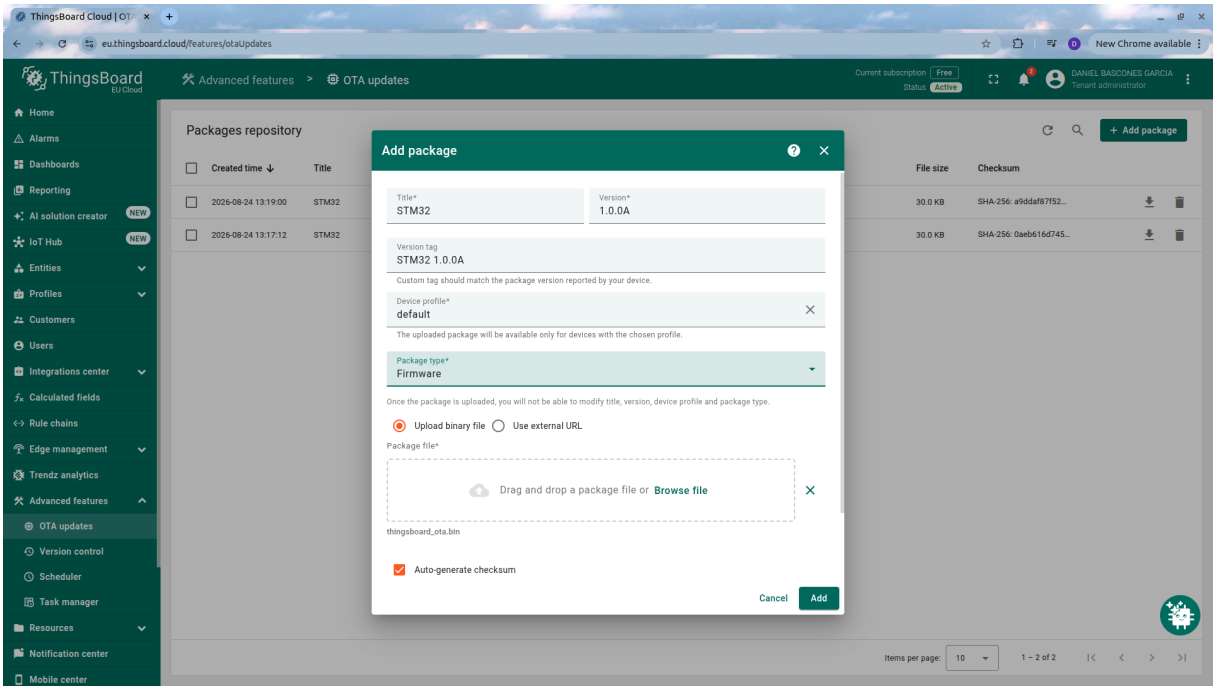


Figura 52: Subida de Firmware para OTA a *ThingsBoard*

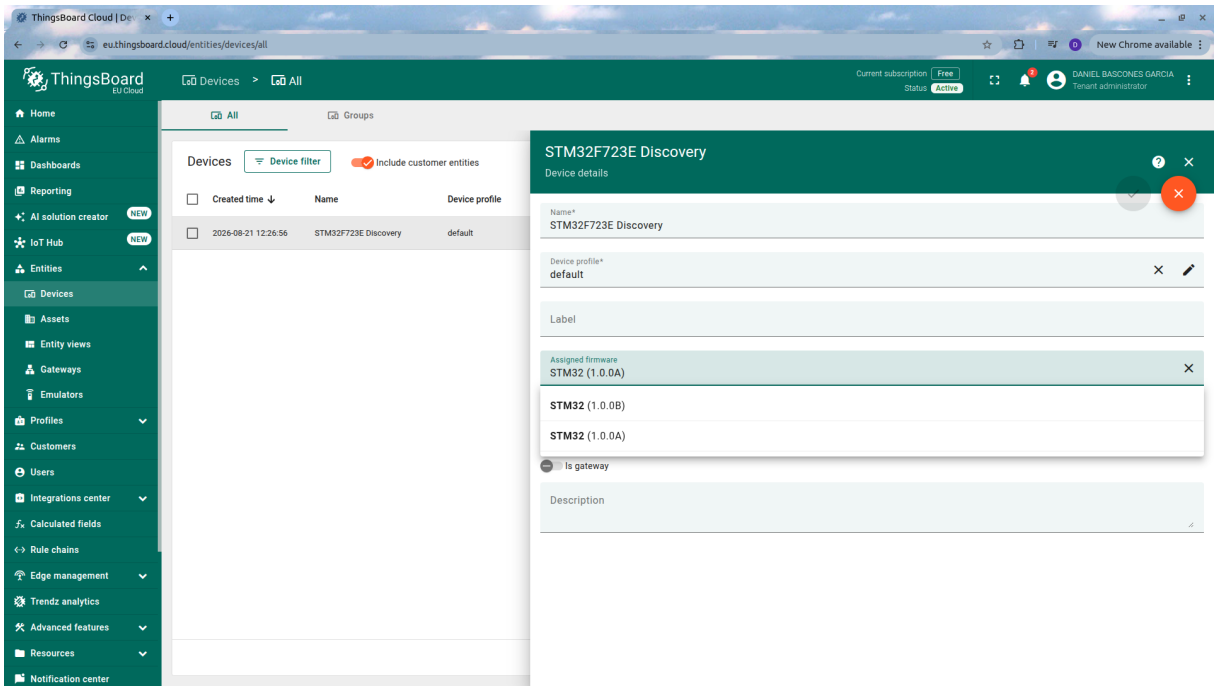


Figura 53: Subida de Firmware para OTA a *ThingsBoard*

```
[ESP] Setting Station Mode...
[ESP] Disabling MUX (Single Conn Mode)...
[ESP] Enabling Transparent Passthrough Mode...
[ESP] Connecting to SSID: guayfi...
[MQTT] Connecting to eu.thingsboard.cloud:1883...
[MQTT] Entering passthrough streaming...
[MQTT RX] CONNACK 0x00: Connection Accepted!
[MQTT RX] SUBACK Received! Granted QoS: 0
[MQTT] Connected & Subscribed successfully!
[SYSTEM] Ready! Streaming over MQTT...
[MQTT TX] {"temp":20.50,"counter":1,"touch_x":15,"touch_y":20}
[OTA] Rejected update 1.0.0A! Device is running in Slot A and requires B binary.
```

Podemos ver que, como el firmware seleccionado es para el mismo Slot, la aplicación lo rechaza y no actualiza nada. Si lo hiciera, colocaría un código que asume su inicio en `0x08000000` en la dirección de B `0x08040000`, donde cualquier salto absoluto resultaría en un fallo de ejecución. Si a continuación cambiamos el firmware objetivo desde *ThingsBoard* para nuestro dispositivo (`Devices >Details >Assigned Firmware`), la aplicación recibirá la actualización y, ahora sí, descargará la actualización:

```
[OTA] New Version Found: 1.0.0B (Size: 18628 bytes)
[OTA] Erasing Slot B (0x08040000)...
[OTA] Target Slot Erased!
[OTA RX] Chunk 1 (512 bytes) -> Total: 512 / 18628
// ..............  Descarga de actualización
[OTA RX] Chunk 37 (196 bytes) -> Total: 18628 / 18628
// ..............  Cambio de arranque
[OTA SUCCESS] Download Finished! Rebooting...
[OTA] Swapping BOOT_ADD0 to 0x0000000U (val: 0x8040000) and resetting...
// ..............  Nuevo arranque desde SLOT B
[BOOT] Running App v1.0.0 from Address: 0x08040000
// ..............  App en marcha! Ahora rechaza la aplicación nueva.
[OTA] Rejected update 1.0.0B! Device is running in Slot B and requires A binary.
[MQTT TX] {"temp":21.00,"counter":2,"touch_x":30,"touch_y":40}
```

Si volvemos a cambiar el Firmware por el A, la aplicación volverá a actualizarse gracias a los avisos que envía *ThingsBoard* con cada cambio de versión.

13.4. Para hacer más

En nuestro caso, estamos descargando "a ciegas" la aplicación, grabándola en la `FLASH`, y dándole el control con mucha confianza. Normalmente, al descargar una imagen binaria, también se envía un CRC o código de comprobación. Así, la aplicación puede calcular el CRC de los datos descargados, comprobarlo contra el CRC enviado por el servidor, y cambiar los bits de arranque únicamente si coinciden, asegurando la integridad de la app. Investiga cómo podría hacerse esto y qué cambios harían falta tanto en *ThingsBoard* como en nuestro código fuente.

14. Práctica 13: Creación automática de BSP

14.1. Objetivos

- Aprender a utilizar herramientas avanzadas de desarrollo de sistemas empotrados como STMCubeMX.
- Comprender la planificación de pines al utilizar un microcontrolador.

14.2. Fundamentos teóricos

Hasta ahora, hemos estado desarrollando nuestros propios paquetes BSP para trabajar con nuestra placa. Así, hemos conseguido facilitar el uso mediante funciones auxiliares que nos permitan realizar configuraciones o acciones. Para saber exactamente cómo trabajar con la placa, hemos tenido que consultar `rm0431`, `um2140`, `ds11853`, `pm0253`, `an2606`, ...

Normalmente (y así es en el caso de ST, desarrolladora del **STM32F723IE**), se proporcionan paquetes software que ya contienen toda la funcionalidad preprogramada, para evitarnos tener que hacerlo manualmente. Estos paquetes son desarrollados por los ingenieros de "software", basándose en los manuales proporcionados por los ingenieros de "hardware".

En ocasiones, se proporciona un único paquete genérico con toda la funcionalidad, y es responsabilidad del usuario final decidir qué y cómo lo utiliza. Aunque esto tiene la ventaja de empaquetar toda la funcionalidad en un único conjunto de librerías, tiene el inconveniente de agrandar el código al incluir mucha funcionalidad que quizá no utilicemos. En el caso de ST, proporcionan una herramienta para crear un BSP a medida con únicamente los elementos que queramos incluir. Es esta funcionalidad la que vemos a continuación.

14.3. Desarrollo de la práctica

14.3.1. Creación del BSP con STMCubeMX

Lo primero de todo es abrir la aplicación STMCubeMX (Figura 54). Tenemos varias funciones:

- Ver los proyectos ya creados.
- Empezar un proyecto eligiendo un procesador, una placa, o un ejemplo preparado por ST.
- Actualizar la aplicación e incluir paquetes software nuevos.

Lo primero que haremos, si es que no está ya instalado, es descargar el paquete STM32F7 desde el apartado "Install or remove embedded software packages" (Figura 55).

A continuación, debemos elegir el componente base de nuestro proyecto. Tenemos dos opciones:

- **MCU**: Empieza el proyecto partiendo de un procesador sin ninguna conexión activa. En este modo, debemos asignar, a cada "pata" o pin del procesador una función concreta, planificando cómo lo soldaremos a una placa de prototipado a medida.

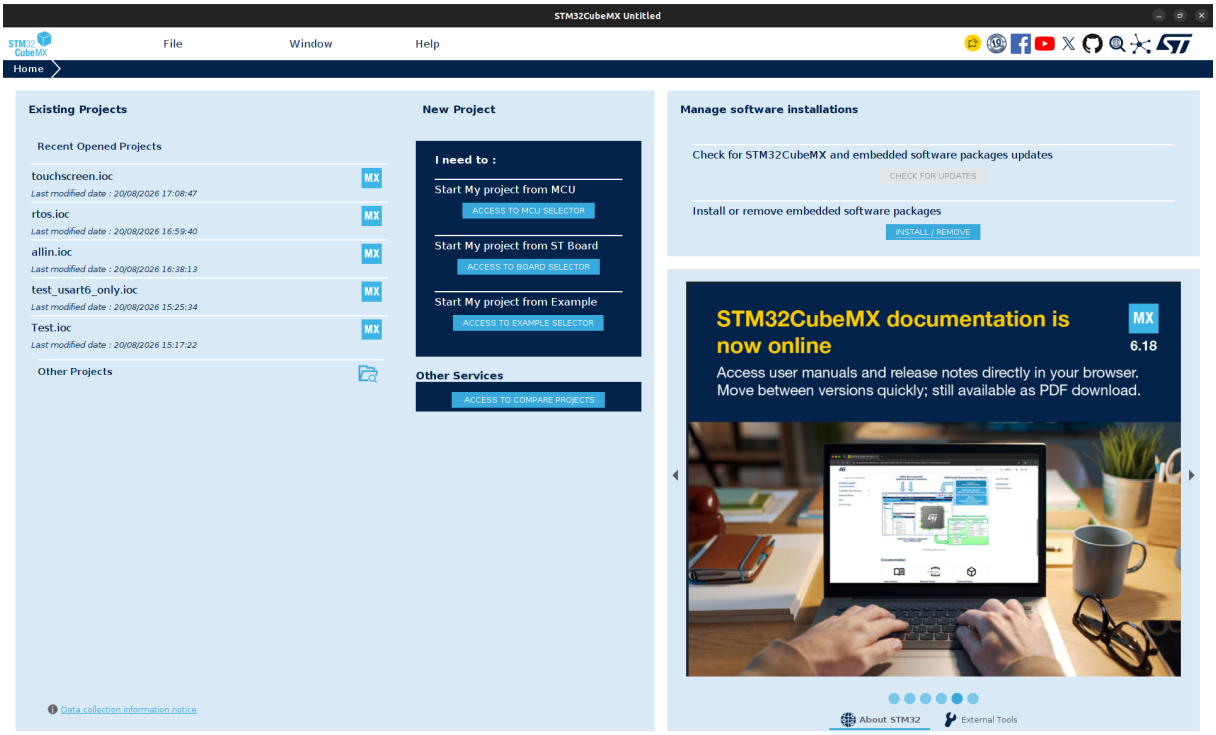


Figura 54: Pantalla inicial de CubeMX

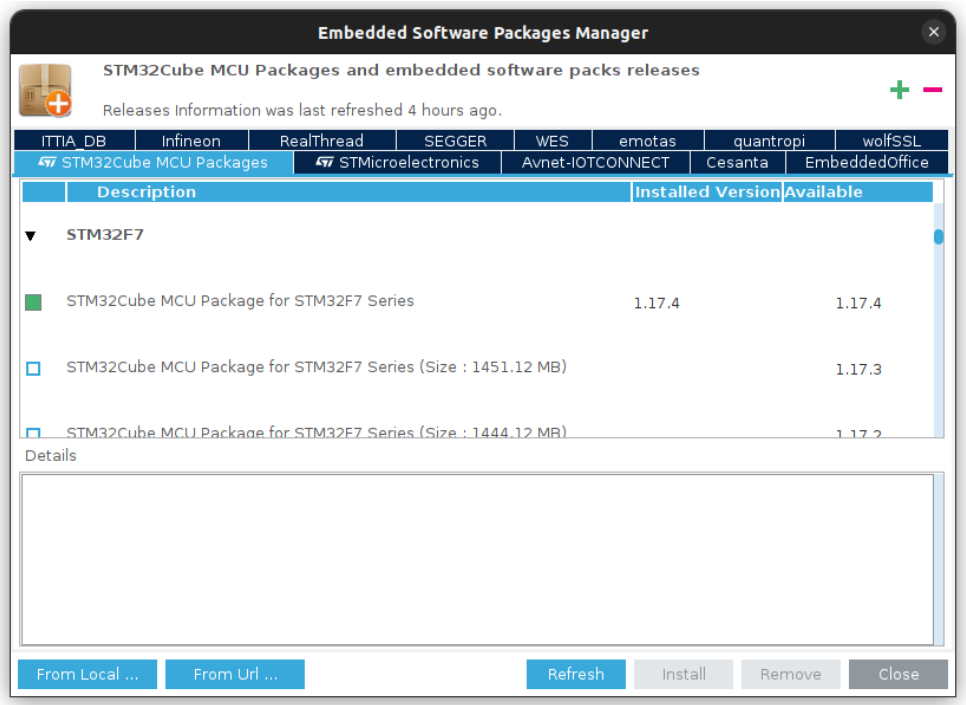


Figura 55: Instalación del paquete STM32F7

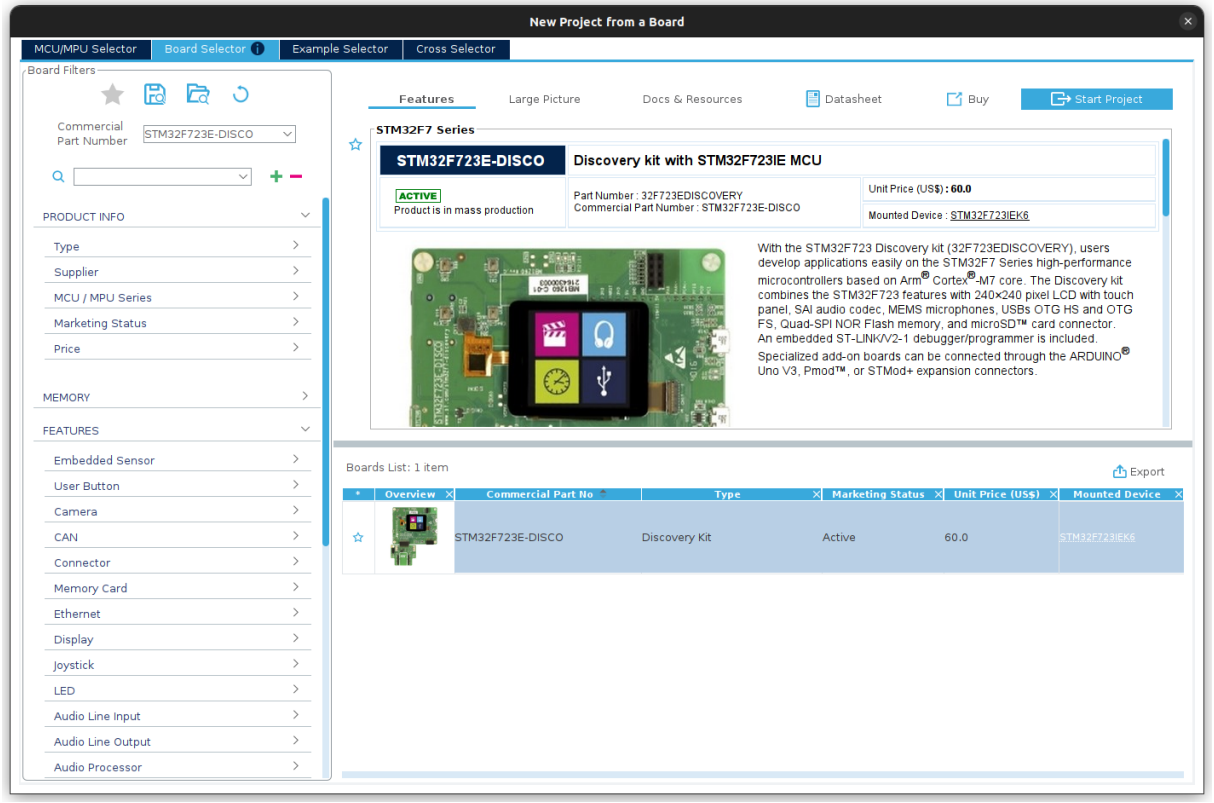


Figura 56: Selector de placas con la STM32F723E-DISCO seleccionada

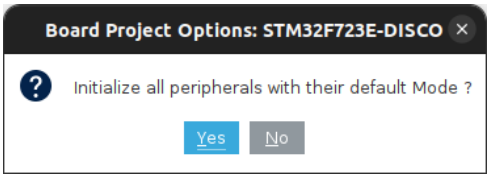


Figura 57: Diálogo de inicialización: seleccionar NO

- **BOARD**: En este caso, partimos de un procesador que ya se encuentra dentro de una placa, y sus pines tienen conexiones concretas establecidas (LEDs, Botones, Pantallas...). En este modo, simplemente deberemos elegir qué funciones activamos, pero no tendremos la dificultad de tener que planificar qué hace cada pin. Utilizaremos este segundo modo eligiendo nuestra placa (723E discovery) (Figura 56).

Al crear el proyecto, nos pedirá una serie de opciones. En concreto, si queremos iniciar **TODOS** los periféricos (Figura 57, diremos que no), y si queremos preconfigurar la MPU (Figura 58, diremos que sí).

Si quisiéramos habilitar TODA la funcionalidad de la placa, deberíamos decir que sí al primer diálogo, pero esto nos hará tener un paquete demasiado grande para las pruebas que queremos hacer.

Una vez creado el proyecto, podemos ver la planificación inicial en la Figura 59:

- Los pines color crema corresponden a alimentación (diferentes niveles de voltaje y tierra).
- Los pines color gris están deshabilitados (no conectados a ningún lado en la placa, por

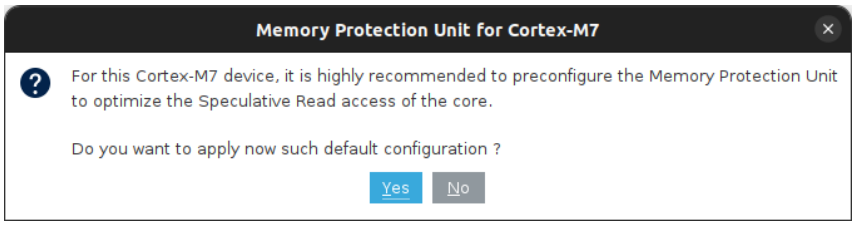


Figura 58: Diálogo de configuración de memoria: seleccionar SÍ

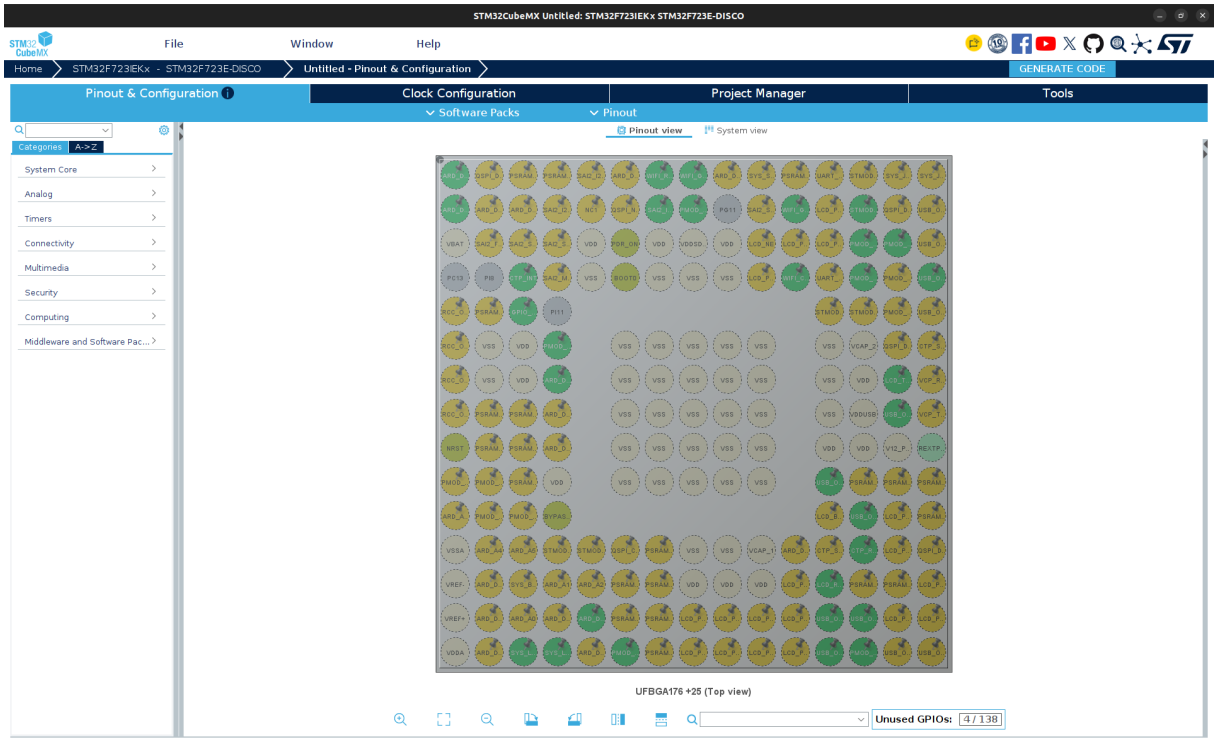


Figura 59: Disposición inicial del procesador en cubemx. Se observan todos los pines y sus funciones

tanto no tienen uso).

- Los pines color verde caqui y aguamarina (solo solo unos pocos) corresponden a funciones de reseteo y la secuencia de arranque.
- Los pines amarillos corresponden a funciones desactivadas pero disponibles (se pueden ir activando como veremos a continuación).
- Los pines verdes corresponden a funciones ya activas y que utilizará nuestra aplicación.

Hay que tener en cuenta que esta configuración de pines es la que se generará automáticamente por la herramienta, lo cual no quita que nosotros podamos cambiarla "al vuelo" dentro de nuestra aplicación, mediante acceso a todos los registros correspondientes de `GPIO`.

Para esta práctica vamos a hacer un par de cambios. El primero es cambiar la función del pin `PA0` para que dispare una interrupción. Deberemos pinchar en él (es el de la posición (3,3) empezando por la esquina inferior izquierda), y seleccionar `GPIO_EXTI0` para habilitar su función de interrupción externa (Figura 60). Finalmente tendremos también que seleccionar la generación de interrupción desde `NVIC` en el menú correspondiente (Figura 61).

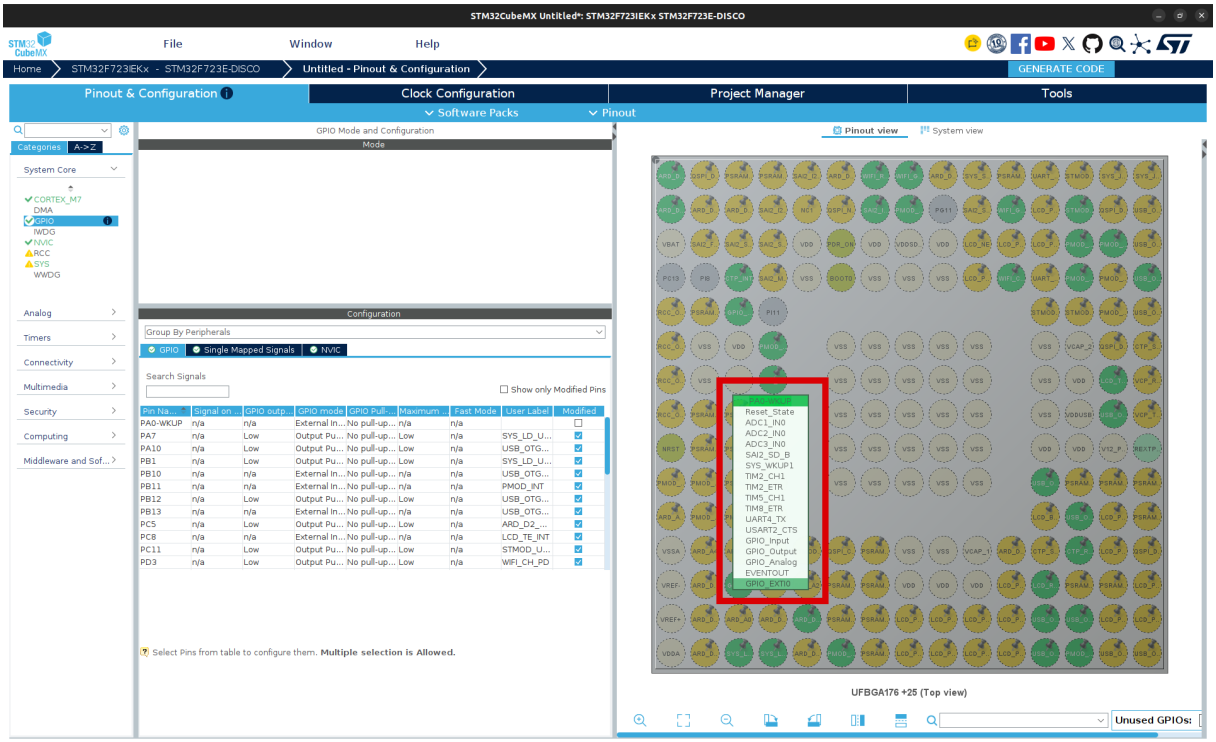


Figura 60: Cambio de función de un pin

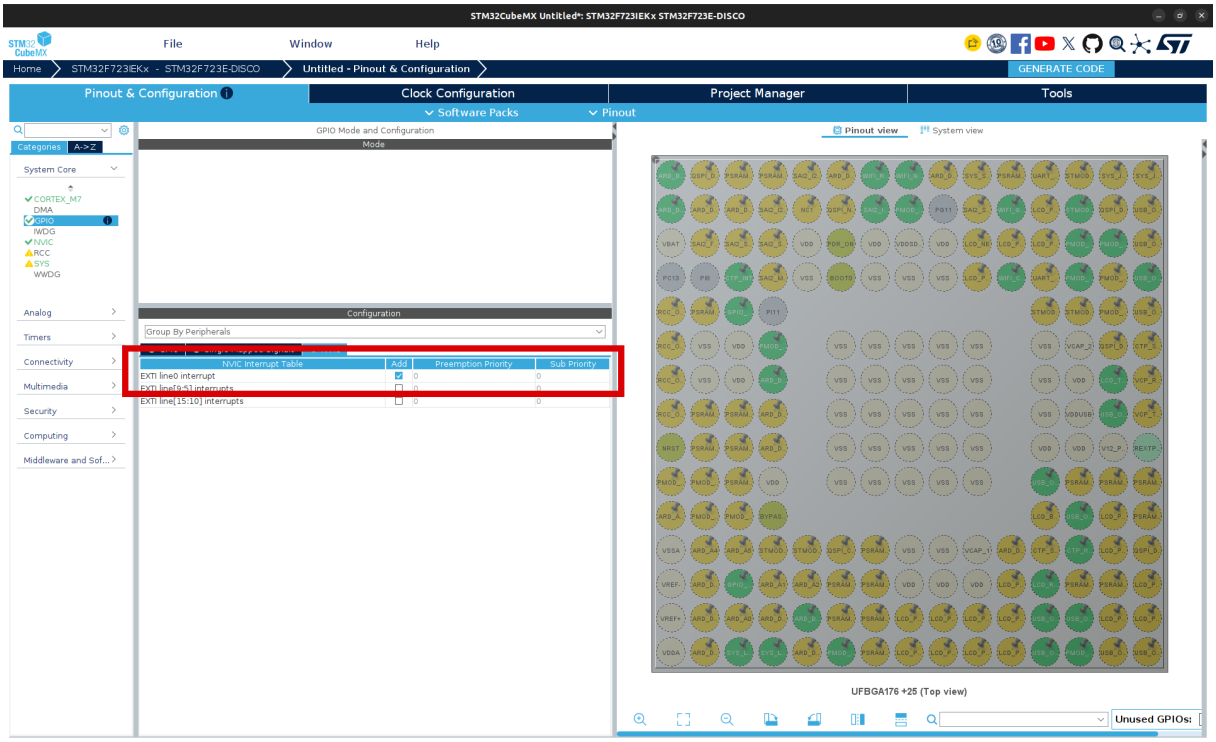


Figura 61: Activación de interrupciones para el pin de `GPIO` A0

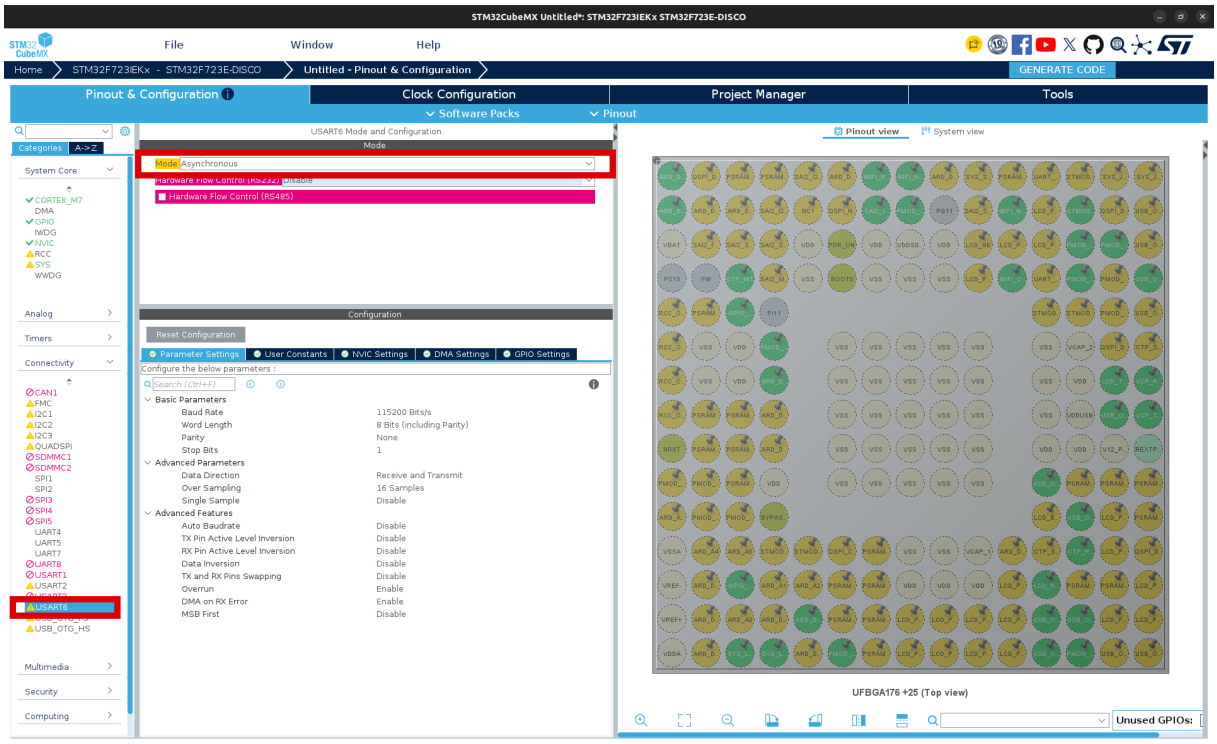


Figura 62: Activación del periférico `USART6`

Además de tratar con los pines, podemos realizar la configuración de todos los periféricos que contiene el procesador. En la barra lateral izquierda, los veremos categorizados. Vamos a activar la `UART6` para poder enviar mensajes al ordenador a través de la conexión serie. Para ello, tenemos que seleccionar en la lista `Connectivity >USART6` y luego activarla poniéndola en modo `Asyncrhonous` (Figura 62).

Veremos que numerosos periféricos y opciones aparecen en color rojo/rosa. Esto es porque la herramienta detecta configuraciones incompatibles que no nos deja cambiar. (Imaginemos por ejemplo que la UART y el bus I2C comparten pines, sólo podríamos tener habilitado uno). Si queremos ver por qué una opción está deshabilitada, basta con dejar el ratón encima, y nos dirá que otras funcionalidades entran en conflicto.

Otra parte bastante potente y que podemos configurar aquí es la gestión del árbol de relojes (Figura 63). El árbol de relojes es la estructura del procesador que toma la oscilación inicial de un cristal o bucle de inversores, y genera frecuencias concretas para todos los elementos internos del procesador. En otras prácticas no hemos estado modificando su configuración, y el reloj por defecto era de 16MHz. Aquí podemos ver que, utilizando diversos multiplicadores y divisores de frecuencia, se consigue un reloj objetivo de 216MHz. Su configuración se generará automáticamente para la placa. Podemos probar a cambiar funciones para ver cómo afecta a todos los relojes del árbol.

Hay que tener en cuenta que, para la configuración de periféricos con reloj propio (como la UART), debemos saber con exactitud la frecuencia de referencia que reciben (por ejemplo, para ajustar valores de contadores), así que un cambio en este punto puede suponer un cambio en nuestro código.

Finalmente, estamos listos para generar el proyecto con nuestro BSP customizado, que incluirá la configuración ya hecha de los periféricos seleccionados y del árbol de reloj. Debemos

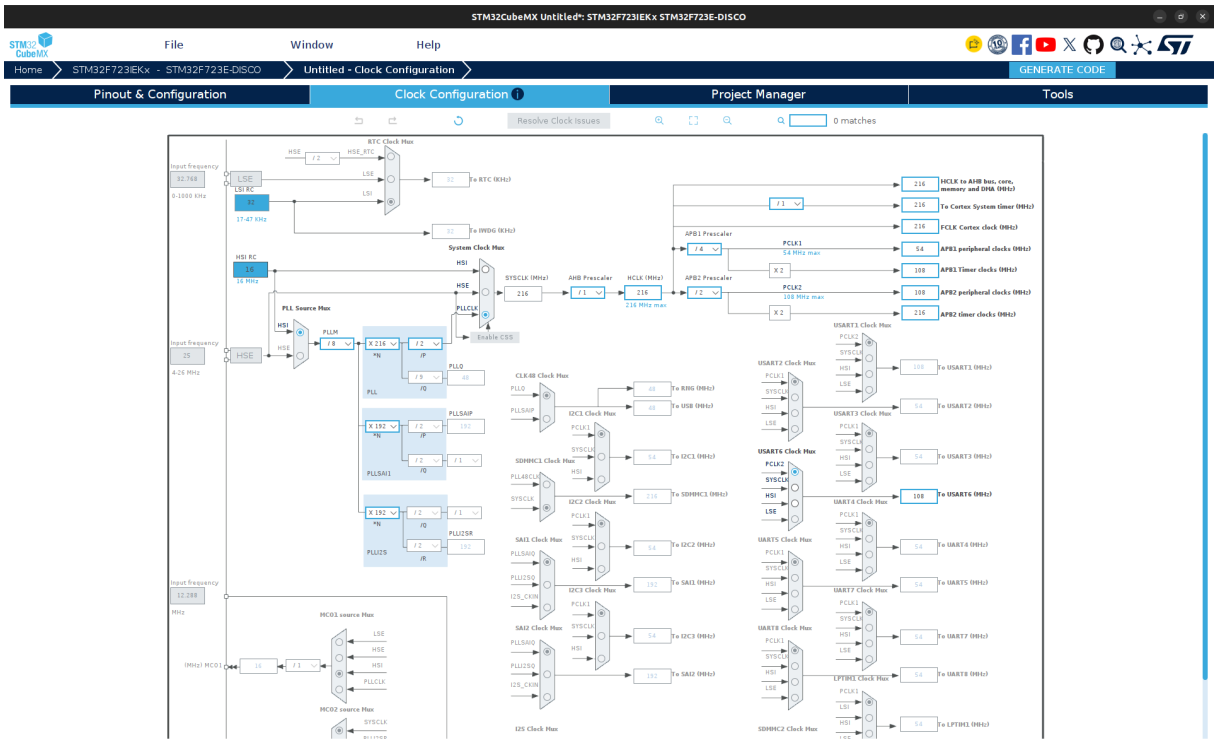


Figura 63: Configuración del reloj

dar un nombre y carpeta al proyecto, además de indicar que lo abriremos con STMCubeIDE (Figura 64). Una vez generado, nos dará la opción directamente de abrir el proyecto (Figura 65).

14.3.2. Uso del BSP desde STMCubeIDE

Ahora ya volvemos al familiar STMCubeIDE. Deberíamos haber visto un diálogo de importación correcta como el de la Figura 66.

En la Figura 67 vemos la estructura de ficheros, así como el código automáticamente generado para la función `main`, que inicializa la MPU, reloj, y periféricos activados (En nuestro caso, algunas inicializaciones genéricas, además de los `GPIO` y `USART6`).

En la Figura 68 podemos ver el código de inicialización del reloj. Una cosa interesante a observar en este BSP, es que suele tener estructuras de configuración complejas (`RCC_OscInitStruct`) que luego ya se vuelcan en registros internos (en este caso del `RCC`) de golpe en las funciones internas.

Para las interrupciones, por ejemplo, vemos que configura el `NVIC` en la Figura 69, y genera el manejador de interrupción en Figura 70. En este caso, por darnos mayor abstracción, ya gestiona automáticamente el borrado de la interrupción pendiente, y nos ofrece una función `__weak` que podemos sobreescribir en nuestro fichero `main.c` para capturar la interrupción.

También vemos cómo ha realizado la configuración de los pines del LED (ya configurado por defecto) (Figura 71) y del botón (Figura 72). También configura multitud de pines extra (para todos los periféricos) aunque no les esté dando uso, ya que no los desactivamos en la vista de *pinout*.

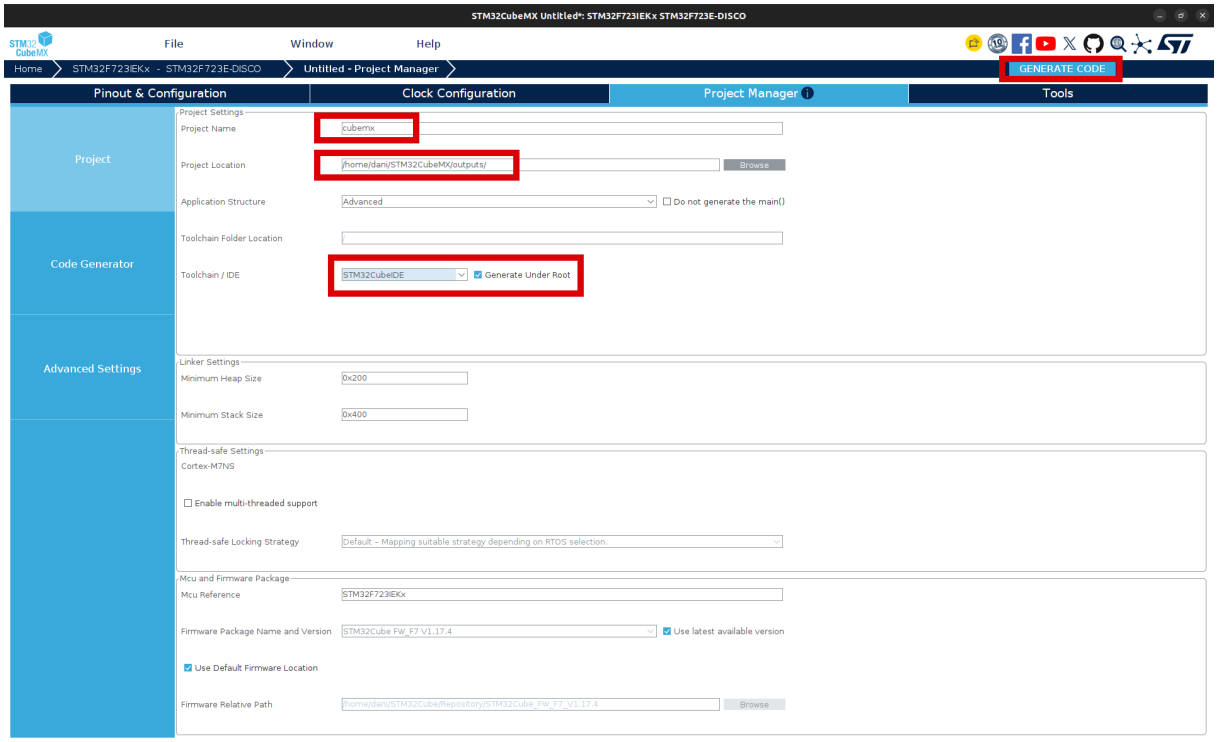


Figura 64: Creación del proyecto con BSP para STMCubeIDE

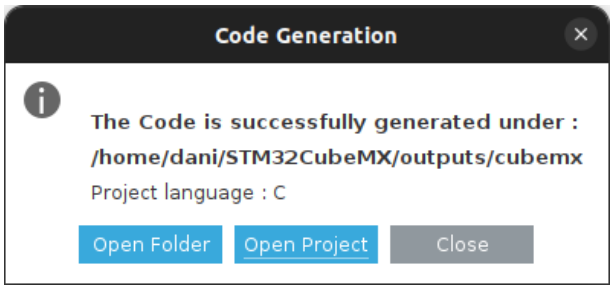


Figura 65: Diálogo de finalización. Seleccionar “Open Project”

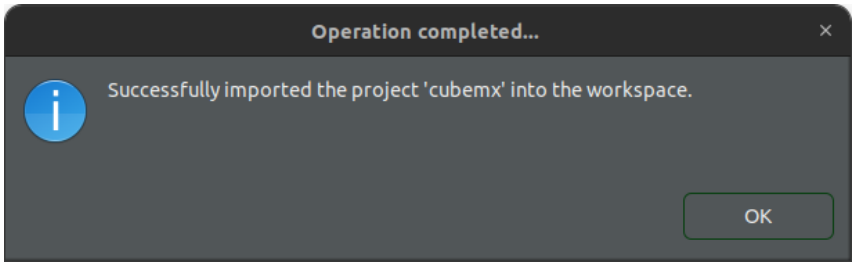


Figura 66: Diálogo de importación exitosa en STMCubeIDE

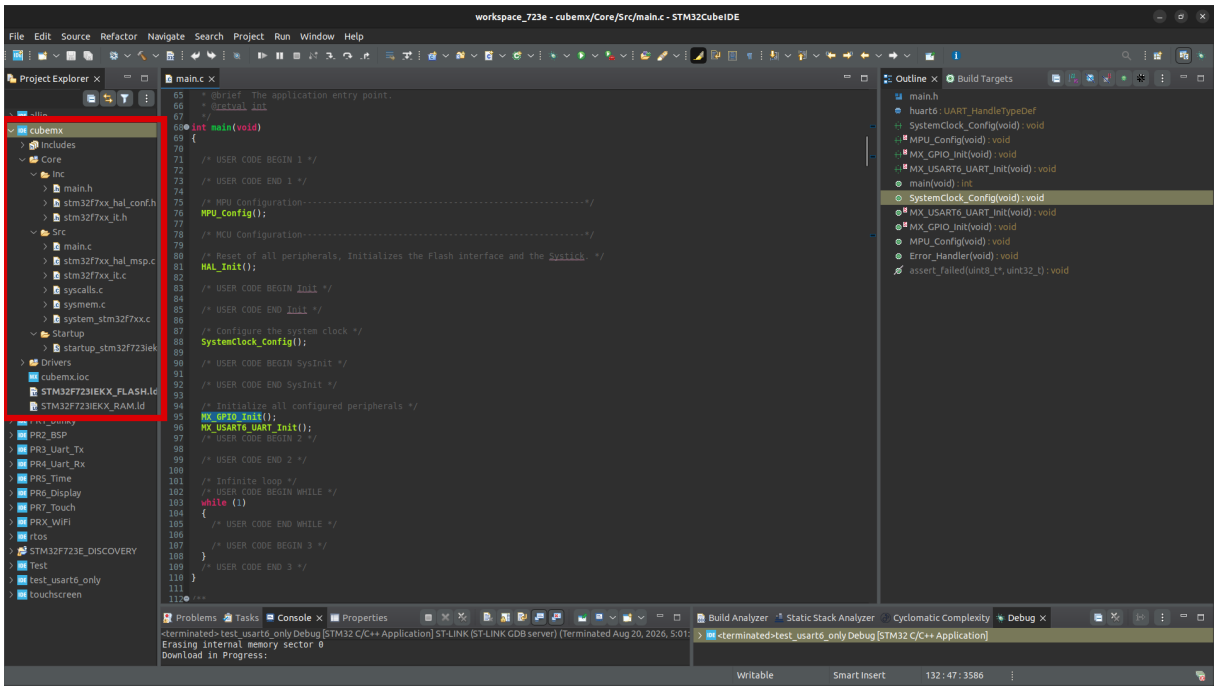


Figura 67: Estructura del proyecto creado y código main

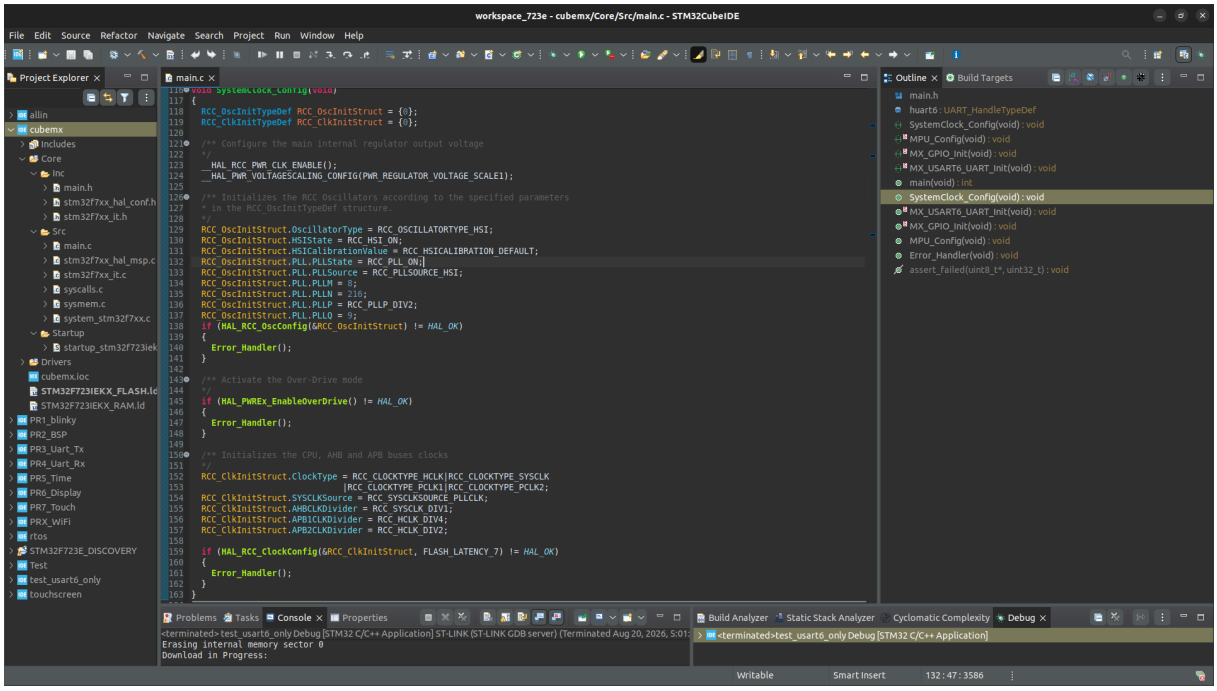


Figura 68: Código creado para inicializar el reloj

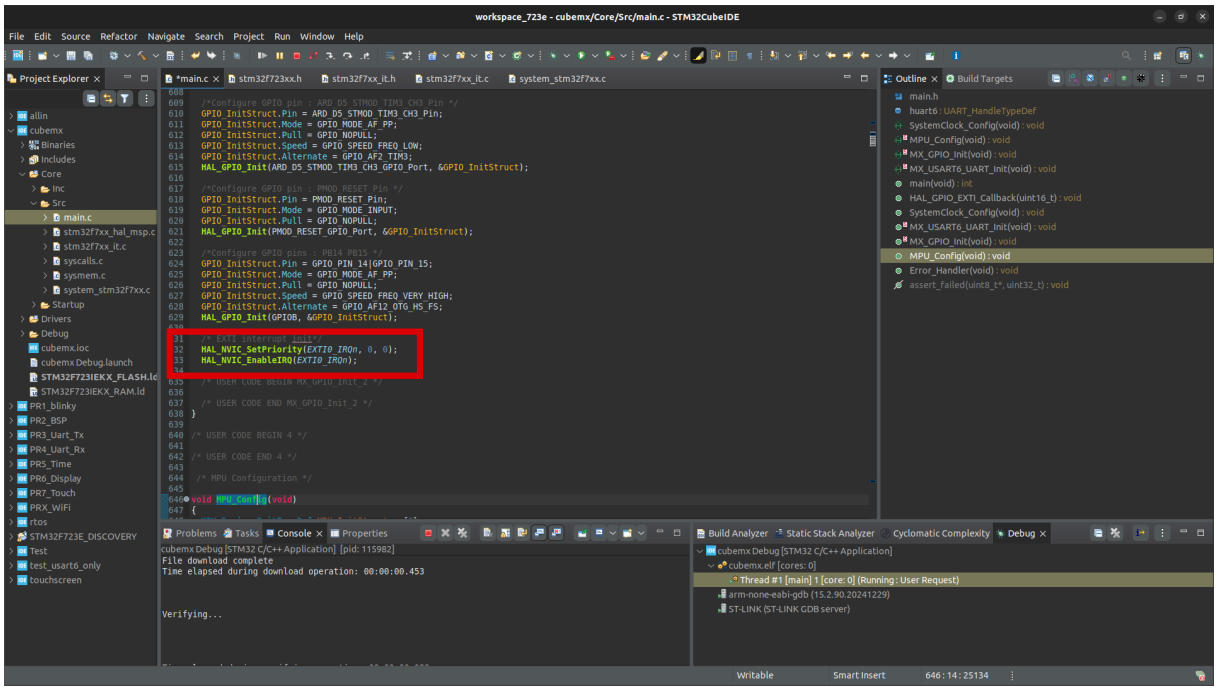


Figura 69: Código generado para inicializar las interrupciones

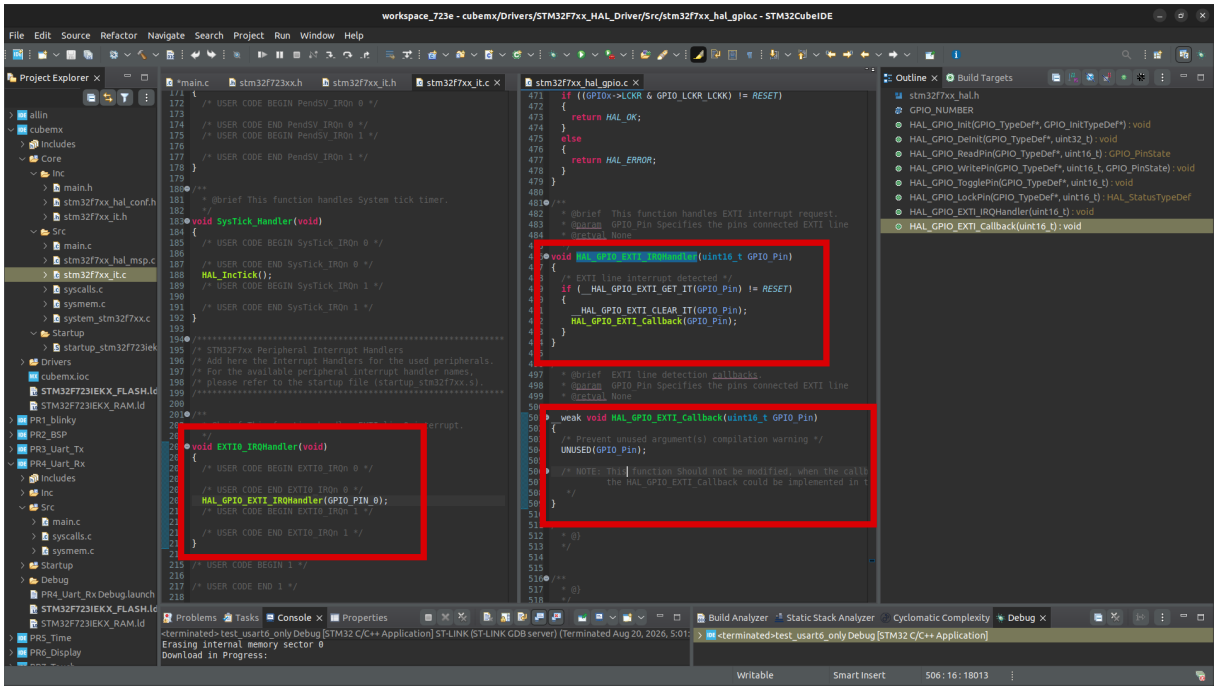


Figura 70: Estructura de las interrupciones, se ofrece mediante un *callback* al usuario

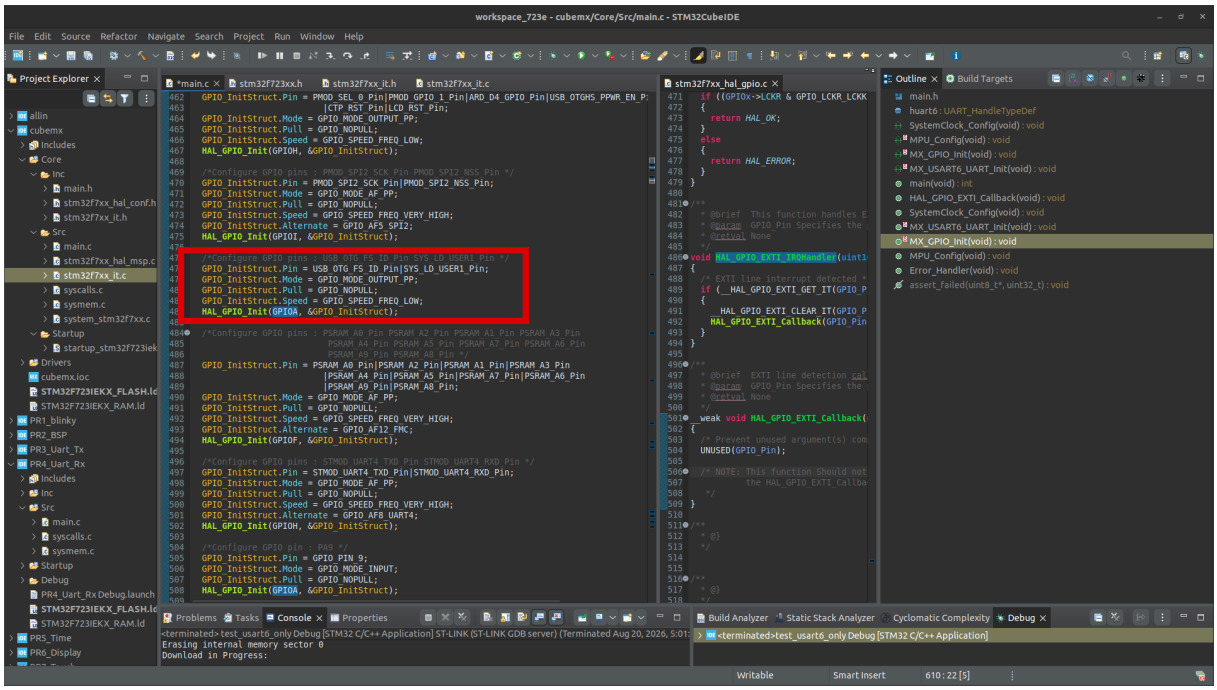


Figura 71: Configuración automática del pin para el LED

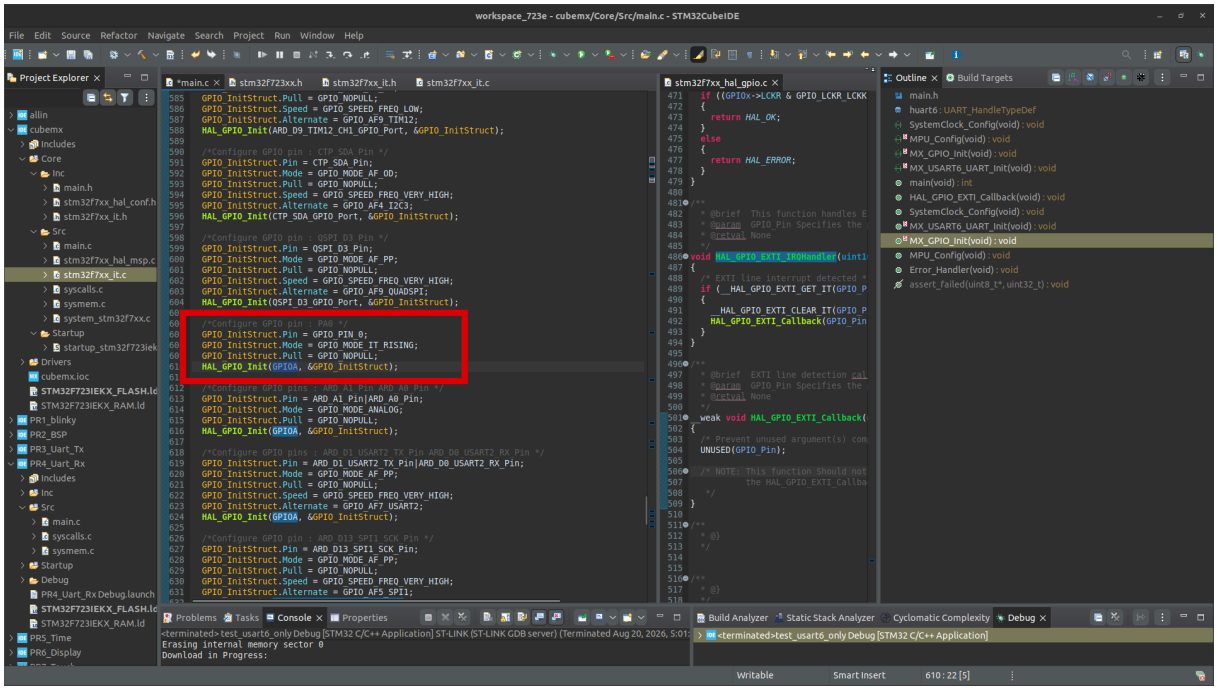


Figura 72: Configuración automática del pin para el Botón

Finalmente, nos queda únicamente hacer uso del paquete que se nos ha dado. Haremos un código sencillo donde, cada cierto tiempo, parpadee un LED. Además, al pulsar el botón, enviaremos por la `UART` un texto.

Si antes esto hubiera supuesto ver la documentación de diferentes periféricos, definir sus estructuras, crear sus funciones auxiliares... ahora podemos directamente hacer uso de las funciones predefinidas en el BSP generado, y nos queda un código muy refinado y corto.

```
int main(void)
{
    /* MPU Configuration--------------------------------------------------------
       */
    MPU_Config();

    /* MCU Configuration--------------------------------------------------------
       */
    /* Reset of all peripherals, Initializes the Flash interface and the Systick
       . */
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART6_UART_Init();
    /* Infinite loop */
    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_7);
        HAL_Delay(1000);
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)
        HAL_UART_Transmit(&huart6, (uint8_t *) "Probando\r\n", 10, 1000);
}
```

14.4. Para hacer más

Investiga cómo podrías añadirle aún más funcionalidad al proyecto incluyendo los ficheros de BSP que están en el repositorio `https://github.com/STMicroelectronics/STM32Cubef7` (`Drivers/BSP` y `Utilities`). Aquí se encuentran drivers de alto nivel para la pantalla o panel táctil, entre otros, así como funciones auxiliares varias.

NOTA: La adición de estos paquetes es manual, y no se incluyen automáticamente desde STMCubeMX. Tendrás que copiar las carpetas que vayas a utilizar a tu proyecto, y añadir sus rutas a la compilación (`botón derecho sobre tu proyecto >properties >C/C++ General >Paths and Symbols`)

15. Práctica 14: Proyecto final - Práctica Libre

15.1. Objetivos

- Realizar un proyecto para la placa **32F723EDISCOVERY**de manera autónoma, utilizando varias de sus capacidades de manera simultánea.
- Demostrar el conocimiento sobre la asignatura mediante una presentación individual del proyecto.

15.2. Requisitos de la práctica

Es necesario para presentar la práctica cumplir con unos requisitos mínimos de desarrollo. Es recomendable comunicar previo al desarrollo la idea al profesor, para asegurarse de que se pueden cumplir los requisitos. El cumplimiento estricto de los mismos no garantiza la nota, pues ésta vendrá de una presentación individual del mismo.

Los requisitos mínimos para la práctica son los siguientes:

- Utilizar la pantalla, tanto para mostrar información, como para recoger pulsaciones a través del panel táctil.
- Utilizar interrupciones para al menos dos periféricos o líneas diferentes.
- Contar con persistencia para guardar algún tipo de configuración de usuario.
- Evitar las esperas activas salvo, como mucho, para el bucle `main`.

Opcionalmente, se pueden integrar:

- Un sistema operativo de tiempo real o algún sistema de multitarea.
- Conectividad Wifi para control o actualizaciones remotas.
- Nuevos periféricos externos conectados mediante los pines de expansión.

Para la presentación, se espera que el estudiante sea capaz de demostrar el conocimiento sobre la totalidad del código desarrollado, entendiendo cómo funciona todo el sistema, tanto a nivel software como hardware. Además de presentar el proyecto, se hará una defensa donde el profesor preguntará, código en mano, por detalles de la práctica. Se deberá además preparar un pequeño documento (1-2 páginas) de manual de uso de la práctica, indicando cómo funciona el sistema, y añadiendo un diagrama de flujo que explique las interacciones entre los diversos elementos (periféricos, interrupciones, código, etc). Se añade un ejemplo del mismo en el Apartado 15.4.

15.3. Ideas de desarrollo para la práctica final

1. **Pulsera de actividad**: El proyecto propuesto consiste en desarrollar una pulsera de actividad capaz de monitorizar la actividad física. Deberá contar con al menos las siguientes características:

- Visualizar iconos y textos en un LCD.
- Usar el touchpad para la interacción con el usuario.
- Leer señales biológicas (emuladas) y calcular indicadores de actividad física.

2. **Marco digital**: El proyecto propuesto consiste en desarrollar un marco digital que visualice un carrusel de fotos con efectos de transición. Deberá contar con al menos las siguientes características:

   - Implementar las transiciones entre fotos mediante DMA (**ojo! nuevo periférico**).
   - Usar el touchpad para interactuar con el usuario.
   - Visualizar imágenes y/o textos en un LCD.

3. **Ciclocomputador**: El proyecto propuesto consiste en desarrollar un ciclocomputador capaz de calcular y visualizar distintos parámetros de la carrera (velocidad, distancia, cronómetro, etc) en base al tren variable de pulsos que generaría un sensor de revolución ubicado en la rueda (que simulamos con el botón). Deberá contar con al menos las siguientes características:

   - Usar el panel táctil y LCD como mecanismos básicos para la interacción con el usuario.
   - Visualizar menús e información de los datos generados

4. **Detector de posicionamiento**: El proyecto propuesto consiste en desarrollar una aplicación para identificar el origen de un sonido. Para ello se utilizará la matriz de cuatro micrófonos disponible en la placa. Se deberá capturar el sonido de todos ellos, y aplicando algún algoritmo de detección de patrones en el audio, averiguar cuándo éste llega a cada uno de ellos, para posicionar el origen. Deberá tener las siguientes características:

   - Utilizar el panel táctil como interfaz del programa.
   - Utilizar el panel LCD para visualizar el origen del sonido, mediante una línea desde el centro de la pantalla hacia donde se esté ubicando el mismo.

5. **Tu idea**: El proyecto propuesto consiste en desarrollar tu idea, con tus funcionalidades deseadas, para hacer algo que te guste y motive. Piensa en algo que se pueda aplicar en tus hobbies, que siempre hayas querido hacer, que te parezca un reto, o que sea lo suficientemente sencillo para cumplir los requisitos sin herniarte a programar.

15.4. Ejemplo de documento final explicativo sobre la práctica

15.4.1. Manual de Usuario

El proyecto **Detector de Posicionamiento por Sonido** permite identificar en tiempo real la dirección angular de procedencia de un evento sonoro, empleando los micrófonos de la placa. El sistema procesa los niveles y retardos de señal entre los diferentes canales de audio, dibujando dinámicamente un vector de posición en la pantalla LCD. La configuración de los diversos parámetros, así como la interacción con el sistema, se hace mediante la interfaz táctil de la pantalla.

Guía de funcionamiento para el usuario:

1. **Arranque e Inicialización:** Al encender la placa, el sistema carga los ajustes previos de la memoria Flash persistente (comprobando si existen). En caso contrario, establece valores por defecto.
2. **Visualización en Pantalla LCD:** En el centro de la pantalla (240 × 240 píxeles) se representa un indicador circular (brújula/radar). Al detectarse un sonido que supere el umbral configurado, se traza una línea desde el centro, orientada hacia la dirección estimada de la fuente sonora. Para el cálculo estimado de la dirección, se utiliza un algoritmo que compara la señal de las diferentes fuentes, detectando el desfase entre ellas y, en función del mismo, calculando la dirección de proveniencia.
3. **Control de la aplicacion:** En la parte inferior del LCD se disponen dos botones virtuales: `[-]` y `[+]`. El usuario puede pulsar sobre ellos para modificar el umbral de disparo de sonido en función del ruido ambiente. Al pulsar el botón físico del usuario, el nivel de sensibilidad actual se almacena de forma permanente en la Flash, confirmándose la operación mediante un parpadeo del LED verde.

15.4.2. Arquitectura del sistema

- **Interfaz Gráfica y Táctil (LCD + CTP):** La pantalla LCD ST7789H2 es controlada mediante el periférico paralelo `FMC` utilizando los drivers proporcionados de ejemplo en la práctica 7. El panel táctil capacitivo FT3x67 se gestiona a través del bus `I2C3`, utilizando igualmente el driver proporcionado en la práctica 8.
- **Uso de Interrupciones:**
  1. **Interrupción EXTI0**: Recoge las pulsaciones del botón de usuario.
  2. **Interrupción EXTI9_5**: Captura las pulsaciones mediante la línea de interrupción del panel táctil.
  3. **Interrupción de Temporización**: Dispara periódicamente el muestreo de los canales de sonido/audio y desencadena el cálculo del algoritmo de posicionamiento angular.
- **Persistencia de Datos en Memoria FLASH**: Se hace uso de las funciones desarrolladas en la práctica de persistencia, para desbloquear y escribir en el Sector 7 de la memoria Flash (`0x08060000`). En dicha dirección se guarda la información de sensibilidad mediante una estructura de datos diseñada a medida. Así se mantienen los valores aunque se apague la placa.

- **Ausencia de Esperas Activas**: El bucle principal de la aplicación (`main`) entra continuamente en modo de bajo consumo mediante la instrucción ensamblador `__asm volatile ("wfi")`. El procesador permanece dormido hasta que una interrupción lo despierta para procesar un evento.

15.4.3. Diagrama de Flujo

A continuación se muestra el diagrama de flujo del sistema, ilustrando la interacción entre las rutinas de servicio de interrupción (ISR), las periferias hardware, el hilo principal y la memoria persistente.

